

ForeSyte

Project Team

Inam Ullah Shaikh	22i-0857
Emaan Fatima	22i-0869
Aden Sial	22i-1313

Session 2022-2026

Supervised by

Ms. Saira Qamar



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction [AS PER SCOPE DOCUMENT]	1
1.1	Existing Solutions	1
1.1.1	How ForeSyte is Different	2
1.2	Problem Statement	3
1.3	Scope	4
1.4	Modules	5
1.4.1	ForeSyte Web Portal	6
1.4.2	Backend Service	6
1.5	Work Division	7
2	Project Requirements [AS PER FYP1 MID REPORT]	8
2.1	Use-cases	8
2.2	Functional Requirements	16
2.3	Non-Functional Requirements	22
2.3.1	Reliability	22
2.3.2	Usability	22
2.3.3	Performance	22
2.3.4	Security	23
2.3.5	Maintainability	23
3	System Overview [AS PER FYP1 MID REPORT]	24
3.1	Architectural Design	24
3.1.1	Box and Line Diagram (Initial Design Stage)	25
3.1.2	High-Level Architectural Style	25
3.1.3	Mapping of Modules/Components	26
3.2	Data Design	26
3.2.1	Data Structures and Organization	26
3.2.2	Major Data Entities	27
3.2.3	Data Flow Diagrams (DFDs)	27
3.2.4	Data Flow and Processing	30
3.3	Domain Model	30
3.4	Design Models [UPTO THE CURRENT ITERATION ONLY]	30
3.4.1	Class Diagram	30

3.4.2	Activity Diagram	33
3.4.3	System Sequence Diagram	33
3.4.4	Sequence Diagrams	35
3.4.5	State Transition Diagram	42
4	Implementation and Testing	44
4.1	Current Progress	44
4.1.1	Front-End	44
4.1.1.1	Admin Dashboard	44
4.1.1.2	Investigator Portal	45
4.1.2	Back-End	45
4.1.2.1	Dashboard API	46
4.1.3	Next Steps	47
4.2	External APIs/SDKs	47
4.2.1	External APIs and SDKs Table	48
4.2.2	Detailed Usage	48
4.3	Algorithm Design	51
4.3.1	Admin Dashboard Aggregation	51
4.3.2	Report Generation Workflow (Planned Design)	52
4.3.3	Investigator Violation Review Flow	53
4.3.4	Seating Plan Upload and Processing	53
4.4	Testing Details	57
4.4.1	Unit Testing	58
4.4.1.1	Investigator Portal Testing	58
4.4.1.2	Student Portal Testing	61
4.4.1.3	Admin Portal Testing	63
	References	66
A	Appendices	67
A.1	Appendix A	68
A.1.1	Admin Portal Screenshots	68
A.1.1.1	Login Page	68
A.1.1.2	Sign Up Page	69
A.1.1.3	Dashboard	70
A.1.1.4	Analytics Page	71
A.1.1.5	Exams Page	72
A.1.1.6	Users Page	73
A.1.1.7	Incidents Page	74
A.1.1.8	Monitoring Page	75
A.1.1.9	Reports Page	76

A.1.1.10	Seating Plan Page	77
A.1.1.11	Upload Seating Plan Page	78
A.1.1.12	Settings Page	79
A.1.1.13	Test Results	80
A.1.2	Investigator Portal Screenshots	81
A.1.2.1	Investigator Dashboard	81
A.1.2.2	Violations Page	82
A.1.2.3	Invigilator Activities Page	83
A.1.2.4	Student Activities Page	84
A.1.2.5	Reports Page	85
A.1.2.6	Live Monitoring Page	86
A.1.2.7	Video Processing Page	87
A.1.2.8	Settings Page	88
A.1.2.9	Test Results	89
A.1.3	Student Portal Screenshots	90
A.1.3.1	Student Dashboard	90
A.1.3.2	Disciplinary Records Page	91
A.1.3.3	Profile Page	92
A.1.3.4	Settings Page	93
A.1.3.5	Test Results	94

List of Figures

1.1	Estimated reduction in cheating rates using ForeSyte AI-based surveillance system	4
1.2	Frequency of students who reported cheating during written examinations [3]	4
1.3	Scope workflow of the ForeSyte system	5
2.1	ForeSyte Use Case Diagram	16
3.1	Box and Line Diagram for ForeSyte AI Exam Surveillance System	25
3.2	High-Level Architectural Design of the ForeSyte AI Exam Surveillance System	26
3.3	DFD Level 0 – ForeSyte System Context Diagram	28
3.4	DFD Level 1 – Detailed ForeSyte System Data Flow	29
3.5	ForeSyte Domain Model Diagram	31
3.6	ForeSyte Class Diagram	32
3.7	ForeSyte Activity Diagram	33
3.8	System Sequence Diagram for ForeSyte	34
3.9	Sequence Diagram for UC-01: View and Manage Reports/Dashboard	35
3.10	Sequence Diagram for UC-02: Manage User Profiles	36
3.11	Sequence Diagram for UC-03: Generate Detection Reports	37
3.12	Sequence Diagram for UC-04: Verify Student Identity & Seat Mapping . . .	38
3.13	Sequence Diagram for UC-05: Monitor Invigilator Activity	39
3.14	Sequence Diagram for UC-06: Monitor Student Behaviour	40
3.15	Sequence Diagram for UC-07: Process Exam Footage (Live/Recorded)	41
3.16	State Transition Diagram	42
A.1	Admin Portal Login Page	68
A.2	Admin Portal Sign Up Page	69
A.3	Admin Portal Dashboard	70
A.4	Admin Portal Analytics Page	71
A.5	Admin Portal Exams Management Page	72
A.6	Admin Portal Users Management Page	73
A.7	Admin Portal Incidents Page	74
A.8	Admin Portal Monitoring Page	75
A.9	Admin Portal Reports Page	76
A.10	Admin Portal Seating Plan Management Page	77

LIST OF FIGURES

A.11 Admin Portal Upload Seating Plan Page	78
A.12 Admin Portal Settings Page	79
A.13 Admin Portal Unit Test Results	80
A.14 Investigator Portal Dashboard	81
A.15 Investigator Portal Violations Management Page	82
A.16 Investigator Portal Invigilator Activities Page	83
A.17 Investigator Portal Student Activities Page	84
A.18 Investigator Portal Reports Page	85
A.19 Investigator Portal Live Monitoring Page	86
A.20 Investigator Portal Video Processing Page	87
A.21 Investigator Portal Settings Page	88
A.22 Investigator Portal Unit Test Results	89
A.23 Student Portal Dashboard	90
A.24 Student Portal Disciplinary Records Page	91
A.25 Student Portal Profile Page	92
A.26 Student Portal Settings Page	93
A.27 Student Portal Unit Test Results	94

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Work Division among Team Members	7
2.1	Summary of Use Cases for ForeSyte	8
2.2	Use Case 1: View & Manage Reports/Dashboard	9
2.3	Use Case 2: Manage User Profiles	10
2.4	Use Case 3: Generate Detection Reports	11
2.5	Use Case 4: Seat Mapping	12
2.6	Use Case 5: Monitor Invigilator Activity	13
2.7	Use Case 6: Monitor Student Behaviour	14
2.8	Use Case 7: Process Exam Footage (Live/Recorded)	15
2.9	Functional Requirements for UC-01	17
2.10	Functional Requirements for UC-02	17
2.11	Functional Requirements for UC-03	17
2.12	Functional Requirements for UC-04	18
2.13	Functional Requirements for UC-05	18
2.14	Functional Requirements for UC-06	18
2.15	Functional Requirements for UC-07	19
2.16	General Functional Requirements	19
4.1	External APIs and SDKs for ForeSyte	48
4.2	Unit Testing for Investigator Portal	58
4.3	Unit Testing for Investigator Portal (continued)	59
4.4	Unit Testing for Student Portal	61
4.5	Unit Testing for Student Portal (continued)	62
4.6	Unit Testing for Admin Portal	63
4.7	Unit Testing for Admin Portal (continued)	64
4.8	Unit Testing for Admin Portal (continued)	65

Chapter 1

Introduction [AS PER SCOPE DOCUMENT]

This project proposal presents **ForeSyte**, an AI-powered CCTV/camera monitoring system designed to detect and log cheating attempts and monitor invigilator activity during examinations. The purpose of this document is to outline the problem, motivation, proposed solution, and stakeholders while providing a detailed background of the system.

ForeSyte integrates computer vision and deep learning techniques to analyze both real-time CCTV/camera feeds and uploaded exam footage. By focusing on seat-specific mapping, the system ensures precise tracking of students and their activities while simultaneously monitoring invigilators' behavior to maintain a fair and transparent examination environment [3].

Furthermore, several studies have highlighted the growing prevalence of academic dishonesty and the limitations of traditional manual invigilation methods. By leveraging AI-driven video analysis, **ForeSyte** aims to improve the accuracy of cheating detection, automate timestamped reporting, and enhance overall exam integrity. The system also acts as a deterrent; as students become aware that their activities are continuously monitored, the likelihood of cheating is expected to decrease significantly over time [4].

1.1 Existing Solutions

Several solutions have been developed to address the problem of cheating detection and exam monitoring, each with distinct approaches and limitations. Below is a critical overview and comparison of the most relevant systems:

- **IndoAI:** Employs proprietary CNN-based gesture and face models running on Edge AI cameras. While it offers AI-driven gesture and face analysis, it does not support

seat-based activity detection or invigilator monitoring. Additionally, it lacks the capability to generate structured, timestamped reports. Its reliance on specialized hardware (Edge AI cameras) limits scalability and integration with existing CCTV systems.

- **IPB University System:** Uses dual webcams (front and rear) for each student, combined with AI software for face and gaze monitoring. This approach is not based on CCTV and does not provide seat-based activity detection or invigilator monitoring. While it offers some level of timestamped reporting, it lacks structured, per-seat violation logs and is less scalable for large exam halls due to the need for individual webcams per student.

Table 1.1: Comparison of Existing Solutions

System Name	Technology	Limitations
IndoAI	Proprietary CNN-based gesture + face models on Edge AI cameras.	No seat-based activity detection; no invigilator monitoring; no structured timestamped reports; requires specialized hardware.
IPB University System	Dual webcam (front + rear) per student; AI software (face/gaze monitoring models); not CCTV.	No seat-based activity detection; no invigilator monitoring; limited but no structured per-seat timestamped reports; not scalable for large halls.

1.1.1 How ForeSyte is Different

ForeSyte stands out from existing solutions by addressing their key limitations:

- **Seat-based Activity Detection:** ForeSyte uniquely maps detected activities to specific seats, ensuring identification of students involved in suspicious behavior.
- **Integration with Existing CCTV:** Unlike other solutions that require specialized hardware or individual webcams, ForeSyte works seamlessly with standard CCTV infrastructure, making it scalable and cost-effective for large exam halls.
- **Invigilator Monitoring:** ForeSyte not only monitors students but also tracks invigilator presence and activity, identifying negligence or prolonged absences.

- **Structured, Timestamped Reports:** The system generates detailed, per-seat violation logs with timestamps, enabling efficient review and transparent reporting.
- **Comprehensive Automation:** By combining real-time and recorded video analysis, ForeSyte automates the entire monitoring and reporting process, reducing manual effort and enhancing exam integrity.

This approach ensures that ForeSyte provides a comprehensive, scalable, and automated exam monitoring solution that overcomes the major shortcomings of existing systems.

1.2 Problem Statement

Academic dishonesty has become a significant concern in educational institutions worldwide. Studies indicate that approximately 61% of university students have admitted to cheating during written examinations, while nearly 68% confess to cheating at least once during their academic career [3]. Traditional manual invigilation methods are often insufficient, as invigilators may unintentionally overlook suspicious activities, and reviewing long hours of CCTV/camera footage after exams is both time-consuming and ineffective.

To address these challenges, **ForeSyte** provides an AI-driven solution for automated examination monitoring. The system processes both real-time CCTV/camera streams and uploaded video footage, detecting suspicious activities such as phone usage, hand waving, bending over the desk, looking around, and standing up using advanced computer vision techniques.

The system focuses on seat-specific mapping, ensuring that every detected activity is linked to the correct student. In addition, **ForeSyte** incorporates an Invigilator Activity Monitoring module to:

- Track invigilators' movements
- Detect prolonged idle periods
- Identify potential negligence

An admin dashboard provides real-time monitoring and automated report generation, enabling discipline committees to directly access timestamped violations instead of manually searching through hours of footage. By combining AI-powered surveillance with intelligent analytics, **ForeSyte** enhances exam security and aims to reduce academic dishonesty over time by providing actionable insights. Furthermore, as students become aware that their activities are continuously monitored and every suspicious action is captured with timestamped reports, the likelihood of cheating is expected to decrease significantly over repeated examination sessions.

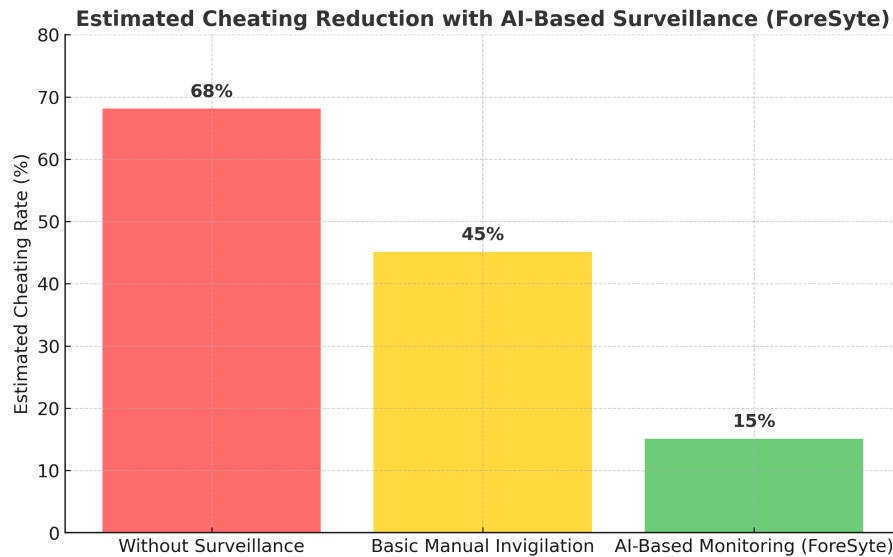


Figure 1.1: Estimated reduction in cheating rates using ForeSyte AI-based surveillance system

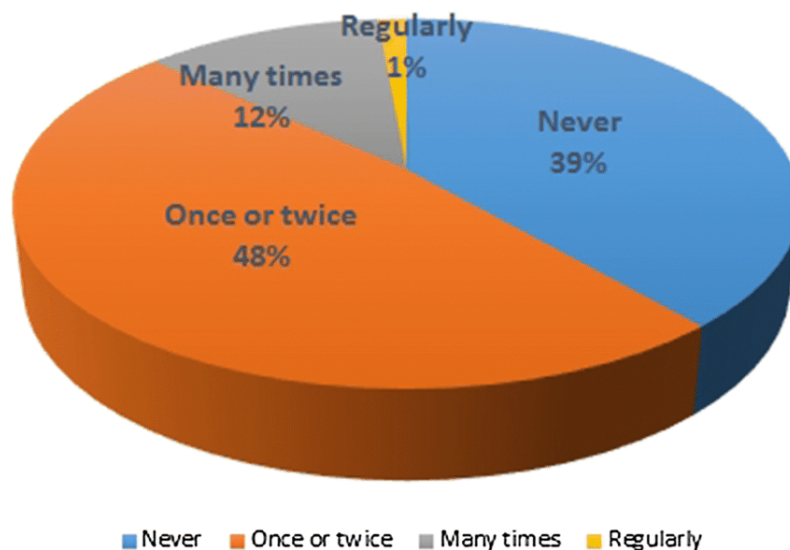


Figure 1.2: Frequency of students who reported cheating during written examinations [3]

1.3 Scope

The scope of the ForeSyte project includes developing an AI-powered exam surveillance system that enhances the integrity of examinations conducted in physical exam halls. Recent research highlights the importance of automated invigilation for maintaining academic integrity [3, 4].

The system will process CCTV/camera footage, both live streams and uploaded recordings, to automatically detect and log suspicious activities such as unauthorized object

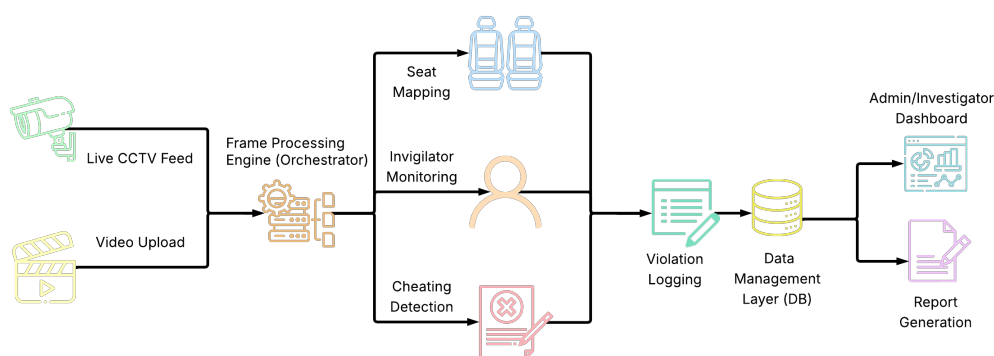


Figure 1.3: Scope workflow of the ForeSyte system

usage and irregular gestures. It will also incorporate seating arrangement verification, ensuring that each detected activity is accurately linked to the correct student through seat mapping, similar to approaches in modern proctoring systems [1, 5].

The project further extends to monitoring invigilator activity within the exam hall, identifying negligence or prolonged absence from duty. Similar works on offline and intelligent monitoring systems support this approach [6, 7]. All detected events will be recorded with precise timestamps, seat numbers and room identifiers, enabling a structured and transparent record of violations.

An administrative dashboard will be developed to centralize monitoring and provide clear violation reports supported with visual evidence. This will reduce the need for institutions to manually review extensive CCTV/camera footage, offering instead a concise and accurate summary of flagged incidents.

The system will be designed for real-time performance using optimized frame sampling and AI models capable of efficient video analysis. The project will be self-funded and developed entirely by our team. Related research also discusses the ethical implications of AI in exam supervision, highlighting fairness and privacy issues.

The boundaries of this project are confined to creating a robust and intelligent surveillance solution that automates detection, verification, and reporting within physical exam environments, thereby ensuring greater fairness, efficiency, and accountability in academic assessments.

1.4 Modules

The **ForeSyte** project is organized into several core modules, each responsible for specific functionalities required to ensure seamless, automated exam monitoring. The modules are divided into two main components: the *ForeSyte Web Portal* for stakeholders and the

Backend Service that powers the AI-driven monitoring capabilities.

1.4.1 ForeSyte Web Portal

The **ForeSyte Web Portal** is a multi-user interface designed for administrators, discipline committee members, and university management. It provides centralized access to seating plans and violation reports, ensuring transparent and efficient management of examination activities.

1. **Dashboard Visualization:** Displays real-time updates of suspicious activities, flagged violations, and invigilator presence across multiple exam halls using data from connected CCTV/camera systems.
2. **Report Access:** Enables authorized users to view, manage, and download detailed violation reports containing timestamps, seat details and room information.
3. **User Roles and Permissions:** Implements secure role-based access, where administrators manage seating plans, camera configurations, and exam schedules, while discipline committee members focus on reviewing reports and initiating actions.
4. **Seating Plan Integration:** Links violations to student seating layouts to ensure accurate identification of candidates involved in flagged incidents.

1.4.2 Backend Service

The **Backend Service** powers the core intelligence of the ForeSyte system, managing video analysis, behavioral detection, invigilator monitoring, and reporting. This module processes video streams from CCTV/camera systems and uploaded exam footage to ensure accurate real-time and recorded exam monitoring.

1. **Video Stream Processing:** Integrates with CCTV/camera systems and uploaded recordings, extracting frames for real-time detection of activities and events.
2. **Cheating Detection Engine:** Uses advanced computer vision techniques to identify suspicious behaviors such as unauthorized object usage, irregular gestures, communication, and frequent seat movements [1, 2].
3. **Seat Mapping:** Maps detected activities to seating plans for accurate violation reporting, ensuring each activity is linked to the correct student seat.
4. **Invigilator Monitoring:** Tracks invigilator's prolonged absences from assigned zones, and identifies potential negligence or unauthorized phone usage[6].

5. **Report Generation:** Compiles timestamped violation logs, including room details, seat numbers, and integrates them directly into the ForeSyte Web Portal for automated PDF/Excel report exports.

1.5 Work Division

For each module and respective feature, responsibility is assigned to a team member.

Name	Responsibility
Inam Ullah Shaikh	• Frontend Development • Model Training • Documentation • Deployment • Backend Integration
Emaan Fatima	• Backend Development • Model Research • Model Training • Documentation • Deployment
Aden Sial	• Data Collection • Model Training • Model Research • Documentation • Deployment

Table 1.2: Work Division among Team Members

Chapter 2

Project Requirements [AS PER FYP1 MID REPORT]

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-cases

The Table 2.1 lists down all the possible use cases for our project. Note that login, sign up and feedback pages are not included in these use cases.

Table 2.1: Summary of Use Cases for ForeSyte

Usecase ID	Use Case Name	Description
UC-01	View & Manage Reports/Dashboard	Authorized users view and manage reports of student and invigilator activity.
UC-02	Manage User Profiles	System administrator manages accounts and roles of all users.
UC-03	Generate Detection Reports	System generates timestamped reports of detected cheating or negligence.
UC-04	Seat Mapping	System maps detected activities to assigned student seats.
UC-05	Monitor Invigilator Activity	System monitors invigilators for negligence, absence, or unauthorized actions.
UC-06	Monitor Student Behaviour	System monitors student actions to detect cheating attempts.
UC-07	Process Exam Footage (Live/Recorded)	System processes live and recorded video streams to detect activities.

Below, you can find the detailed use case description for each use case.

2.1.1 UC-01

Table 2.2: Use Case 1: View & Manage Reports/Dashboard

Use Case Detail	Description
Use Case ID	UC-01
Use Case Name	View & Manage Reports/Dashboard
Scope	ForeSyte Web Portal
Level	User Goal
Brief Description	Users view, filter, and manage real-time dashboards and past reports.
Actor(s) Involved	System Administrator, Investigator, Discipline Committee
Stakeholders	• Discipline Committee: Reviews violation data • Admins: Configure reports
Pre-conditions	Reports and logs already generated by detection system.
Post-conditions	Users access detailed dashboards with downloadable reports.
Main Success Scenario	1. Actor logs into the ForeSyte Web Portal successfully. 2. Actor navigates to the Reports/Dashboard section from the main menu. 3. System retrieves all available violation reports from the database. 4. System displays a dashboard with filters (e.g., by student ID, seat number, violation type, or timestamp). 5. Actor applies filters to refine the report view. 6. System dynamically updates the dashboard according to the selected filters. 7. Actor selects a specific report for detailed view. 8. System displays complete details of the selected report (student, invigilator, time, violation evidence). 9. Actor chooses the option to download or export the report. 10. System generates the report in the selected format (PDF/Excel) and confirms successful export.
Special Requirements	Role-based access, secure data visualization

2.1.2 UC-02

Table 2.3: Use Case 2: Manage User Profiles

Use Case Detail	Description
Use Case ID	UC-02
Use Case Name	Manage User Profiles
Scope	ForeSyte Web Portal
Level	User Goal
Brief Description	System administrator manages user accounts, roles, and permissions.
Actor(s) Involved	System Administrator
Stakeholders	• Admins: Maintain structured access
Pre-conditions	Admin is authenticated.
Post-conditions	User accounts created, modified, or deleted.
Main Success Scenario	1. Admin logs into the ForeSyte Web Portal. 2. Admin navigates to the User Management module. 3. System displays the list of existing users with their roles. 4. Admin selects an action (add, edit, or delete a user). 5. If adding: Admin enters new user details including user-name, role, and credentials. 6. If editing: Admin updates user details or permissions. 7. If deleting: Admin selects a user and confirms deletion. 8. System validates the request and updates the database accordingly. 9. System confirms success with a message (user added/updated/deleted). 10. Admin logs out or continues with further management tasks.
Special Requirements	Secure password policies, role-based restrictions

2.1.3 UC-03

Table 2.4: Use Case 3: Generate Detection Reports

Use Case Detail	Description
Use Case ID	UC-03
Use Case Name	Generate Detection Reports
Scope	Backend Service + Web Portal
Level	User Goal
Brief Description	System generates timestamped detection reports of suspicious activity.
Actor(s) Involved	Investigator, Discipline Committee
Stakeholders	• Discipline Committee: Reviews reports • University Management
Pre-conditions	Detection system has analyzed video footage.
Post-conditions	Report generated and stored in database.
Main Success Scenario	1. AI detection engine processes exam footage. 2. System identifies suspicious activity (e.g., phone usage, looking around, invigilator negligence). 3. System logs details including timestamp, seat number, room, and involved actor. 4. System compiles the violation data into a structured report format. 5. Investigator accesses the Web Portal and navigates to the Reports section. 6. System displays the list of generated reports. 7. Investigator selects a specific report for review. 8. System displays detailed evidence including video snippets or screenshots. 9. Investigator verifies and approves the report for further review. 10. Report is stored in the system and available for export (PDF/Excel).
Special Requirements	Reports exportable to PDF/Excel

2.1.4 UC-04

Table 2.5: Use Case 4: Seat Mapping

Use Case Detail	Description
Use Case ID	UC-04
Use Case Name	Seat Mapping
Scope	Backend Service
Level	User Goal
Brief Description	System maps detected activities to assigned student seats.
Actor(s) Involved	System (Automated)
Stakeholders	• University Management
Pre-conditions	Seating plan and student ID data available.
Post-conditions	Detected activities are mapped to correct student seats.
Main Success Scenario	1. Exam seating plan is uploaded into the system. 2. System processes video feeds from CCTV or camera sources. 3. System detects suspicious activities in the video frames. 4. System maps detected activities to specific seat locations using seating plan data. 5. System links each detected activity to the student assigned to that seat. 6. System logs the activity along with seat ID and student ID. 7. Mapped data is linked to subsequent activity monitoring and reports.
Special Requirements	Accurate seat mapping with reliable coordinate detection

2.1.5 UC-05

Table 2.6: Use Case 5: Monitor Invigilator Activity

Use Case Detail	Description
Use Case ID	UC-05
Use Case Name	Monitor Invigilator Activity
Scope	Backend Service
Level	User Goal
Brief Description	System tracks invigilators for negligence, absence, or unauthorized actions.
Actor(s) Involved	Invigilator, Discipline Committee
Stakeholders	• University Management
Pre-conditions	Cameras active and invigilator zones defined.
Post-conditions	Invigilator activity log generated.
Main Success Scenario	1. Surveillance cameras start recording invigilator activity. 2. System tracks invigilator's presence across different zones of the exam hall. 3. AI analyzes the video feed to detect movement or prolonged inactivity. 4. If invigilator leaves their zone for too long, the system flags negligence. 5. If invigilator is using unauthorized devices (e.g., phone), system captures it. 6. System logs the timestamp, location, and detected action. 7. Discipline Committee accesses the dashboard to view invigilator logs. 8. System displays flagged negligence cases in the report. 9. Committee reviews and validates the report. 10. Finalized invigilator activity log is stored and exported if needed.
Special Requirements	Accurate zone mapping

2.1.6 UC-06

Table 2.7: Use Case 6: Monitor Student Behaviour

Use Case Detail	Description
Use Case ID	UC-06
Use Case Name	Monitor Student Behaviour
Scope	Backend Service
Level	User Goal
Brief Description	Detects cheating behaviors (phone use, gestures, looking around).
Actor(s) Involved	Student, Investigator
Stakeholders	• Discipline Committee • University Management
Pre-conditions	Camera feed and detection models active.
Post-conditions	Violations logged with timestamps.
Main Success Scenario	1. Exam begins and surveillance cameras monitor all students. 2. AI detection model continuously analyzes student actions. 3. System identifies suspicious behavior such as phone use, hand gestures, or looking around repeatedly. 4. When detected, system flags the behavior as a potential violation. 5. System records timestamp, seat ID, and video evidence. 6. Investigator accesses the dashboard to review logged violations. 7. System displays detailed violation logs with associated video frames. 8. Investigator examines the evidence for each flagged action. 9. Investigator confirms or dismisses the violation. 10. Validated violations are added to the final exam report.
Special Requirements	Must differentiate normal vs cheating actions

2.1.7 UC-07

Table 2.8: Use Case 7: Process Exam Footage (Live/Recorded)

Use Case Detail	Description
Use Case ID	UC-07
Use Case Name	Process Exam Footage (Live/Recorded)
Scope	Backend Service
Level	User Goal
Brief Description	System processes live and recorded exam video footage for analysis.
Actor(s) Involved	System Administrator, Investigator
Stakeholders	• University Management
Pre-conditions	Camera streams or uploaded recordings available.
Post-conditions	Footage analyzed and results stored.
Main Success Scenario	1. System connects to live CCTV cameras or receives uploaded exam recordings. 2. System validates video input and prepares it for analysis. 3. AI engine processes video frames in real time or batch mode. 4. System identifies and logs suspicious student or invigilator behavior. 5. System maps detected activity to the correct student seat. 6. All activities are logged with timestamps and stored in the database. 7. Investigator accesses the Reports module. 8. System displays results of processed footage (violations, seat mapping). 9. Investigator reviews the logged activities and evidence. 10. Reports are finalized and exported in required format.
Special Requirements	Efficient processing, storage optimization

2.1.8 Use Case Diagram of ForeSyte

The Figure 2.1 highlights the use case diagram of the entire ForeSyte Web Application.

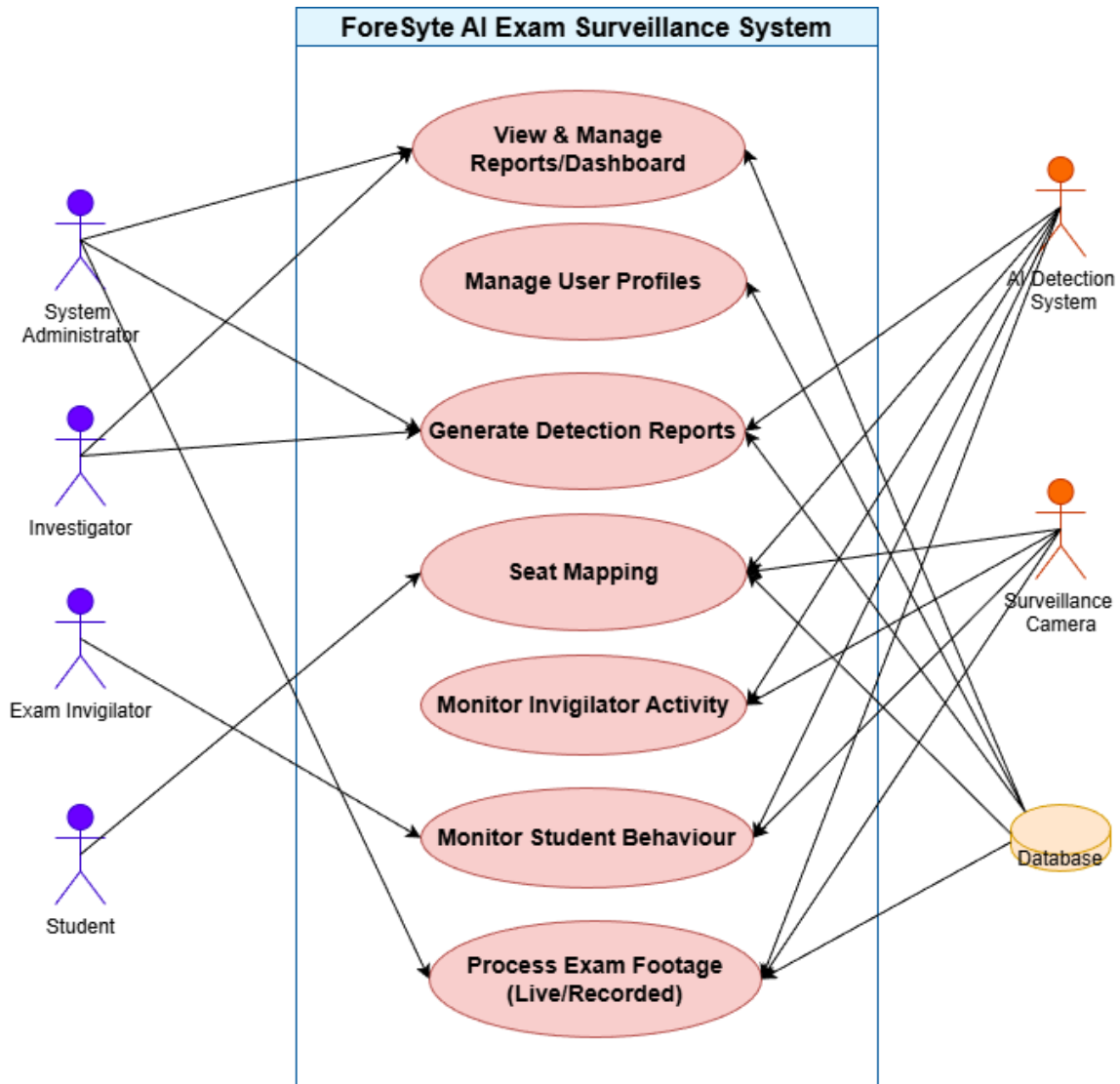


Figure 2.1: ForeSyte Use Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of ForeSyte arranged in a use case-based organizational method.

2.2.1 UC-01 Functional Requirement

Table 2.9: Functional Requirements for UC-01

Functional Requirement ID	Description
FR-01	The system shall allow authorized users to log in and access the Reports/Dashboard module.
FR-02	The system shall display real-time and historical reports of student and invigilator activity.
FR-03	The system shall provide filtering options (by student ID, seat number, violation type, timestamp).
FR-04	The system shall allow exporting reports in PDF and Excel formats.
FR-05	The system shall enforce role-based access to reports.

2.2.2 UC-02 Functional Requirement

Table 2.10: Functional Requirements for UC-02

Functional Requirement ID	Description
FR-06	The system shall allow the administrator to add new user accounts with role assignments.
FR-07	The system shall allow editing of existing user details and permissions.
FR-08	The system shall allow deleting user accounts securely.
FR-09	The system shall validate inputs (username, password, role) before saving.
FR-10	The system shall enforce strong password and security policies.

2.2.3 UC-03 Functional Requirement

Table 2.11: Functional Requirements for UC-03

Functional Requirement ID	Description
FR-11	The system shall automatically generate detection reports when violations are flagged.
FR-12	The system shall include timestamps, seat numbers, and violation type in reports.
FR-13	The system shall store generated reports in a database.
FR-14	The system shall allow investigators to review and approve reports.
FR-15	The system shall allow reports to be exported in PDF/Excel format.

2.2.4 UC-04 Functional Requirement

Table 2.12: Functional Requirements for UC-04

Functional Requirement ID	Description
FR-16	The system shall map detected activities to assigned seat numbers.
FR-17	The system shall link activities to students based on seating plan data.
FR-18	The system shall log activity mapping results (student ID, seat ID, activity type).
FR-19	The system shall ensure accurate coordinate-based seat detection.
FR-20	The system shall handle cases where seat mapping cannot be determined.

2.2.5 UC-05 Functional Requirement

Table 2.13: Functional Requirements for UC-05

Functional Requirement ID	Description
FR-21	The system shall monitor invigilators during exam sessions.
FR-22	The system shall track invigilator presence in defined zones.
FR-23	The system shall detect unauthorized activities (e.g., phone use).
FR-24	The system shall log negligence events with timestamp and location.
FR-25	The system shall provide invigilator activity logs to the Discipline Committee.

2.2.6 UC-06 Functional Requirement

Table 2.14: Functional Requirements for UC-06

Functional Requirement ID	Description
FR-26	The system shall analyze student actions in real time.
FR-27	The system shall detect suspicious behaviors (phone use, hand gestures, looking around).
FR-28	The system shall differentiate between normal and abnormal behaviors.
FR-29	The system shall log violations with seat ID, timestamp, and video snippet.
FR-30	The system shall allow investigators to review and confirm logged violations.

2.2.7 UC-07 Functional Requirement

Table 2.15: Functional Requirements for UC-07

Functional Requirement ID	Description
FR-31	The system shall process both live CCTV feeds and uploaded recordings.
FR-32	The system shall analyze frames using AI detection models.
FR-33	The system shall log activities with timestamps and seat IDs.
FR-34	The system shall store processed results in the database.
FR-35	The system shall make analyzed results available in the Reports module.

2.2.8 General Functional Requirements

Table 2.16: General Functional Requirements

Functional Requirement ID	Description
FR-36	The system shall ensure processing of violations and reports within 20 seconds.
FR-37	The system shall implement secure storage and encryption of student/exam data.
FR-38	The system shall allow a responsive web interface for investigators and administrators.
FR-39	The system shall maintain audit logs for all activities.
FR-40	The system shall support scalability for multiple exam halls simultaneously.

2.2.9 Module 1: ForeSyte Web Portal

Following are the requirements for Module 1 (ForeSyte Web Portal):

1. Dashboard Visualization

- (a) FR-01: The system shall allow authorized users to log in and access the Reports/Dashboard module.
- (b) FR-02: The system shall display real-time and historical reports of student and invigilator activity.
- (c) FR-03: The system shall provide filtering options (by student ID, seat number, violation type, timestamp).
- (d) FR-04: The system shall allow exporting reports in PDF and Excel formats.

- (e) FR-05: The system shall enforce role-based access to reports.

2. Report Access

- (a) FR-11: The system shall automatically generate detection reports when violations are flagged.
- (b) FR-12: The system shall include timestamps, seat numbers, and violation type in reports.
- (c) FR-13: The system shall store generated reports in a database.
- (d) FR-14: The system shall allow investigators to review and approve reports.
- (e) FR-15: The system shall allow reports to be exported in PDF/Excel format.

3. User Roles and Permissions

- (a) FR-06: The system shall allow the administrator to add new user accounts with role assignments.
- (b) FR-07: The system shall allow editing of existing user details and permissions.
- (c) FR-08: The system shall allow deleting user accounts securely.
- (d) FR-09: The system shall validate inputs (username, password, role) before saving.
- (e) FR-10: The system shall enforce strong password and security policies.

2.2.10 Module 2: Backend Service

Following are the requirements for Module 2 (Backend Service):

1. Video Stream Processing

- (a) FR-31: The system shall process both live CCTV feeds and uploaded recordings.
- (b) FR-32: The system shall analyze frames using AI detection models.
- (c) FR-33: The system shall log activities with timestamps and seat IDs.
- (d) FR-34: The system shall store processed results in the database.
- (e) FR-35: The system shall make analyzed results available in the Reports module.

2. Cheating Detection Engine

- (a) FR-26: The system shall analyze student actions in real time.

- (b) FR-27: The system shall detect suspicious behaviors (phone use, hand gestures, looking around).
- (c) FR-28: The system shall differentiate between normal and abnormal behaviors.
- (d) FR-29: The system shall log violations with seat ID, timestamp, and video snippet.
- (e) FR-30: The system shall allow investigators to review and confirm logged violations.

3. Seat Mapping

- (a) FR-16: The system shall map detected activities to assigned seat numbers.
- (b) FR-17: The system shall link activities to students based on seating plan data.
- (c) FR-18: The system shall log activity mapping results (student ID, seat ID, activity type).
- (d) FR-19: The system shall ensure accurate coordinate-based seat detection.
- (e) FR-20: The system shall handle cases where seat mapping cannot be determined.

4. Invigilator Monitoring

- (a) FR-21: The system shall monitor invigilators during exam sessions.
- (b) FR-22: The system shall track invigilator presence in defined zones.
- (c) FR-23: The system shall detect unauthorized activities (e.g., phone use).
- (d) FR-24: The system shall log negligence events with timestamp and location.
- (e) FR-25: The system shall provide invigilator activity logs to the Discipline Committee.

5. Report Generation and Security

- (a) FR-36: The system shall ensure processing of violations and reports within 20 seconds.
- (b) FR-37: The system shall implement secure storage and encryption of student/exam data.
- (c) FR-38: The system shall allow a responsive web interface for investigators and administrators.
- (d) FR-39: The system shall maintain audit logs for all activities.
- (e) FR-40: The system shall support scalability for multiple exam halls simultaneously.

2.3 Non-Functional Requirements

This section specifies the non-functional requirements of the ForeSyte system. These requirements are practical, measurable, and verifiable to ensure the system can be realistically implemented in an academic setting.

2.3.1 Reliability

- REL-1: The system shall achieve at least 95% uptime during scheduled exam sessions.
- REL-2: In case of camera or feed failure, the system shall log the event and notify the administrator within 1 minute.
- REL-3: The system shall store video footage and violation data redundantly to prevent data loss.

2.3.2 Usability

- USE-1: An administrator shall be able to generate a violation report in no more than 3 interactions.
- USE-2: New users (administrators or investigators) shall be able to learn the core functions of the system with less than 2 hours of training.
- USE-3: The dashboard shall present violation summaries in a clear tabular format with filters.

2.3.3 Performance

- PER-1: The system shall process live camera feeds with a maximum latency of 10 seconds.
- PER-2: The cheating detection engine shall analyze at least 20 frames per second per camera feed.
- PER-3: Violation reports shall be generated within 20 minutes of the end of an exam session.

2.3.4 Security

- SEC-1: Only authenticated users shall access the system dashboard.
- SEC-2: User access shall be role-based (administrator, investigator).
- SEC-3: All access attempts shall be logged for audit purposes.

2.3.5 Maintainability

- MAI-1: The system shall follow a modular architecture so that detection models can be updated independently.
- MAI-2: Minor bugs shall be diagnosable and fixable within 48 hours.
- MAI-3: The documentation shall describe system modules, and troubleshooting in detail for developers.

Chapter 3

System Overview [AS PER FYP1 MID REPORT]

ForeSyte AI Exam Surveillance System is an intelligent monitoring solution designed to enhance the integrity of in-person examinations by automating student and invigilator monitoring. The system integrates CCTV cameras, computer vision, and deep learning models to detect suspicious student behaviors such as phone usage, frequent seat movements, or unauthorized communication, while also tracking invigilator activities to ensure accountability. Each activity is mapped to seating plans to ensure accurate identification of students. Detected events are logged as timestamped violations, which are compiled into structured reports accessible through an administrative dashboard. Administrators manage user profiles, seating plans, and exam sessions, while disciplinary investigators utilize the violation reports to review cases and make informed decisions. The system follows a modular architecture consisting of two primary components: the ForeSyte Web Portal, a multi-user web interface for dashboard visualization and report management, and the Backend Service, the core AI engine handling video stream processing, cheating detection, seat mapping, and automated report generation. This architectural design ensures separation of concerns, scalability, and maintainability while providing seamless integration between the user interface and the underlying AI processing capabilities.

3.1 Architectural Design

The ForeSyte AI Exam Surveillance System is designed using a modular architecture to ensure scalability, maintainability, and clear separation of concerns. At a high level, the system is divided into front-end modules, back-end modules, and supporting external systems. Each module plays a distinct role but collaborates with others to achieve the overall goal of automated, AI-powered exam monitoring.

The architecture follows a **two-tier client-server architecture** with **layered components**, implementing a **RESTful API** design pattern. The front-end (presentation tier) handles user input, report visualization, and dashboard interactions through a React-based single-page application. The back-end (application tier) processes video streams, performs AI-based detection and analysis, and manages business logic through FastAPI routes. The data management layer (data tier) ensures persistence and retrieval of information using PostgreSQL for relational data and MongoDB for high-velocity event logs. This decomposition ensures that the AI detection logic, data storage, and user interfaces remain loosely coupled and independently upgradable.

3.1.1 Box and Line Diagram (Initial Design Stage)

In the initial design stage, a simple box and line diagram is used to represent the main system components and their interactions. This diagram provides a high-level overview of how the core modules connect, without detailing the internal architecture or data flow.

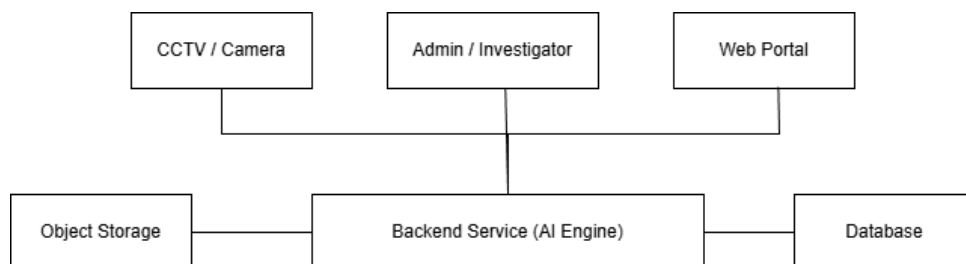
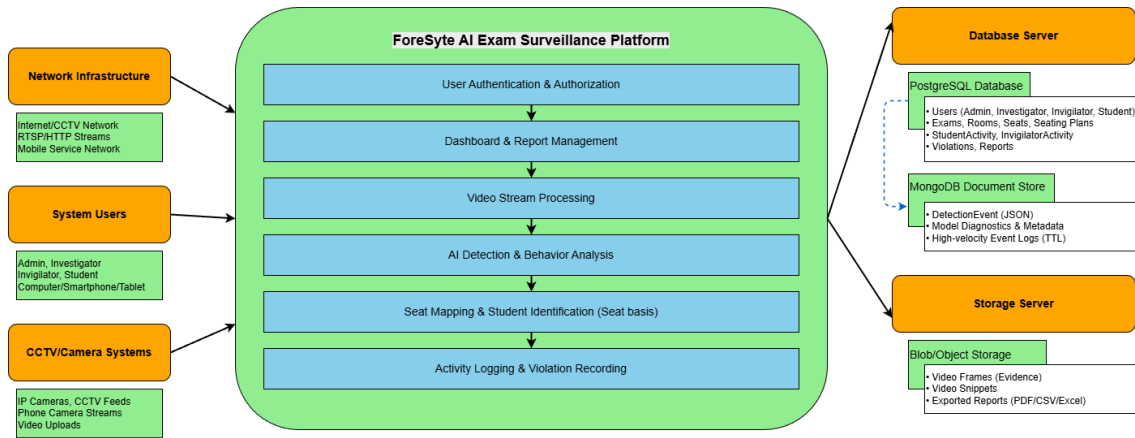


Figure 3.1: Box and Line Diagram for ForeSyte AI Exam Surveillance System

3.1.2 High-Level Architectural Style

After the initial overview, the finalized architecture style/pattern diagram is presented. The system uses a two-tier client-server architecture with layered components, providing clear separation between the presentation layer (React frontend), application layer (FastAPI backend with embedded business logic), and data management layer (PostgreSQL and MongoDB). Figure 3.2 illustrates the high-level architecture of the system, showing the main subsystems and their interactions.

Figure 3.2: High-Level Architectural Design of the ForeSyte AI Exam Surveillance System



3.1.3 Mapping of Modules/Components

Each module/component is mapped to its respective part of the architecture, ensuring modularity and scalability. The architecture supports integration with external systems (CCTV, storage, databases) and provides a robust foundation for AI-powered exam monitoring.

3.2 Data Design

The data design for **ForeSyte** focuses on how the system's information domain is represented and transformed into data structures across relational, document, and object stores. In this section, we outline the major system entities, how they are stored, processed, and managed to ensure robust operation and secure handling of user information and evidence.

3.2.1 Data Structures and Organization

ForeSyte deals with multiple data classes: user identities, seating plans, live/recorded video evidence, AI detections, activities, violations, and reports. The core data is organized so that:

- **Relational (PostgreSQL)** holds normalized, queryable facts: Users, Exams, Rooms, Seats, StudentActivity, InvigilatorActivity, Violations, Reports, and seat assignments; enforced with foreign keys.
- **Document (MongoDB)** holds high-velocity *DetectionEvent* JSON and model diagnostics with TTL, enabling fast ingest and optional replay/debug.

- **Blob/Object Storage** (evidence bucket) stores immutable artifacts: frame crops, video snippets, and exported PDF reports.

Data lifecycle. Streams or uploads produce detection events; an aggregation service converts them into activities and violations with evidence links; dashboards and investigators read relational views and open the referenced evidence in object storage.

3.2.2 Major Data Entities

Key logical entities:

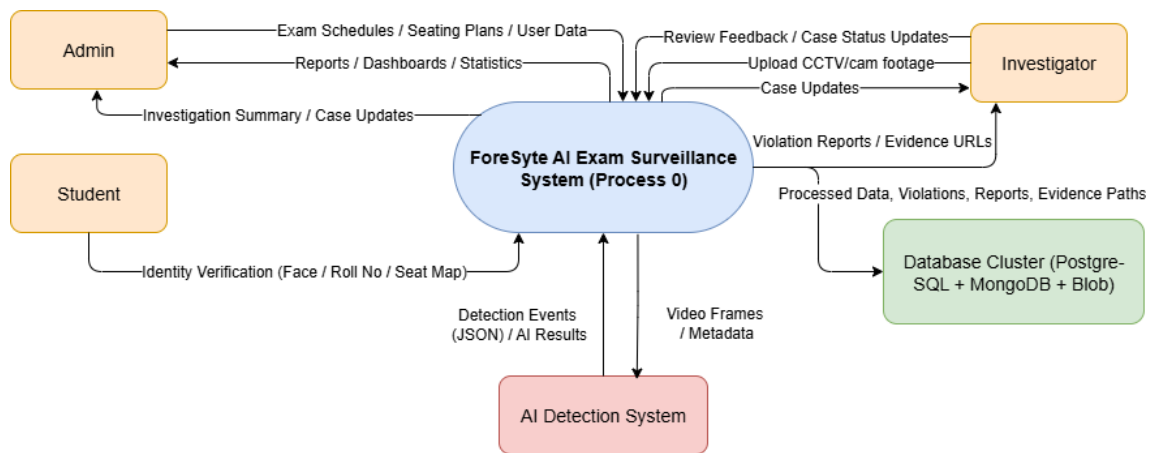
- **User Information:** Admin, Invigilator, Investigator, and Student profiles with credentials and roles.
- **Exam/Room/Seat:** Exam sessions; rooms with cameras; seats mapped to students and exams.
- **DetectionEvent** (document): Raw detector output with timestamps, camera/room source, and model metadata.
- **StudentActivity/InvigilatorActivity:** Curated activities derived from detections; include severity, confidence, and optional evidence URL.
- **Violation:** Confirmed cheating events; include type, timestamp, severity, status, and optional evidence URL.
- **Report:** Generated summaries (PDF) of violations and activities for investigators.

3.2.3 Data Flow Diagrams (DFDs)

This subsection presents the logical data flow within the ForeSyte system at two abstraction levels. Level 0 provides an overview of the main system interactions, while Level 1 elaborates on internal data processing — especially how seat mapping, activity detection, and behavior monitoring are interconnected.

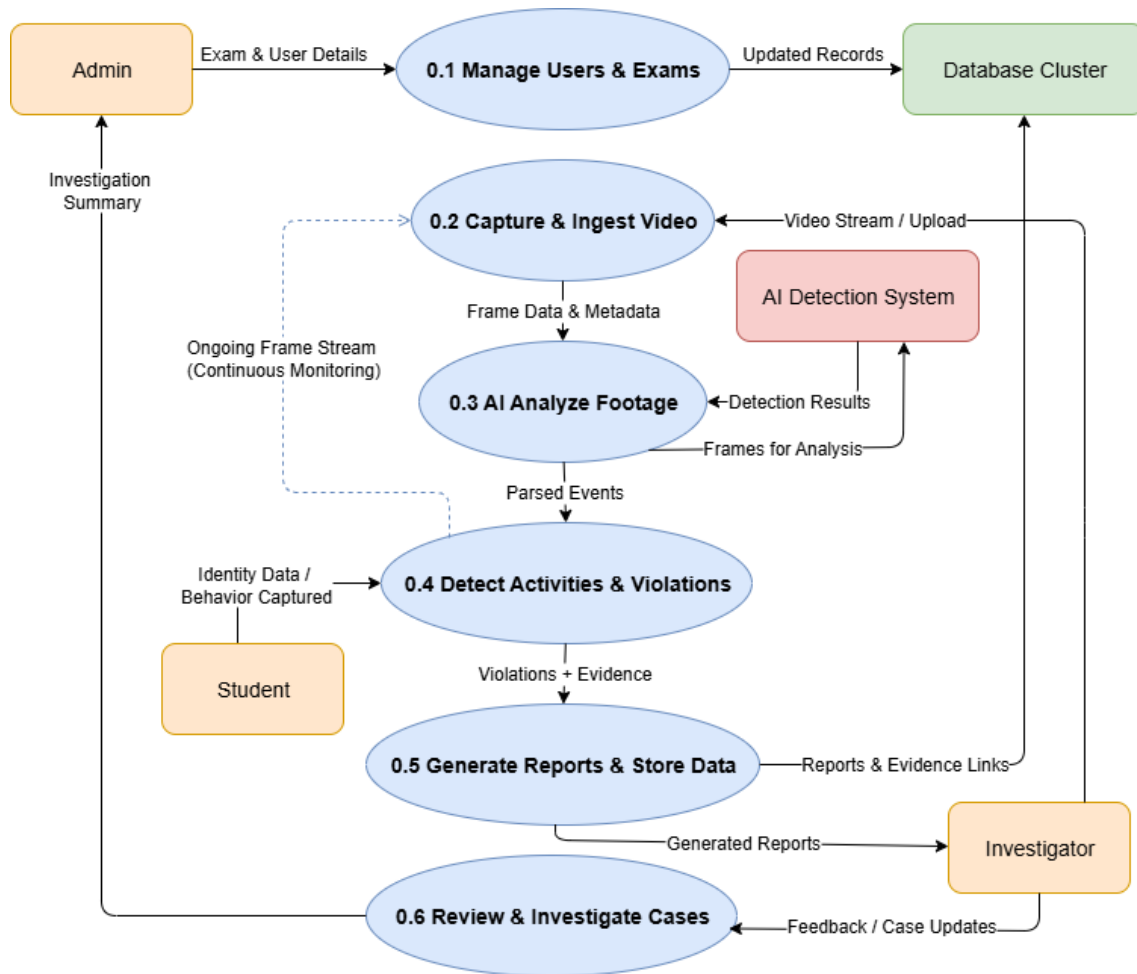
Level 0 – Context Diagram Figure 3.3 illustrates the high-level data flow between external entities (such as Students, Invigilators, and Administrators) and the ForeSyte System. It highlights how examination footage, attendance data, and violation reports circulate through the system.

Figure 3.3: DFD Level 0 – ForeSyte System Context Diagram



Level 1 – System Decomposition Figure 3.4 expands the ForeSyte process into its major internal modules: Seat Mapping, Cheating Detection Engine, Invigilator Monitoring, and Report Generation. Data from entry-point cameras are first processed to map activities to specific seats; this mapping is then propagated through subsequent modules for activity analysis and reporting.

Figure 3.4: DFD Level 1 – Detailed ForeSyte System Data Flow



3.2.4 Data Flow and Processing

The system processes exam footage via two paths that converge into a unified pipeline:

1. **Real-time (Live) Monitoring:** Camera streams feed the AI service, which emits per-frame *DetectionEvents* (JSON).
2. **Uploaded Footage:** After the exam, recorded video is ingested; the AI service produces the same *DetectionEvents* offline.
3. **Aggregation/Rules:** Business rules transform events into *StudentActivity/InvigilatorActivity*; suspicious patterns yield *Violation* rows with evidence links.
4. **Persistence:** Normalized facts land in PostgreSQL; evidence media is stored in Blob/Object storage; optionally, transient raw events remain in MongoDB (TTL).
5. **Consumption:** Admin dashboards query relational views; investigators open pre-signed evidence URLs and download reports.

3.3 Domain Model

Figure 3.5 represents the domain model for ForeSyte. It outlines the key entities, their attributes, and the relationships that define data interactions within the ForeSyte system.

3.4 Design Models [UPTO THE CURRENT ITERATION ONLY]

The ForeSyte system employs an object-oriented development approach, utilizing several key design models to represent the system's structure, behavior, and interactions. These models provide comprehensive documentation of the system's architecture and facilitate understanding of complex relationships between system components.

3.4.1 Class Diagram

The system's class diagram represents the core entities and their relationships within the ForeSyte domain model.

Figure 3.5: ForeSys Domain Model Diagram

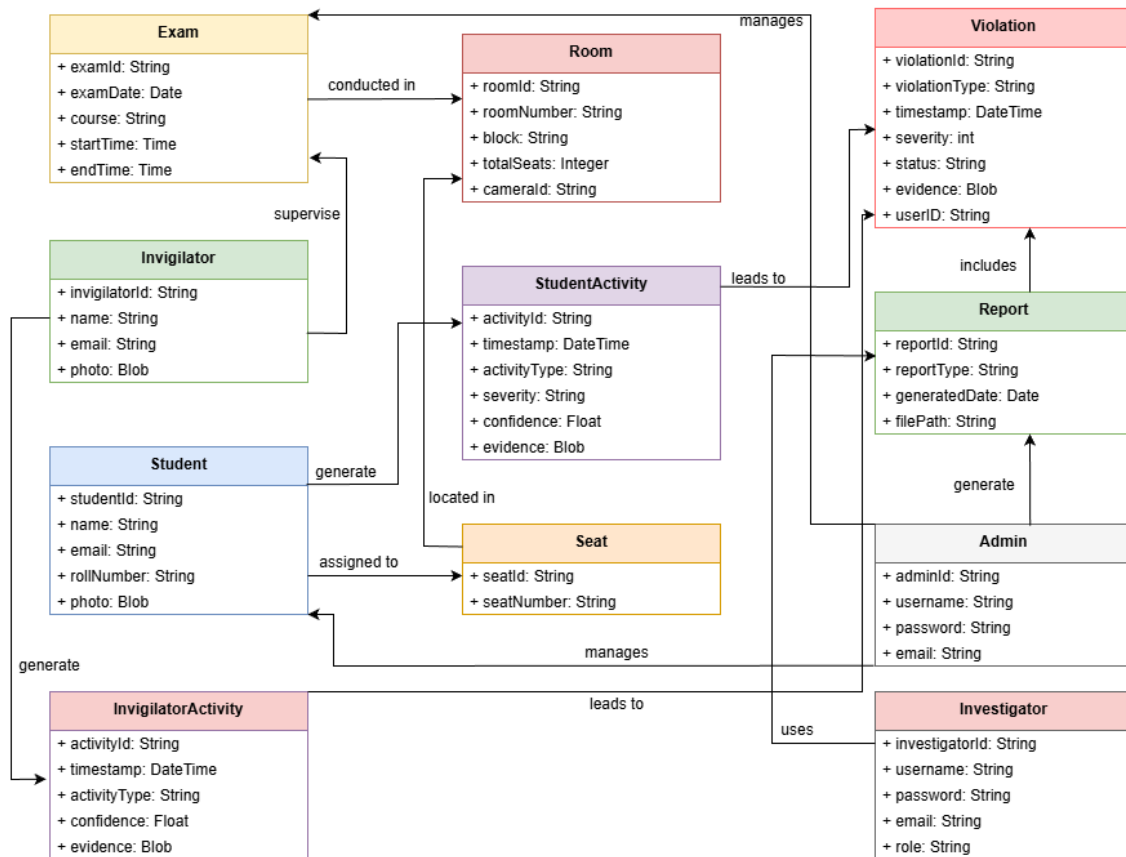
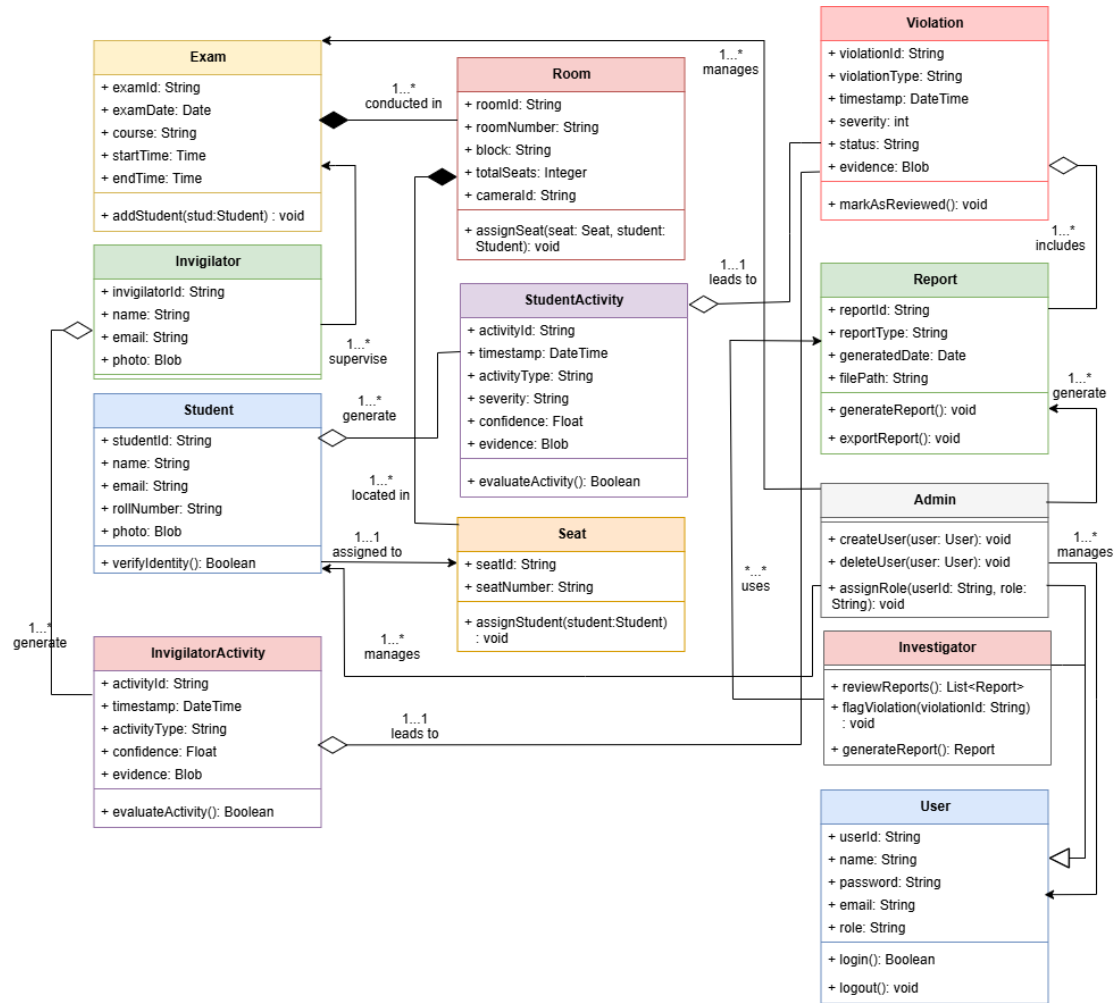


Figure 3.6: ForeSysTe Class Diagram



3.4.2 Activity Diagram

The system's activity diagram illustrates the main processes and workflows:

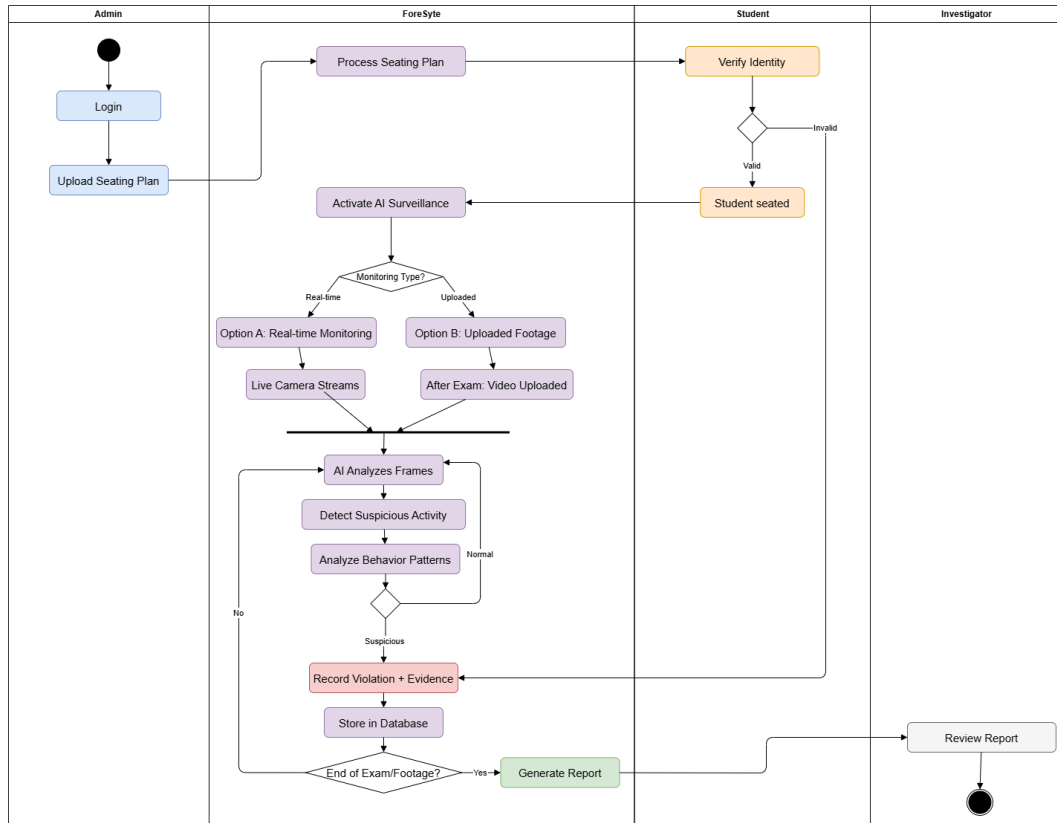


Figure 3.7: ForeSysy Activity Diagram

3.4.3 System Sequence Diagram

Figure 3.8 illustrates the high-level System Sequence Diagram (SSD) for ForeSysy. It depicts the main interactions between external actors (Students, Invigilators, Administrators, and the Discipline Committee) and the ForeSysy AI System across the examination workflow. This diagram provides an overview of how user actions trigger system processes such as attendance tagging, cheating detection, invigilator monitoring, and report generation.

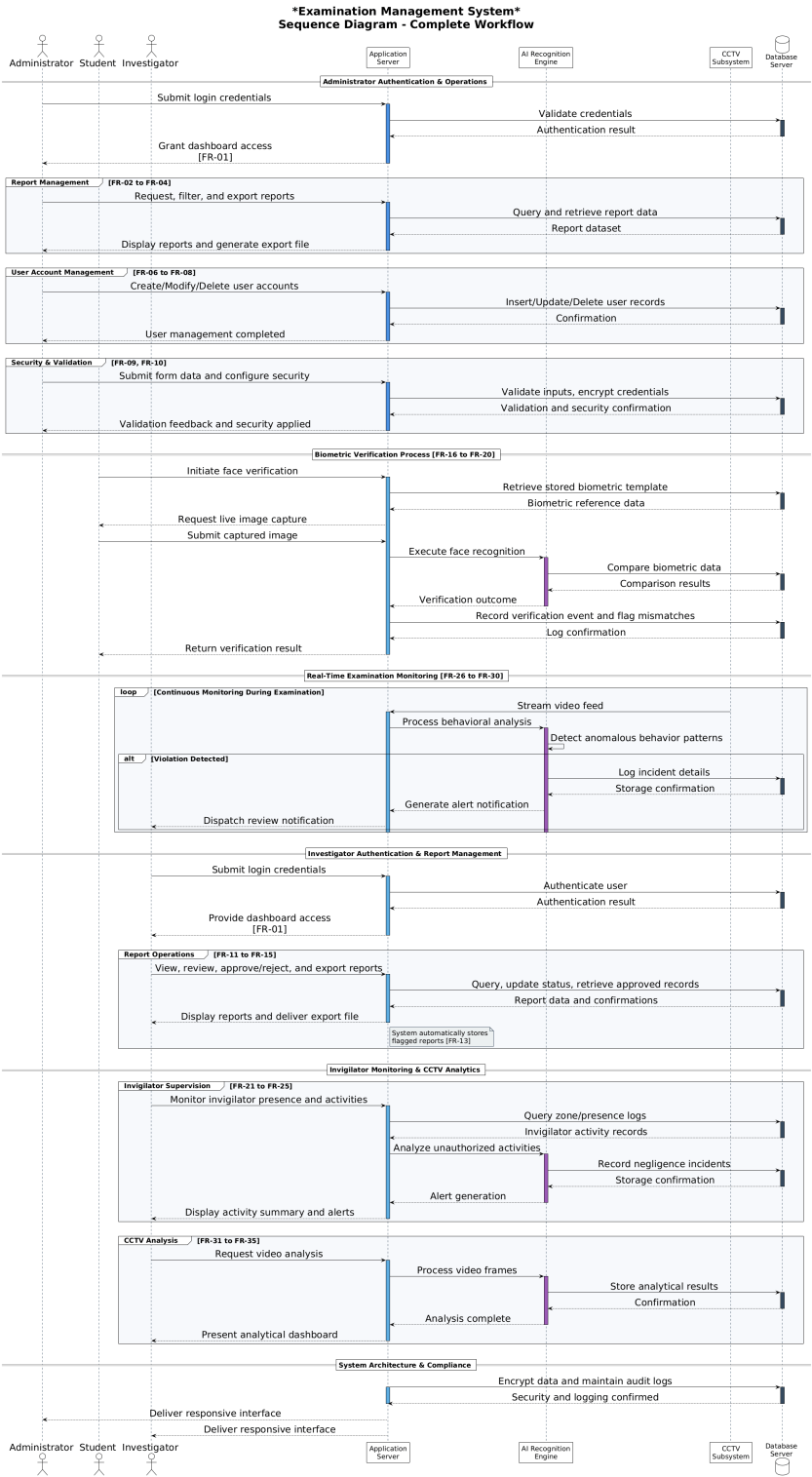


Figure 3.8: System Sequence Diagram for ForeSyte

3.4.4 Sequence Diagrams

This section presents the sequence diagrams for all identified use cases (UC-01 to UC-07). Each diagram illustrates the dynamic interaction between actors, the ForeSyte AI System, and domain entities.

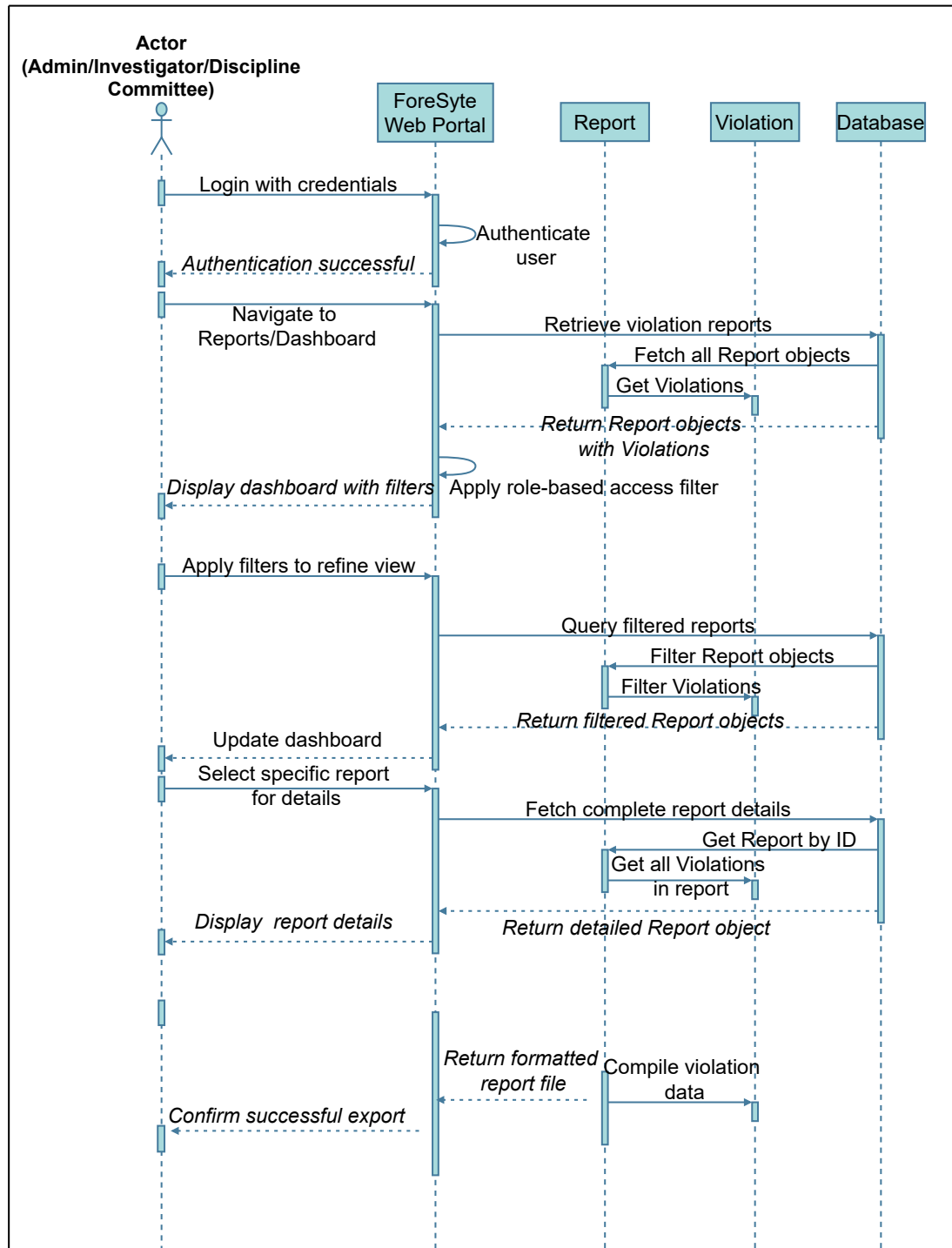


Figure 3.9: Sequence Diagram for UC-01: View and Manage Reports/Dashboard

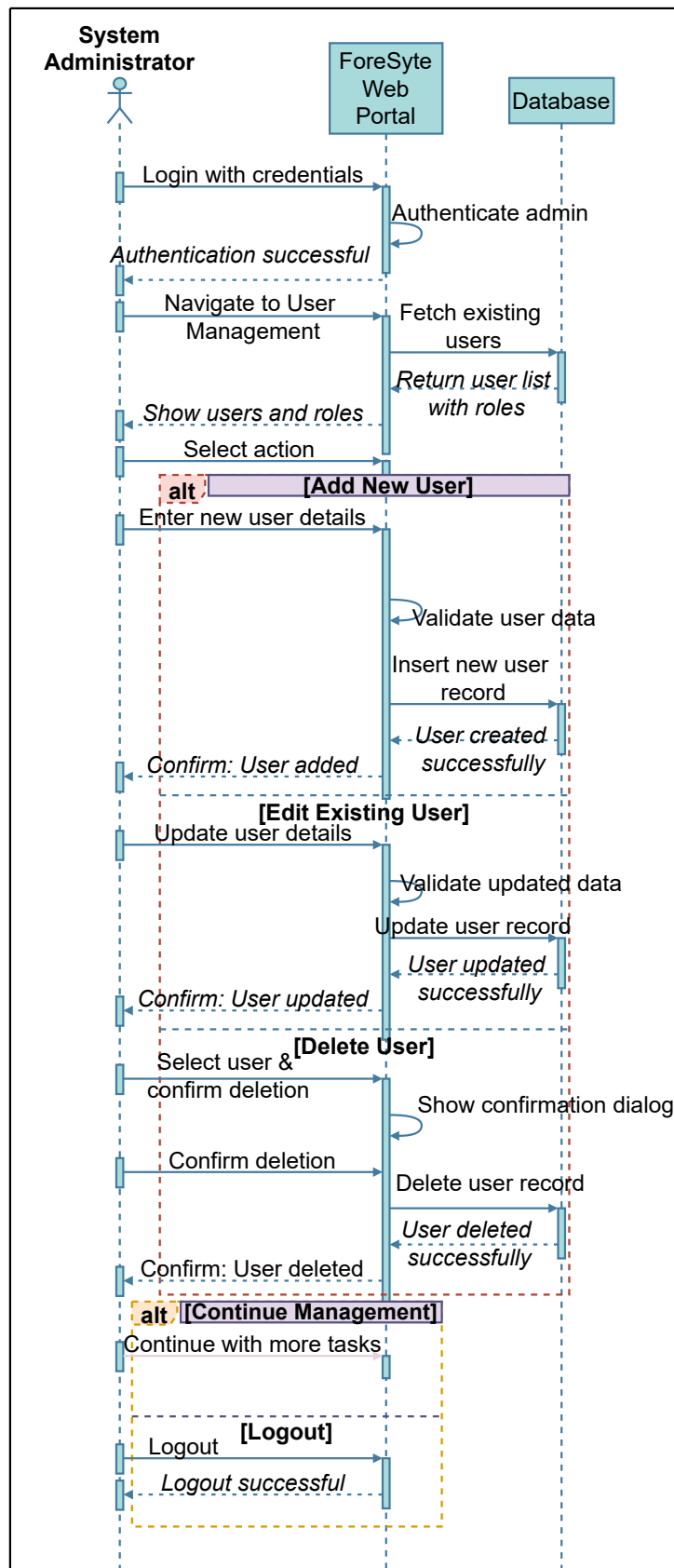


Figure 3.10: Sequence Diagram for UC-02: Manage User Profiles

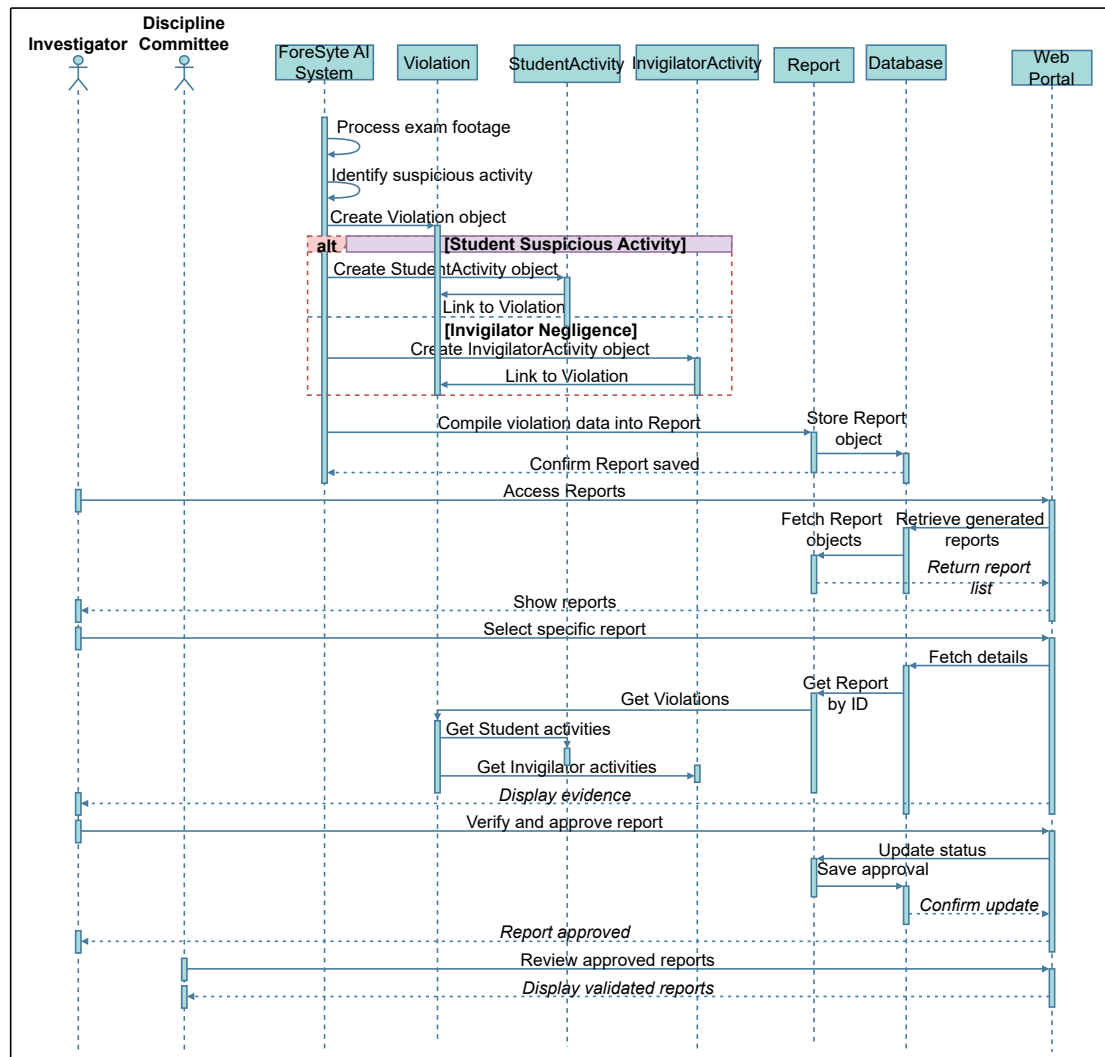


Figure 3.11: Sequence Diagram for UC-03: Generate Detection Reports

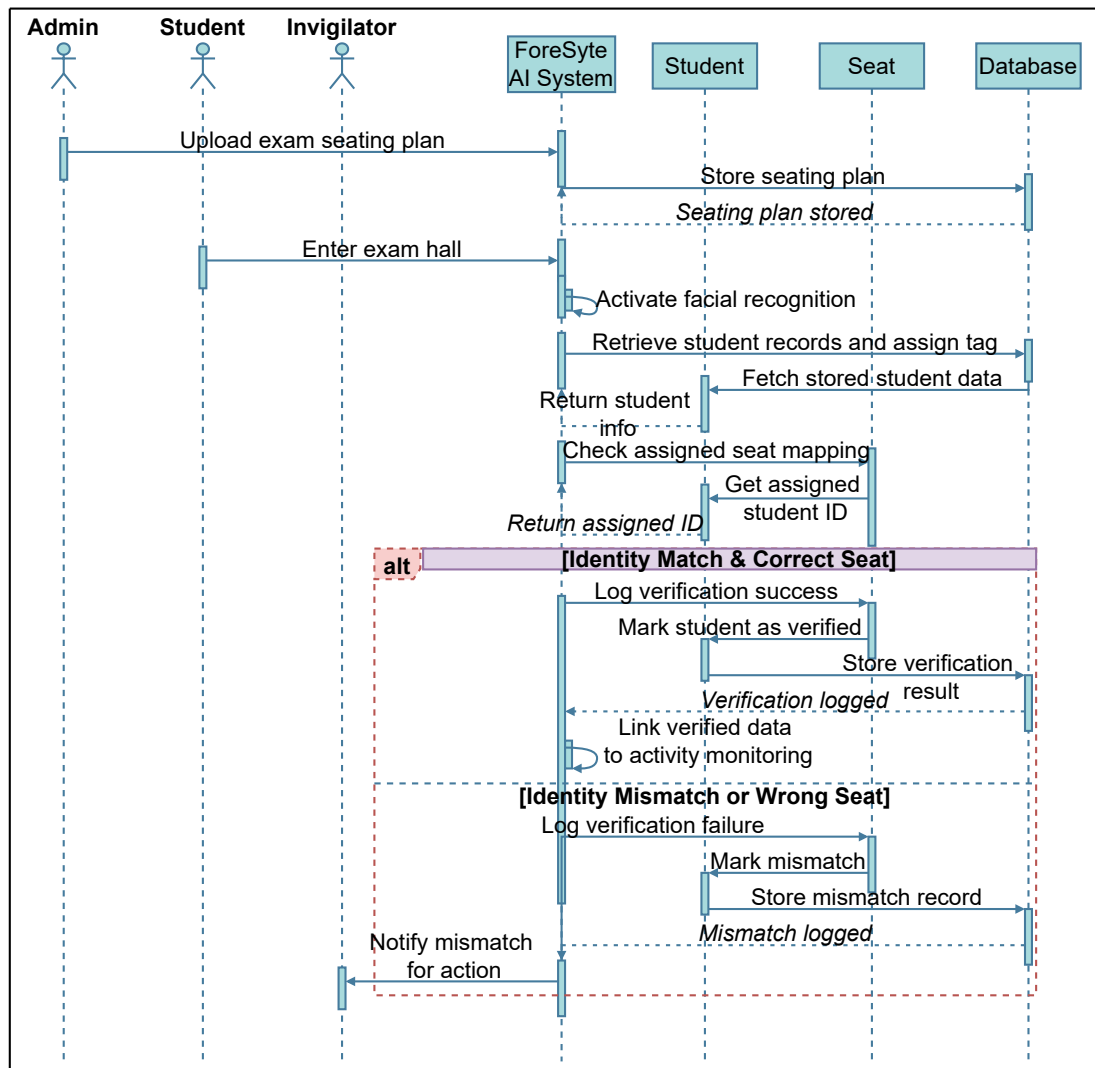


Figure 3.12: Sequence Diagram for UC-04: Verify Student Identity & Seat Mapping

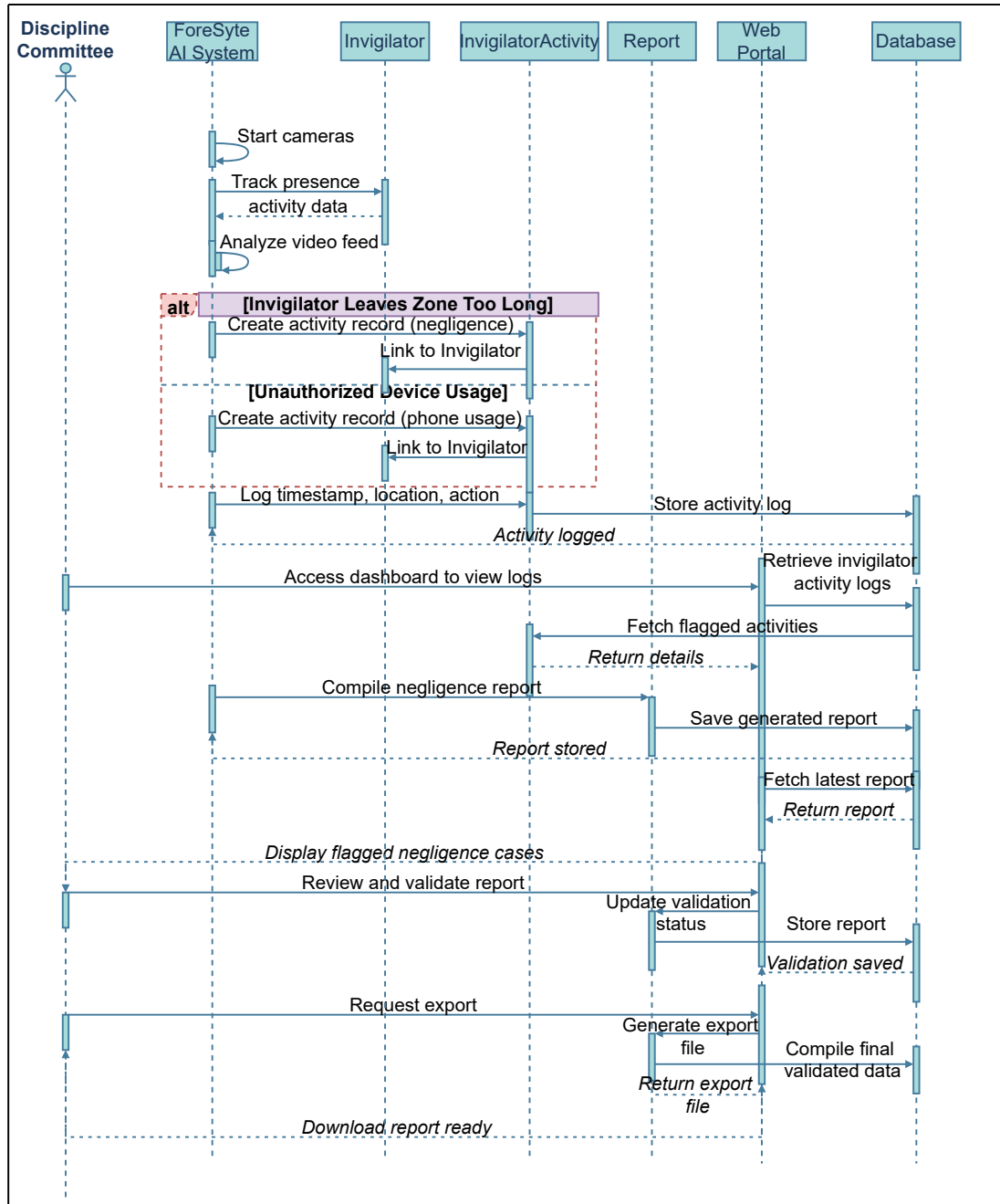


Figure 3.13: Sequence Diagram for UC-05: Monitor Invigilator Activity

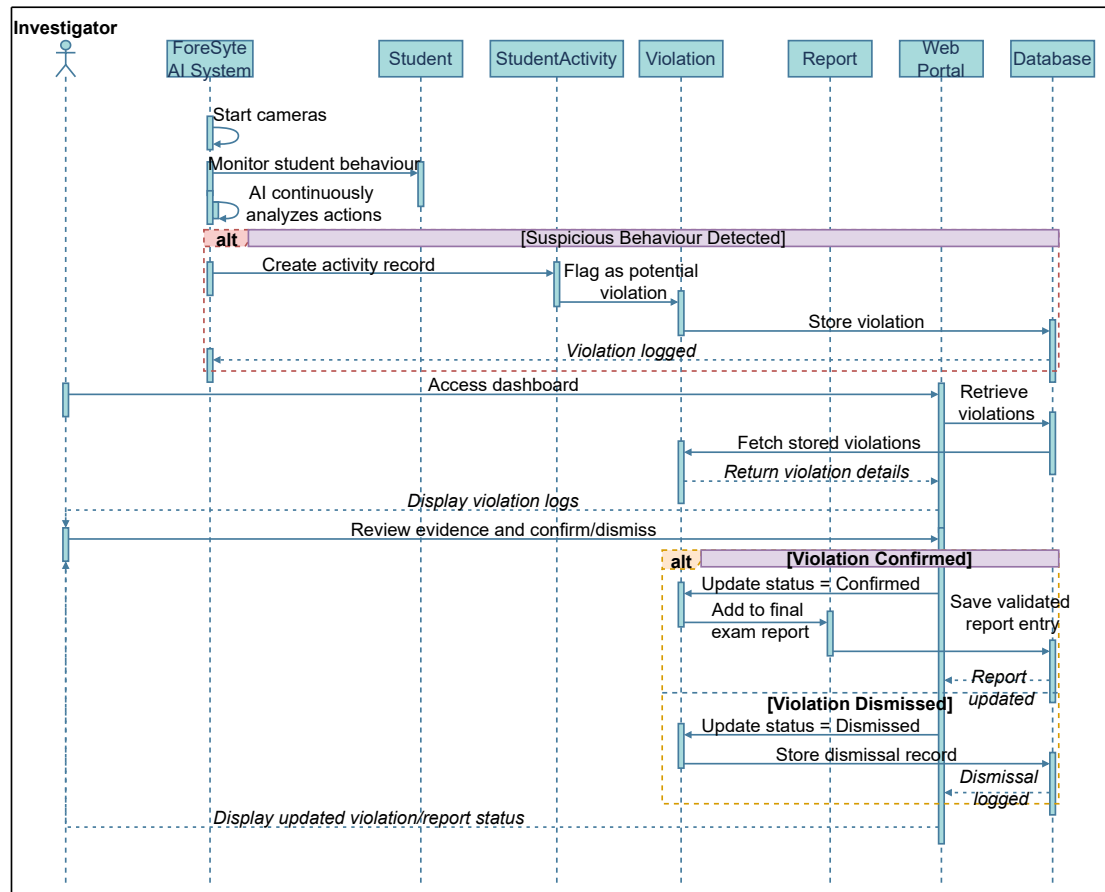


Figure 3.14: Sequence Diagram for UC-06: Monitor Student Behaviour

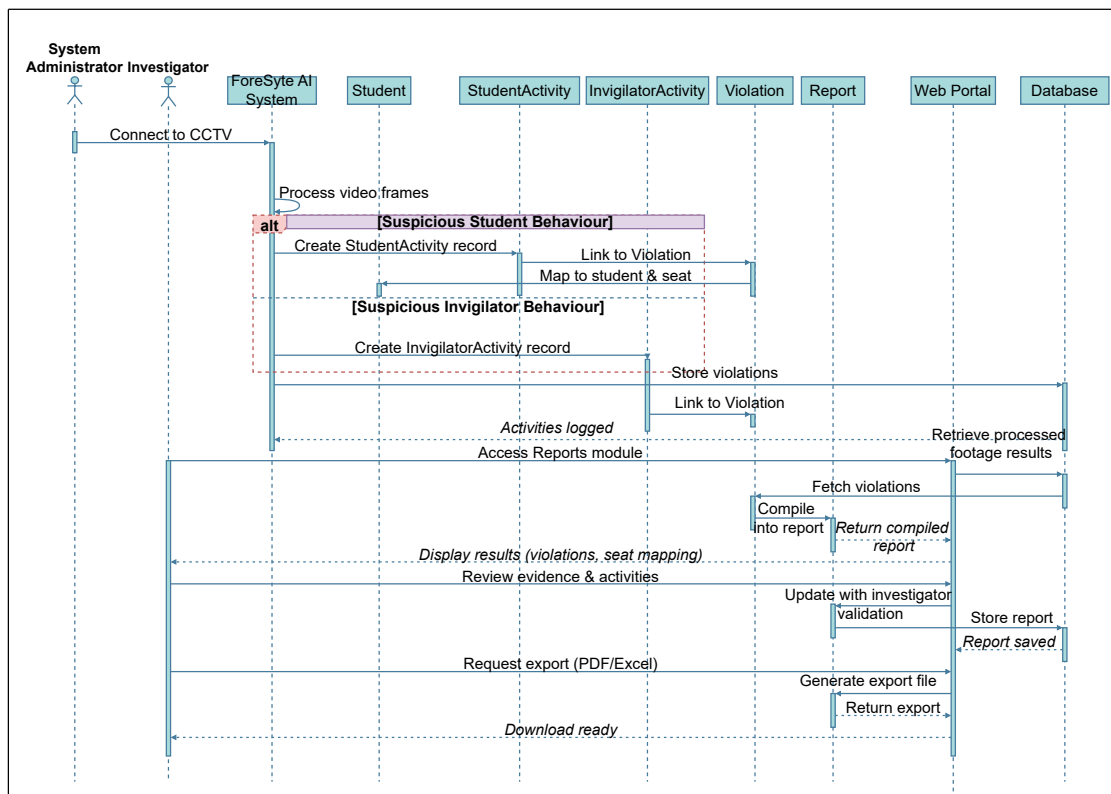


Figure 3.15: Sequence Diagram for UC-07: Process Exam Footage (Live/Recorded)

3.4.5 State Transition Diagram

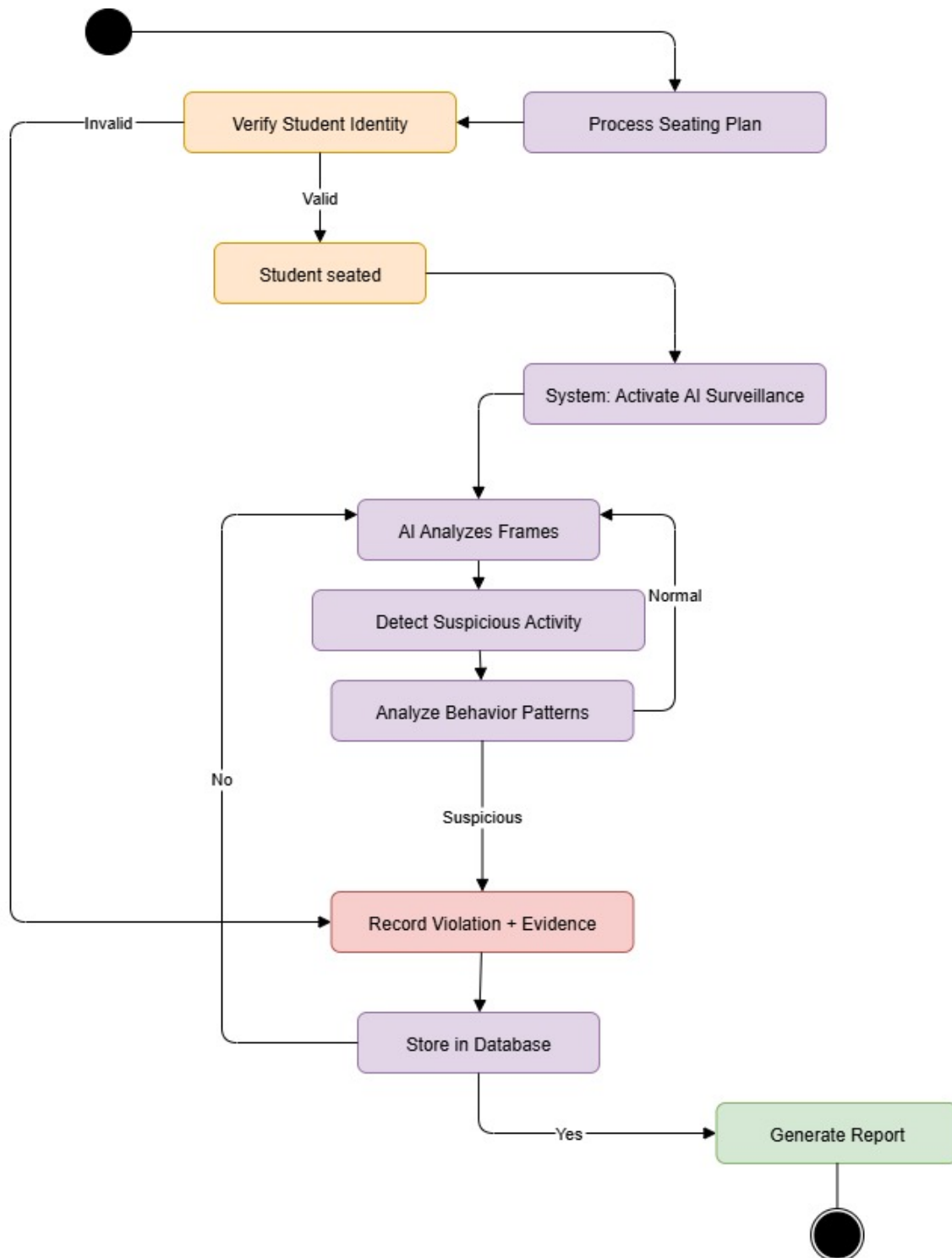


Figure 3.16: State Transition Diagram

This model collectively provide a comprehensive view of the ForeSyte system's architecture, ensuring clear understanding of system components, their interactions, and the

overall system behavior throughout the examination monitoring process.

Chapter 4

Implementation and Testing

This chapter describes the current implementation status of the **ForeSyte** AI Exam Surveillance System and the testing performed up to this iteration. The focus of this phase is on the **Admin** and **Investigator** dashboards and their supporting backend services. Computer-vision based monitoring and the Student Portal are planned components and therefore remain out of scope for this iteration.

4.1 Current Progress

4.1.1 Front-End

The front-end of ForeSyte is implemented as a modern single-page application using React, TypeScript, Vite, and Tailwind CSS. The interface is split into role-specific areas so that administrators and investigators see different dashboards and workflows.

4.1.1.1 Admin Dashboard

The main admin dashboard presents a high-level overview of system activity. It displays:

- **Active Exams** – number of exams scheduled for the current or future dates.
- **Incidents Detected** – count of student activity records (currently using placeholder data; actual cheating detection is planned for future iterations).
- **Students Monitored** – number of students currently under surveillance based on seating plans and exam sessions.
- **Average Exam Duration** – average exam length computed from start and end times.

These statistics are fetched via the backend `/dashboard/stats` endpoint with a period parameter (today, week, or month). The dashboard also includes:

- An **Activity Overview** bar chart that compares the number of incidents and exams across time buckets using the `/dashboard/activity` endpoint.
- A **Recent Incidents** panel that lists the latest flagged student activities with severity tags using the `/dashboard/recent-incidents` endpoint.
- A **Quick Actions** area that links to live monitoring, reports, user management, and system settings.
- A **System Status** section that summarizes the health of the detection pipeline, camera network, analytics, and database.

The Admin Dashboard is therefore fully wired to the backend aggregation APIs. Loading states, error handling, and period switching are implemented in the front-end.

4.1.1.2 Investigator Portal

The Investigator Portal is implemented as a separate area in the React application, with its own header and sidebar components to clearly separate workflows from the admin role.

- The **Investigator Dashboard** displays summary cards for total violations, pending reviews, violations resolved today, and active exams. It also lists recent activities (e.g., phone usage, invigilator negligence, unauthorized movement) with severity and status badges and a period filter (today, week, month).
- The **Reports Page** shows a grid of generated reports with metadata such as report type (incident/exam), exam name, number of violations, generation date, and author. It provides filters for text search and report type, and a modal workflow for configuring a new report (type, export format, inclusion of video evidence links).

At this iteration, these investigator views use mock data to validate layout and interactions while backend report listing endpoints are being finalized. However, the UI contracts, filter behaviour, and state management are implemented and ready to be connected to live APIs.

4.1.2 Back-End

The backend is built with FastAPI and SQLAlchemy on top of PostgreSQL, with MongoDB used for flexible logging and high-volume events. Authentication uses JWT-based

tokens with role-based access control (RBAC) to differentiate between Admin, Investigator, Invigilator, and Student users.

Two backend modules are particularly important for this iteration:

- **Dashboard API** (/dashboard): aggregates data for the Admin and Investigator dashboards.
- **Reports API** (/reports): manages report generation and retrieval for incidents and exams.

4.1.2.1 Dashboard API

The /dashboard/stats endpoint computes:

- **Active Exams:** exams scheduled for the current or future dates.
- **Incidents Detected:** count of StudentActivity records in the selected period (note: actual cheating detection via computer vision is not yet implemented; this metric reflects activity records that would be populated by the detection pipeline in future iterations).
- **Students Monitored:** distinct count of students with seat assignments in active exams.
- **Average Exam Duration:** average duration (in minutes) based on start and end times of exams in the selected period.
- **Trends:** percentage changes for incidents, students, and exams when compared to the previous period.

The /dashboard/activity endpoint returns a sequence of time buckets, each containing the number of incidents and exams in that interval. For today, 4-hour slots are used; for week and month, daily buckets are used. The /dashboard/recent-incidents endpoint returns the most recent student activities, joined with student information to show names, types, severity levels, timestamps, and model confidence values.

All dashboard endpoints enforce RBAC policies and only allow access to users with admin or investigator roles.

Reports API

The /reports router provides the API structure for managing report records. **Note:** The actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration; the endpoints currently create database records and return metadata, but do not produce downloadable report files.

- Standard CRUD endpoints (POST /, GET /, GET /{id}, PUT /{id}, DELETE /{id}) operate on the Report model and validate related Violation and Investigator records. These endpoints are functional for managing report meta-data.
- POST /reports/incidents accepts a list of incident IDs, desired export format, and whether to include video links. It validates and resolves the incident IDs into StudentActivity records, ensures there is an associated Violation, creates a new Report entry in the database, and returns a JSON response containing the report ID, file path, format, and status. **The actual report file generation (PDF/CSV/Excel) is planned for a future iteration.**
- POST /reports/exams/{exam_id} is designed to generate an exam-level report by aggregating all StudentActivity entries for the given exam and producing a Report record. **Currently, this endpoint creates the database record but does not generate the actual report file.**

These APIs establish the data model and workflow structure for report management, but the actual document generation logic will be implemented in subsequent iterations.

4.1.3 Next Steps

The completed work in this iteration establishes the core dashboard infrastructure and API structure for reports. Upcoming work will:

- Connect the Investigator Portal to live dashboard and report APIs instead of mock data.
- Implement actual report file generation (PDF, CSV, Excel) for incident and exam reports.
- Integrate the video processing pipeline (camera ingestion, AI models, and real-time cheating detection) into the monitoring endpoints.
- Finalize and expose the Student Portal, including student-facing views of violation histories and exam schedules.

4.2 External APIs/SDKs

This section describes the third-party APIs/SDKs used in the **ForeSyte** project implementation.

4.2.1 External APIs and SDKs Table

Table 4.1: External APIs and SDKs for ForeSyte

API and version	Description	Purpose of usage	API endpoint/function/class used
Google OAuth 2.0 (via Authlib 1.6.5)	Google authentication and authorization service	User authentication using Google accounts for single sign-on (SSO)	<code>https://accounts.google.com/.well-known/openid-configuration</code> , <code>oauth.register()</code> , <code>google.authorize_redirect()</code> , <code>google.authorize_access_token()</code>
OpenCV (cv2) 4.12.0.88	Computer vision and image processing library	Video stream processing, frame extraction from live CCTV feeds and recorded videos, image analysis for behavior detection	<code>cv2.VideoCapture()</code> , <code>cv2.imread()</code> , <code>cv2.CAP_PROP_FRAME_COUNT</code> , <code>cv2.CAP_PROP_BUFFERSIZE</code>
PDFplumber 0.11.7	PDF text and table extraction library	Parsing seating plan PDFs to extract student information, roll numbers, and exam schedules	<code>pdfplumber.open()</code> , <code>page.extract_tables()</code> , <code>page.extract_text()</code>
Requests 2.32.5	HTTP library for making API calls	Fetching video streams from phone cameras, proxying video streams, testing connection endpoints	<code>requests.get()</code> , <code>requests.exceptions.Timeout</code> , <code>requests.exceptions.ConnectionError</code>
Python-JOSE (jose) 3.5.0	JSON Web Token (JWT) implementation	Creating and validating JWT access tokens for user authentication and authorization	<code>jwt.encode()</code> , <code>jwt.decode()</code> , <code>JWTError</code>
Pillow (PIL) 11.3.0	Python Imaging Library	Image processing and manipulation for video frames and uploaded images	Image processing utilities (used indirectly through OpenCV)
NumPy 2.2.6	Numerical computing library	Array operations and mathematical computations for image/video processing	<code>numpy</code> arrays for image data manipulation

4.2.2 Detailed Usage

Google OAuth 2.0

- **Location:** `src/database/auth.py`
- **Implementation:** Uses Authlib library to integrate with Google's OAuth 2.0 service
- **Endpoints Used:**

- `https://accounts.google.com/.well-known/openid-configuration`
 - OpenID Connect discovery endpoint
- `/auth/google` – Initiates Google OAuth flow
- `/auth/google/callback` – Handles OAuth callback and user authentication

- **Functions:**

- `oauth.register()` – Registers Google OAuth client
- `google.authorize_redirect()` – Redirects user to Google login
- `google.authorize_access_token()` – Exchanges authorization code for access token

OpenCV (cv2)

- **Location:**

- `src/app/video_processing/stream_handler.py`
- `src/app/video_processing/processor.py`
- `src/app/live_phone_feeds/phone_stream_receiver.py`

- **Purpose:**

- Capturing video frames from live streams and recorded videos
- Processing video streams for behavior analysis
- Frame extraction for AI-based detection

- **Key Functions:**

- `cv2.VideoCapture()` – Opens video file or stream URL
- `cap.read()` – Reads frames from video
- `cv2.imread()` – Loads image frames for processing
- `cap.get(cv2.CAP_PROP_FRAME_COUNT)` – Gets total frame count

PDFplumber

- **Location:**

- `src/app/seating_plan/upload_plan.py`
- `import_students_from_pdf.py`

- **Purpose:**

- Extracting student information from seating plan PDFs
- Parsing exam schedules and room assignments
- Extracting roll numbers and student names from PDF tables
- **Key Functions:**
 - `pdfplumber.open()` – Opens PDF file
 - `page.extract_tables()` – Extracts table data from PDF pages
 - `page.extract_text()` – Extracts text content from PDF pages

Requests

- **Location:**
 - `src/database/api/stream_proxy.py`
 - `src/app/live_phone_feeds/phone_stream_receiver.py`
 - `src/app/live_phone_feeds/connection_tester.py`
- **Purpose:**
 - Making HTTP GET requests to fetch video streams
 - Proxying video streams to frontend
 - Testing connectivity to phone camera endpoints
- **Key Functions:**
 - `requests.get()` – Makes HTTP GET requests
 - `requests.exceptions.Timeout` – Handles timeout errors
 - `requests.exceptions.ConnectionError` – Handles connection errors

Python-JOSE

- **Location:** `src/database/auth.py`
- **Purpose:**
 - Encoding JWT tokens for authenticated users
 - Decoding and validating JWT tokens in protected routes
 - Managing user session tokens
- **Key Functions:**
 - `jwt.encode()` – Creates JWT access tokens
 - `jwt.decode()` – Validates and decodes JWT tokens
 - `JWTError` – Exception handling for invalid tokens

Pillow (PIL)

- **Location:** Used indirectly through OpenCV and other image processing operations
- **Purpose:** Image manipulation and processing utilities

NumPy

- **Location:** Used in video processing modules
- **Purpose:** Array operations for image data and mathematical computations in computer vision tasks

4.3 Algorithm Design

This section summarizes the main algorithms and data flows used to support the implemented features in ForeSyte.

4.3.1 Admin Dashboard Aggregation

The Admin Dashboard relies on server-side aggregation logic to compute statistics for different time windows. The high-level algorithm for computing dashboard statistics is:

1. **Determine Time Window:** Given $\text{period} \in \{\text{today}, \text{week}, \text{month}\}$, compute the current window (start_date) and the immediately preceding window (prev_start , prev_end) for trend comparison.
2. **Count Active Exams:** Query Exam where exam_date is greater than or equal to the current date. Perform the same query on the previous window to obtain prev_active_exams .
3. **Count Incidents and Students:**
 - Incidents: count `StudentActivity` records with timestamps within the current window.
 - Previous incidents: count `StudentActivity` records within the previous window.
 - Students monitored: count distinct `Seat.student_id` in rooms and exams that fall inside the current and previous windows.
4. **Compute Average Exam Duration:** For all exams in the current window with non-null start and end times, compute the duration in minutes and average the result.

5. **Compute Trends:** For each metric (incidents, students, exams), compute percentage change between current and previous counts, handling the edge case when the previous value is zero.
6. **Return Structured Response:** Package all calculated values into a `DashboardStats` object and return it to the client.

On the client side, the React dashboard:

- Calls `/dashboard/stats` to populate the metric cards.
- Calls `/dashboard/activity` to render a bar chart comparing incidents and exams per time bucket.
- Calls `/dashboard/recent-incidents` to display the latest suspicious activities with relative time labels.

4.3.2 Report Generation Workflow (Planned Design)

The report generation workflow is designed to be configurable and format-agnostic. **Note:** The following describes the planned design; actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration.

Incident Reports (Planned)

1. The investigator selects one or more incident IDs and chooses an export format (e.g., PDF, CSV, JSON), with options such as including video evidence links.
2. The front-end constructs an `IncidentReportRequest` and sends it to `POST /reports/incidents`.
3. The backend validates permissions, resolves the incident IDs into `StudentActivity` records, ensures there is an associated `Violation`, and generates a unique report filename.
4. A new `Report` record is inserted in the database, and the API returns the report ID, file path, format, and status to the client. **In the current iteration, the actual report file is not generated; only the database record is created.**

Exam Reports (Planned)

1. The investigator selects an exam and desired output format from the Reports Page.
2. The front-end sends an `ExamReportRequest` to `POST /reports/exams/{exam_id}`.

3. The backend verifies the exam, aggregates its related `StudentActivity` records, ensures a linked `Violation` exists, and creates a `Report` entry in the database.
4. The API responds with the report metadata. **The actual report file generation (PDF/CSV/Excel) will be implemented in a future iteration.**

4.3.3 Investigator Violation Review Flow

The Investigator Dashboard is designed to give investigators a structured way to triage violations:

1. Load dashboard summary counters (total violations, pending reviews, resolved today, active exams).
2. Display a list of recent activities with type, severity, and status tags.
3. Allow investigators to toggle the period filter to focus on today, week, or month.
4. Provide a *Review* action for each activity which, in the next iteration, will navigate to a detailed case view and allow updating violation status and adding investigation notes.

4.3.4 Seating Plan Upload and Processing

The seating plan upload algorithm processes PDF files containing exam schedules and extracts room, student, and seat assignment information. The algorithm handles PDF parsing, data extraction, database operations, and visualization generation. The high-level pseudocode for this process is:

```

1  START timer

    READ uploaded PDF bytes
    EXTRACT full text from all pages
    NORMALIZE text (remove extra whitespace)

6
    DEFINE block_pattern to detect each exam block inside PDF
    FIND all exam blocks using regex

    CREATE empty list: matching_exams

11
    FOR each exam block:
        EXTRACT exam date
        EXTRACT exam time slot

```

```

    EXTRACT room number
16 IF failed -> continue

    CHECK if user-specified room_no matches (if provided)
    CHECK if user-specified time_slot matches (if provided)
    IF mismatch -> continue

21 EXTRACT course name + section
    EXTRACT invigilator name

    FIND all students using regex:
26     serial_no, roll_no, name, seat_no

    ADD exam entry to matching_exams list:
        date, time, course, section, room, invigilator,
        student_list, total_students, uploaded_at

31 IF matching_exams is empty:
    RETURN "No seating plan found"

    SELECT first exam -> selected_exam
36 STORE selected_exam in global latest_room_data

    PARSE exam_date, start_time, end_time using parse_date_time()

    -----
41 DATABASE OPERATIONS
    -----

    # --- EXAM ---
    QUERY Exam table:
46     WHERE course == selected_exam.course
        AND exam_date == exam_date
        AND start_time == start_time

    IF exam exists:
51     use existing
    ELSE:
        CREATE new Exam object
        COMMIT to DB

56 # --- ROOM ---
    Split room number into block + room_num
    QUERY Room table:
        WHERE room_number == room_num AND exam_id == exam.exam_id

61 IF room exists:
    use existing

```

```

ELSE:
    CREATE Room with:
        room_number, block, total_seats, exam_id, camera_id
66 COMMIT to DB

# --- STUDENTS + SEATS ---
FOR each student in selected_exam.students:
    roll_number = student.roll_no
71 name = student.name

    QUERY Student table WHERE roll_number matches
    IF not exists:
        CREATE new Student(name, email_from_roll, roll_number)
76 COMMIT

    seat_number = student.seat_no

    QUERY Seat table WHERE (room_id, seat_number)
81 IF exists:
        UPDATE seat.student_id
    ELSE:
        CREATE new Seat(room_id, seat_number, student_id)

86 COMMIT all changes

-----
SAVE JSON EXPORT
-----

91 GENERATE timestamp-based filename
SAVE selected_exam as JSON file in extractions folder

-----
96 VISUALIZATION PROCESS
-----

LOAD CCTV frame image
LOAD seat_map.json (polygon coordinates)
101
CALCULATE max_column in seating plan:
    For each student seat (C#R#)
        Extract column number
        Track the maximum found
106
DEFINE mapping rules based on max_column:
    if max_col == 6 -> map 6 columns to 1,3,5,6,8,10
    if max_col == 5 -> map 5 columns to 1,4,6,8,10
    else -> identity mapping

```

```

111  DEFINE function map_seat_to_seat_map(seat_no):
      Extract C#, R#
      Convert input col -> mapped column
      RETURN seat_map ID like seat_cXrY
116
      CREATE:
          filled_seats = all seat IDs containing students
          student_seat_map = map seat_id -> student

121  FOR each seat_id in seat_map:
      IF seat_id NOT in filled_seats -> skip

      GET polygon points
      DRAW polygon in green

126
      FIND student assigned
      CALCULATE centroid
      DRAW student name + roll number at centroid

131  SAVE annotated image in extractions folder

      -----
      RETURN RESPONSE
      -----

136
      RETURN:
          message
          exam_date
          exam_time
141      course
          room_no
          students_count
          exam_id
          room_id
146      json_file path
          annotated_image path
          processing_time

```

The algorithm processes seating plan PDFs through several stages: (1) **PDF Parsing and Text Extraction**, where the uploaded PDF is read and normalized; (2) **Pattern Matching**, where exam blocks are identified using regex patterns and filtered by user-specified room number and time slot; (3) **Data Extraction**, where course information, invigilator details, and student seat assignments are extracted; (4) **Database Operations**, where Exam, Room, Student, and Seat records are created or updated; (5) **JSON Export**, where the extracted data is saved as a JSON file; and (6) **Visualization**, where the seating plan is

overlaid on a CCTV frame image with student names and roll numbers annotated at their respective seat positions.

4.4 Testing Details

Testing for this iteration concentrates on validating backend APIs, role-based access control, and the correctness of aggregated statistics, along with front-end behaviour under typical and edge-case conditions.

4.4.1 Unit Testing

4.4.1.1 Investigator Portal Testing

Table 4.2: Unit Testing for Investigator Portal

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC001	Verify dashboard renders header and sidebar correctly	Valid token exists in sessionStorage	1. Insert token in storage. 2. Mock Stats, Violations, Incidents APIs. 3. Render Investigator Dashboard	Token: "test-token"	Header and sidebar appear on screen	As Expected	Pass
TC002	Dashboard statistics load correctly	API returns valid stats + violations	1. Mock Stats API: active_exams=3. 2. Mock Violations API: 2 violations. 3. Render component	Stats: {active_exams: 3}. 2 violations	UI shows: Active Exams = 3, Total Violations = 2, Pending Reviews ≥ 1	As Expected	Pass
TC003	Dashboard displays recent activities correctly	API returns incident data	1. Mock Incidents API with type='Cheating'. 2. Render dashboard. 3. Verify activity display	Incident: {type: "Cheating", student: "John Doe", severity: "high"}	Recent activities section shows "Cheating" as h3 heading	As Expected	Pass
TC004	Period filter changes dashboard data	Dashboard loaded with initial data	1. Render dashboard. 2. Click "This Week" button. 3. Verify API call	Initial period: "today"	getDashboardStats called with period='week' parameter	As Expected	Pass
TC005	Violations page renders and displays violations list	Valid token and API returns violations	1. Mock getViolations API. 2. Render ViolationsPage. 3. Verify table display	Violation: {student_name: "John Doe", type: "cheating", status: "pending"}	Student name and violation type appear in table row	As Expected	Pass
TC006	Violations can be filtered by search term	Multiple violations loaded	1. Render page with 2 violations. 2. Enter "Jane" in search input. 3. Verify filtered results	Students: "John Doe", "Jane Smith"	Only "Jane Smith" violation visible, "John Doe" hidden	As Expected	Pass
TC007	Violations can be filtered by status	Violations with different statuses loaded	1. Render page with pending and confirmed violations. 2. Select "pending" in status filter. 3. Verify filtered results	Statuses: "pending", "confirmed"	Only pending violation visible, confirmed violation hidden	As Expected	Pass
TC008	Violation details modal opens and confirms violation	Violation exists in system	1. Render ViolationsPage. 2. Click "View" button. 3. Click "Confirm" in modal. 4. Verify API call	Violation ID: "v1"	Modal opens with "Violation Details" heading; confirmViolation API called with ID "v1"	As Expected	Pass

Table 4.3: Unit Testing for Investigator Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC009	Investigator activities page displays and filters by type	Multiple activity types exist	1. Mock activities API with phone_usage and absent_from_zone. 2. Render page. 3. Select "phone_usage" filter. 4. Verify display	Activities: phone_usage, absent_from_zone	phone_usage appears in table cell (td element) after filtering	As Expected	Pass
TC010	Student activities page displays and filters by search	Multiple student activities loaded	1. Mock API with activities for John and Jane. 2. Render page. 3. Search for "Jane". 4. Verify results	Students: "John Doe", "Jane Smith"	Only Jane's activities visible; phone_usage type shown in table cell	As Expected	Pass
TC011	Reports page opens generate modal and displays reports list	Valid token and reports exist	1. Mock getReports API. 2. Render ReportsPage. 3. Click "Generate New Report" button. 4. Verify modal	Report: {type: "incident", exam: "Calculus I"}	Report displayed; modal opens with "Generate New Report" heading	As Expected	Pass
TC012	All pages handle API errors gracefully	API returns network error	1. Mock API to reject with error. 2. Render any investigator page. 3. Verify error handling	Error: "Network error"	Page renders with header visible; no crash or unhandled exception	As Expected	Pass

4.4.1.2 Student Portal Testing

Table 4.4: Unit Testing for Student Portal

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC001	Verify dashboard renders header and sidebar correctly	Valid token exists in sessionStorage	1. Insert token in storage. 2. Mock Profile, Activities, Violations APIs. 3. Render Student Dashboard	Token: "test-token", User ID: "student-123"	Header and sidebar appear on screen	As Expected	Pass
TC002	Dashboard displays welcome message with student name	API returns valid profile data	1. Mock getStudentProfile API with name="John Doe". 2. Mock Activities and Violations APIs. 3. Render dashboard	Profile: {name: "John Doe", email: "john@test.com", roll_number: "22I-1234"}	UI shows: "Welcome back, John Doe" message	As Expected	Pass
TC003	Dashboard displays disciplinary cases count correctly	API returns violations data	1. Mock Profile API. 2. Mock getStudentViolations with 2 violations. 3. Render dashboard	Violations: [{violation_id: "v1", status: "pending"}, {violation_id: "v2", status: "confirmed"}]	UI shows: Disciplinary Cases = 2	As Expected	Pass
TC004	Dashboard displays recent activities list	API returns activities data	1. Mock Profile API. 2. Mock getStudentActivitiesByStudentId with head_movement activity. 3. Render dashboard	Activities: [{activity_type: "head_movement", severity: "high", timestamp: "2024-01-01T10:00:00Z"}]	Recent activities section shows "head_movement" activity	As Expected	Pass
TC005	Disciplinary records page displays violations list	Valid token and API returns violations	1. Mock getStudentProfile API. 2. Mock getStudentViolations with cheating violation. 3. Render dcRecords page	Violation: {violation_type: "cheating", severity: 3, status: "pending", timestamp: "2024-01-01T10:00:00Z"}	Violation type "cheating" appears as heading (h3 element)	As Expected	Pass
TC006	Disciplinary records can be expanded to view details	Violations loaded on page	1. Render dcRecords page with violations. 2. Find clickable record card. 3. Click on record. 4. Verify details display	Violation: {violation_type: "cheating", status: "pending"}	Record expands; "Details" text appears on screen	As Expected	Pass
TC007	Profile page allows editing student name	Valid profile data loaded	1. Mock getStudentProfile with name="John Doe". 2. Render profile page. 3. Click "Edit" button. 4. Change name to "Jane Doe". 5. Click save button. 6. Verify API call	Original: "John Doe", Updated: "Jane Doe"	updateStudentProfile API called with name="Jane Doe"; change saved successfully	As Expected	Pass

Table 4.5: Unit Testing for Student Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC008	Profile page cancels editing when cancel button clicked	Profile in edit mode with changes	1. Render profile page. 2. Click "Edit" button. 3. Change name to "Jane Doe". 4. Click cancel button. 5. Verify revert	Changed: "Jane Doe", Original: "John Doe"	Input field disappears; original name "John Doe" displayed; updateStudentProfile not called	As Expected	Pass
TC009	Settings page allows password change with validation	Valid profile and matching passwords	1. Mock getStudentProfile API. 2. Render settings page. 3. Switch to Password tab. 4. Enter current="oldpass123", new="newpass123", confirm="newpass123". 5. Click "Update Password"	Passwords: {current: "oldpass123", new: "newpass123", confirm: "newpass123"}	changePassword API called with current_password="oldpass123" and new_password="newpass123"	As Expected	Pass
TC010	Upload page displays uploaded documents and allows file removal	Documents exist in system	1. Mock getStudentDocuments with id_card.pdf. 2. Render upload page. 3. Find remove button. 4. Click remove. 5. Verify removal	Document: {filename: "id_card.pdf", size: 1024000, status: "verified"}	Document "id_card.pdf" appears initially; after clicking remove button, document no longer visible	As Expected	Pass

4.4.1.3 Admin Portal Testing

Table 4.6: Unit Testing for Admin Portal

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC001	Verify dashboard renders header and sidebar correctly	Valid token exists in sessionStorage	1. Insert token in storage. 2. Mock Stats, Activity, Incidents APIs. 3. Render Admin Dashboard	Token: "test-token", User type: "admin"	Header and sidebar appear on screen	As Expected	Pass
TC002	Dashboard displays welcome message and statistics	API returns valid stats data	1. Mock getDashboardStats with active_exams=5. 2. Mock other APIs. 3. Render dashboard	Stats: {active_exams: 5, incidents_detected: 10, students_monitored: 150}	UI shows: "Welcome back" message, Active Exams = 5, Incidents = 10	As Expected	Pass
TC003	Dashboard displays recent incidents list	API returns incidents data	1. Mock getRecentIncidents with "Suspicious Movement" incident. 2. Render dashboard. 3. Verify display	Incident: {student_name: "John Doe", type: "Suspicious Movement", severity: "high"}	Recent incidents section shows "John Doe" name	As Expected	Pass
TC004	Analytics page fetches and displays metrics correctly	Valid token and API returns metrics	1. Mock getAnalyticsMetrics with total_incidents=128. 2. Mock other analytics APIs. 3. Render AnalyticsPage	Metrics: {total_incidents: 128, resolution_rate: 87.5, avg_response_time: 4.2}	UI shows: 128 incidents, 87.5% resolution rate	As Expected	Pass
TC005	Analytics page displays incident type distribution	API returns distribution data	1. Mock getIncidentTypeDistribution with types. 2. Render AnalyticsPage. 3. Verify display	Types: [{type: "Suspicious Movement", count: 45}, {type: "Device Detected", count: 32}]	"Suspicious Movement" and "Device Detected" appear on screen	As Expected	Pass
TC006	Exams page renders and displays exams list	Valid token and API returns exams	1. Mock getExams with Calculus I and Data Structures. 2. Render ExamsPage. 3. Verify display	Exams: [{name: "Calculus I", course_code: "MATH101"}, {name: "Data Structures", course_code: "CS201"}]	"Calculus I" and "Data Structures" appear on screen	As Expected	Pass
TC007	Exams page opens create modal when add button clicked	Exams page loaded	1. Render ExamsPage. 2. Click "Create Exam" button. 3. Verify modal display	Initial exams: []	Modal opens with "Create New Exam" heading	As Expected	Pass

Table 4.7: Unit Testing for Admin Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC008	Exams page filters by status correctly	Multiple exams with different statuses loaded	1. Render ExamsPage. 2. Find status filter select. 3. Change to "active". 4. Verify API call	Statuses: "scheduled", "active", "completed"	getExams API called with status='active' parameter	As Expected	Pass
TC009	Login page handles successful authentication	Valid credentials provided	1. Mock fetch with success response. 2. Render LoginPage. 3. Enter email="admin@test.com" and password="password123". 4. Click submit button	Credentials: {email: "admin@test.com", password: "password123"}. Response: {access_token: "test-token", user_type: "admin"}	Login API called; token stored in sessionStorage; navigate to "/dashboard"	As Expected	Pass
TC010	Login page validates input and shows/hides password	Login form rendered	1. Render LoginPage. 2. Verify password input type is "password". 3. Click toggle button. 4. Verify type changes	Initial type: "password"	Password input type changes to "text" when toggle clicked, then back to "password"	As Expected	Pass
TC011	SignUp page validates password requirements	SignUp form rendered	1. Render SignUpPage. 2. Enter password="12345". 3. Enter confirm="12345". 4. Click submit. 5. Verify validation	Passwords: {password: "12345", confirm: "12345"}	Alert shown: "Password must be at least 6 characters long!"	As Expected	Pass
TC012	Users page fetches and displays users list	Valid token and API returns users	1. Mock getUsers with John Doe and Jane Smith. 2. Render UsersPage. 3. Verify display	Users: [{name: "John Doe", email: "john@test.com", user_type: "student"}, {name: "Jane Smith", email: "jane@test.com", user_type: "admin"}]	"John Doe" and "Jane Smith" appear on screen	As Expected	Pass
TC013	Users page opens add user modal when button clicked	Users page loaded	1. Mock getUsers API. 2. Render UsersPage. 3. Click "Add User" button. 4. Verify modal	Initial users: []	Modal opens with "Add New User" heading	As Expected	Pass
TC014	Incidents page renders and displays incidents list	Valid token and API returns incidents	1. Mock getIncidents with "Suspicious Movement" incident. 2. Render Incidents page. 3. Verify display	Incident: {student_name: "John Doe", type: "Suspicious Movement", severity: "high", status: "investigating"}	"John Doe" and "Suspicious Movement" appear on screen	As Expected	Pass

Table 4.8: Unit Testing for Admin Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC015	Monitoring page fetches and displays camera feeds	Valid token and API returns feeds	1. Mock getMonitoringFeeds with Room A101 and B205. 2. Render MonitoringPage. 3. Verify display	Feeds: [{room_name: "Room A101", status: "active"}, {room_name: "Room B205", status: "inactive"}]	"Room A101" and "Room B205" appear on screen	As Expected	Pass
TC016	Monitoring page opens phone camera monitor when button clicked	Monitoring page loaded	1. Mock getMonitoringFeeds API. 2. Render MonitoringPage. 3. Click "Start Phone Camera" button. 4. Verify modal	Feeds: []	Phone camera monitor component appears with testid "phone-camera-monitor"	As Expected	Pass
TC017	Reports page renders and displays reports list	Valid token and API returns reports	1. Mock getReportHistory with exam and incident reports. 2. Mock getExams and getIncidents. 3. Render ReportsPage	Reports: [{title: "Exam Report - Calculus I", type: "exam"}, {title: "Incident Report", type: "incident"}]	Report headings "Exam Report - Calculus I" and "Incident Report" appear	As Expected	Pass
TC018	Reports page opens generate modal when button clicked	Reports page loaded	1. Mock all reports APIs. 2. Render ReportsPage. 3. Click "Incident Report" button. 4. Verify modal	Reports: []	Modal opens with "Generate Incident Report" heading	As Expected	Pass
TC019	Seating Plans page fetches and displays plans list	Valid token and API returns plans	1. Mock getSeatingPlans with 2 PDF files. 2. Render SeatingPlansPage. 3. Verify display	Plans: [{filename: "seating_plan_2024.pdf"}, {filename: "seating_plan_2025.pdf"}]	"seating_plan_2024.pdf" and "seating_plan_2025.pdf" appear on screen	As Expected	Pass
TC020	Settings page displays tabs and allows logout	Valid session exists	1. Render SettingsPage. 2. Verify tabs display. 3. Switch to Security tab. 4. Click Logout button. 5. Verify navigation	Tabs: General, Notifications, Security, System	All tabs appear; logout clears session and navigates to "/login"	As Expected	Pass

Bibliography

- [1] Saravanan Arumugam. Deep learning-based smart invigilation system for enhanced exam integrity. *Proceedings of Engineering and Technology Innovation*, 29:99–115, 2025.
- [2] A. Bhatti and T. Aslam. Smart exam monitoring system using computer vision. *Journal of Emerging Technologies*, 2023.
- [3] A. Buccioli, S. Cicognani, and N. Montinari. Cheating in university exams: the relevance of social factors. *ResearchGate*, 2020.
- [4] M. Khalid and H. Shah. Impact of surveillance systems on academic integrity in higher education. *Journal of Educational Research*, 2021.
- [5] Mohammad M. Masud, Khadim Hayawi, Sujith Samuel Mathew, Temesgen Mikael Abraha, and Mai ElBarachi. Smart online exam proctoring assist for cheating detection. *ResearchGate*, 2025.
- [6] Reuben Moyo, Stanley Ndebvu, Michael Zimba, and Jimmy Mbelwa. A video-based detector for suspicious activity in examination with openpose. *Unknown Journal*, 2023.
- [7] –. Intelligent offline exam monitoring system (iems) for identifying suspicious behavior. *International Journal of Computer Applications*, 2025.

Appendix A

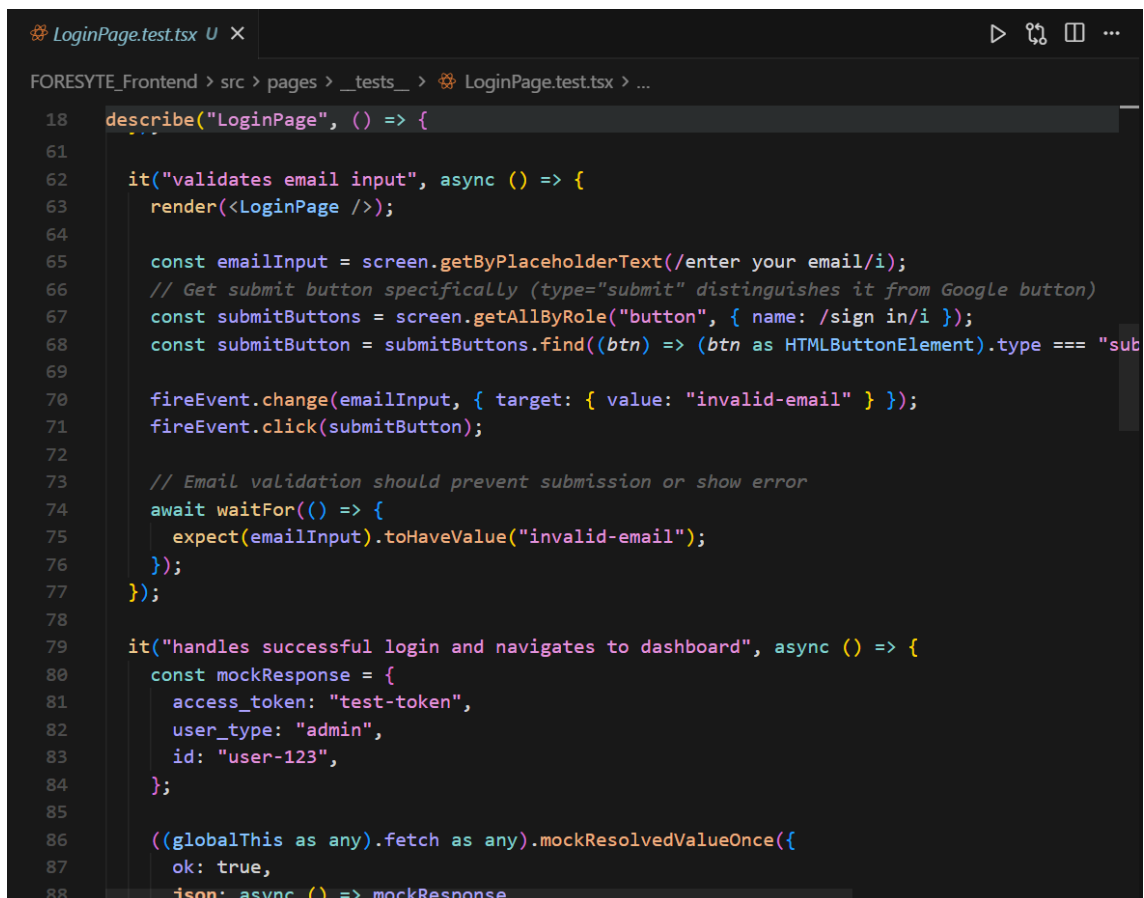
Appendices

A.1 Appendix A

A.1.1 Admin Portal Screenshots

A.1.1.1 Login Page

Figure A.1: Admin Portal Login Page

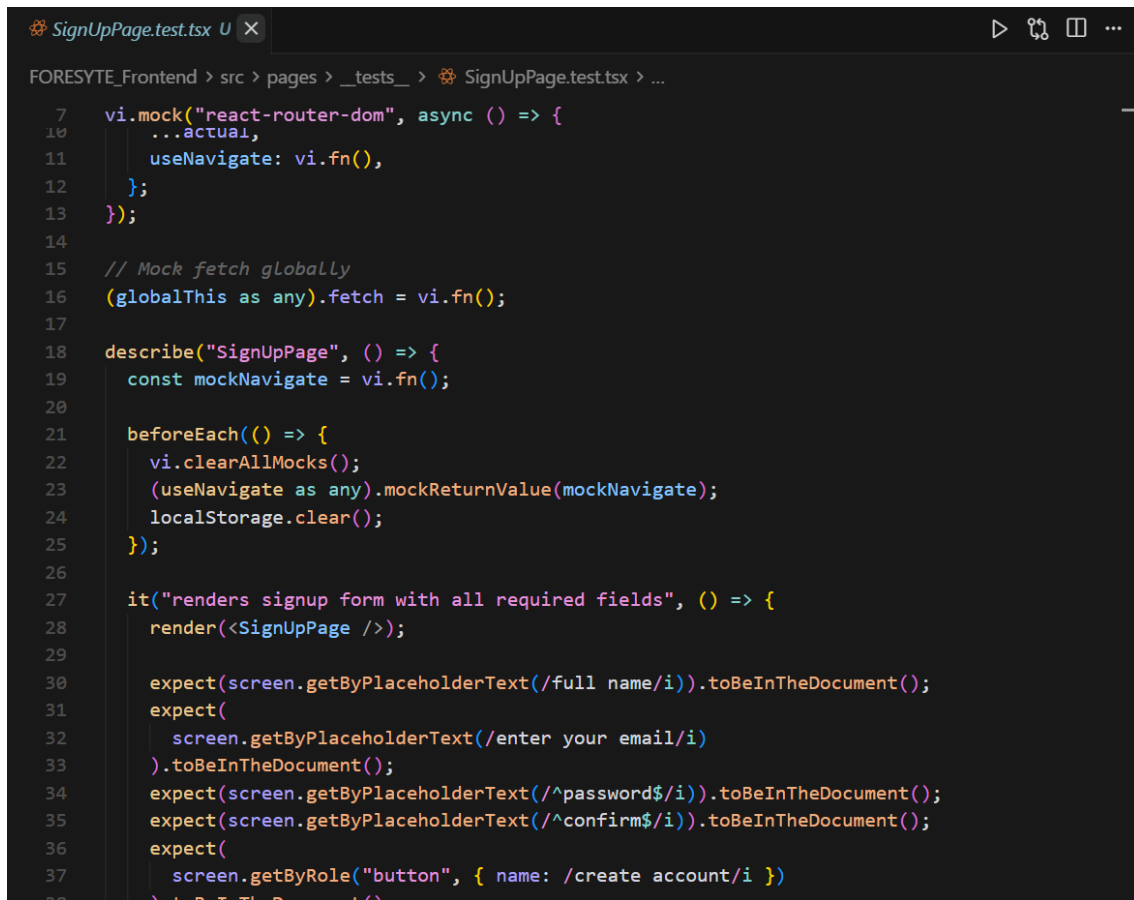


```
LoginPage.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > LoginPage.test.tsx > ...

18 describe("LoginPage", () => {
61
62   it("validates email input", async () => {
63     render(<LoginPage />);
64
65     const emailInput = screen.getByPlaceholderText(/enter your email/i);
66     // Get submit button specifically (type="submit" distinguishes it from Google button)
67     const submitButtons = screen.getAllByRole("button", { name: /sign in/i });
68     const submitButton = submitButtons.find((btn) => (btn as HTMLButtonElement).type === "submit");
69
70     fireEvent.change(emailInput, { target: { value: "invalid-email" } });
71     fireEvent.click(submitButton);
72
73     // Email validation should prevent submission or show error
74     await waitFor(() => {
75       expect(emailInput).toHaveValue("invalid-email");
76     });
77   });
78
79   it("handles successful login and navigates to dashboard", async () => {
80     const mockResponse = {
81       access_token: "test-token",
82       user_type: "admin",
83       id: "user-123",
84     };
85
86     ((globalThis as any).fetch as any).mockResolvedValueOnce({
87       ok: true,
88       json: async () => mockResponse,
```

A.1.1.2 Sign Up Page

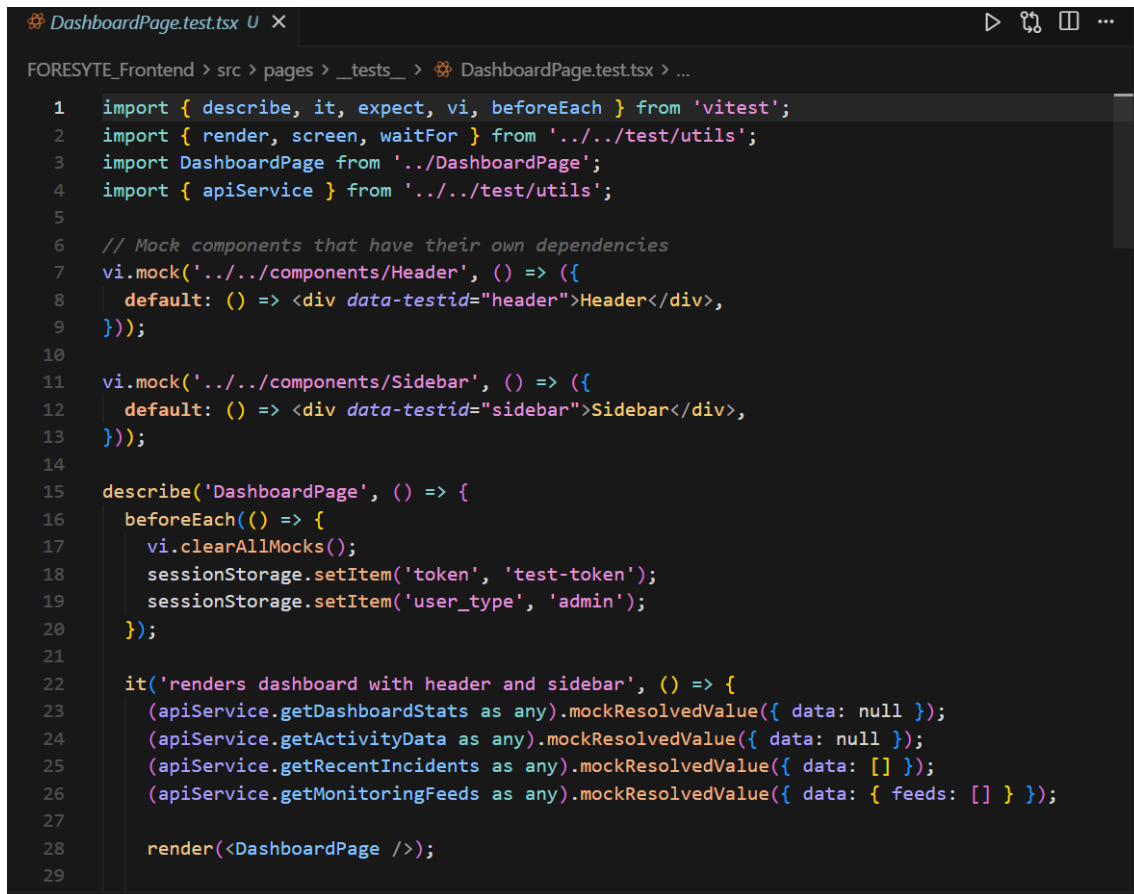
Figure A.2: Admin Portal Sign Up Page



```
7 vi.mock("react-router-dom", async () => {
10   ...actual,
11   useNavigate: vi.fn(),
12 });
13 });
14
15 // Mock fetch globally
16 (globalThis as any).fetch = vi.fn();
17
18 describe("SignUpPage", () => {
19   const mockNavigate = vi.fn();
20
21   beforeEach(() => {
22     vi.clearAllMocks();
23     (useNavigate as any).mockReturnValue(mockNavigate);
24     localStorage.clear();
25   });
26
27   it("renders signup form with all required fields", () => {
28     render(<SignUpPage />);
29
30     expect(screen.getByPlaceholderText(/full name/i)).toBeInTheDocument();
31     expect(
32       screen.getByPlaceholderText(/enter your email/i)
33     ).toBeInTheDocument();
34     expect(screen.getByPlaceholderText(/password$/i)).toBeInTheDocument();
35     expect(screen.getByPlaceholderText(/confirm$/i)).toBeInTheDocument();
36     expect(
37       screen.getByRole("button", { name: /create account/i })
38     ).toBeInTheDocument();
39   });
40 });
```

A.1.1.3 Dashboard

Figure A.3: Admin Portal Dashboard

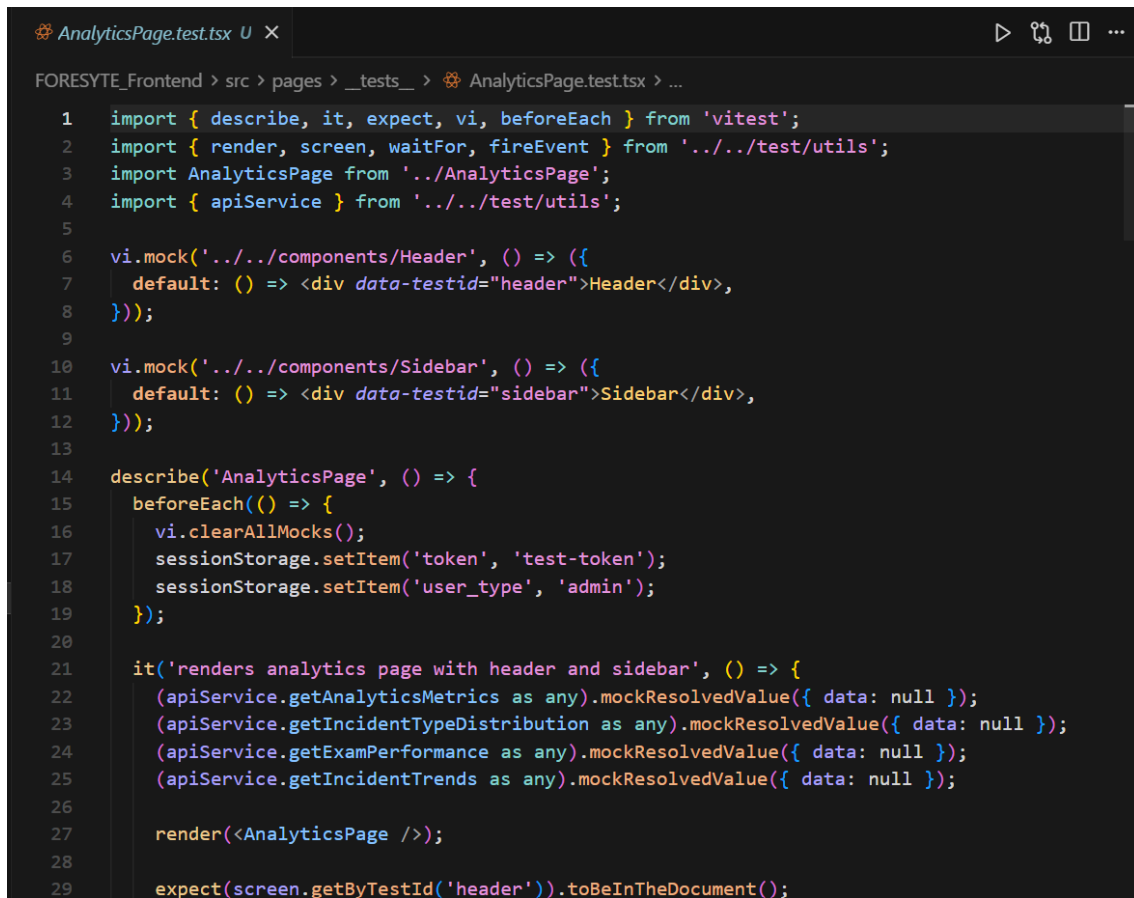


The screenshot shows a code editor with a file named `DashboardPage.test.tsx`. The code is a Jest test file for the `DashboardPage` component. It includes imports for `vitest` and `utils`, mocks for `Header` and `Sidebar` components, and a `describe` block for the `DashboardPage` component. The `describe` block contains a `beforeEach` hook that sets up the test environment and an `it` block that renders the `DashboardPage` component and asserts that it renders the header and sidebar.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor } from '../test/utils';
3 import DashboardPage from '../DashboardPage';
4 import { apiService } from '../test/utils';
5
6 // Mock components that have their own dependencies
7 vi.mock('../components/Header', () => ({
8   default: () => <div data-testid="header">Header</div>,
9 }));
10
11 vi.mock('../components/Sidebar', () => ({
12   default: () => <div data-testid="sidebar">Sidebar</div>,
13 }));
14
15 describe('DashboardPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.setItem('token', 'test-token');
19     sessionStorage.setItem('user_type', 'admin');
20   });
21
22   it('renders dashboard with header and sidebar', () => {
23     (apiService.getDashboardStats as any).mockResolvedValue({ data: null });
24     (apiService.getActivityData as any).mockResolvedValue({ data: null });
25     (apiService.getRecentIncidents as any).mockResolvedValue({ data: [] });
26     (apiService.getMonitoringFeeds as any).mockResolvedValue({ data: { feeds: [] } });
27
28     render(<DashboardPage />);
29   });
30 }
```

A.1.1.4 Analytics Page

Figure A.4: Admin Portal Analytics Page

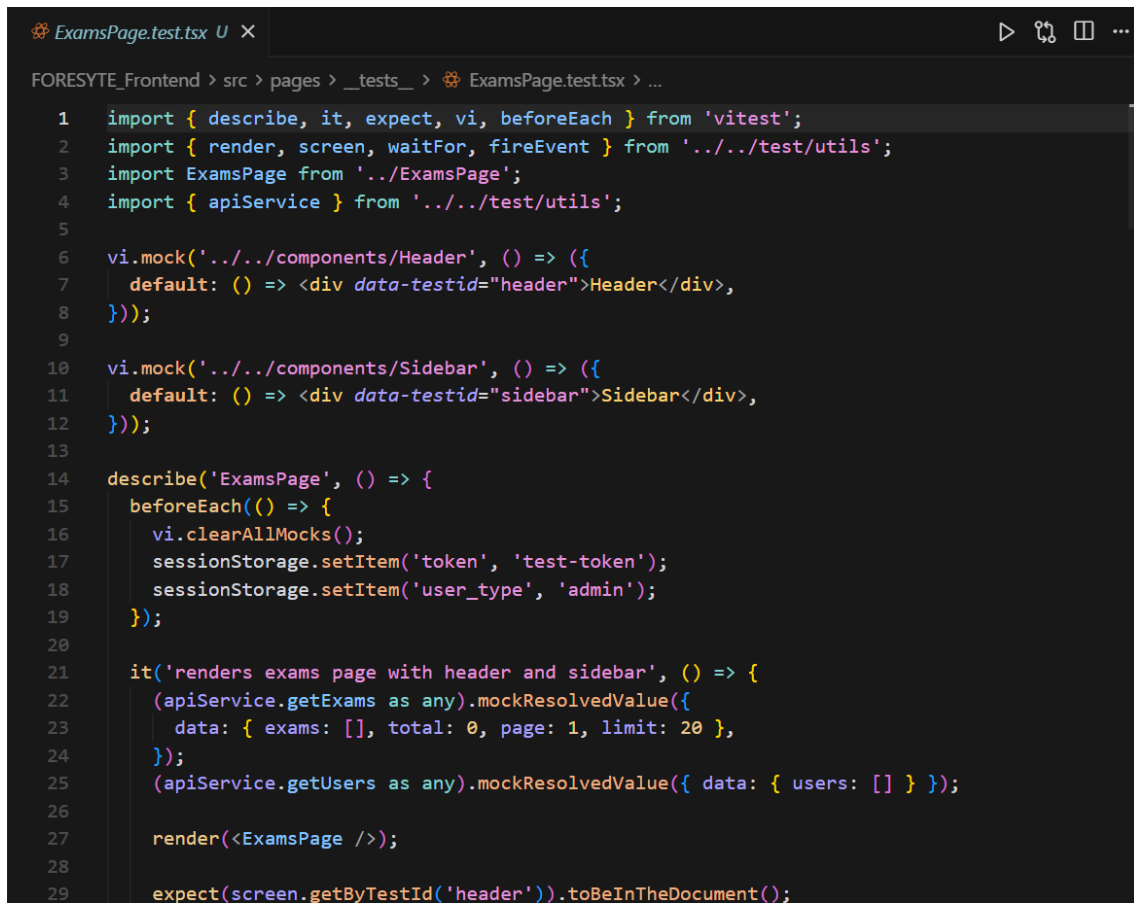


```
AnalyticsPage.test.tsx U X
FORESYTE_Frontend > src > pages > __tests__ > AnalyticsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import AnalyticsPage from '../AnalyticsPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 describe('AnalyticsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders analytics page with header and sidebar', () => {
22     (apiService.getAnalyticsMetrics as any).mockResolvedValue({ data: null });
23     (apiService.getIncidentTypeDistribution as any).mockResolvedValue({ data: null });
24     (apiService.getExamPerformance as any).mockResolvedValue({ data: null });
25     (apiService.getIncidentTrends as any).mockResolvedValue({ data: null });
26
27     render(<AnalyticsPage />);
28
29     expect(screen.getByTestId('header')).toBeInTheDocument();
```

A.1.1.5 Exams Page

Figure A.5: Admin Portal Exams Management Page

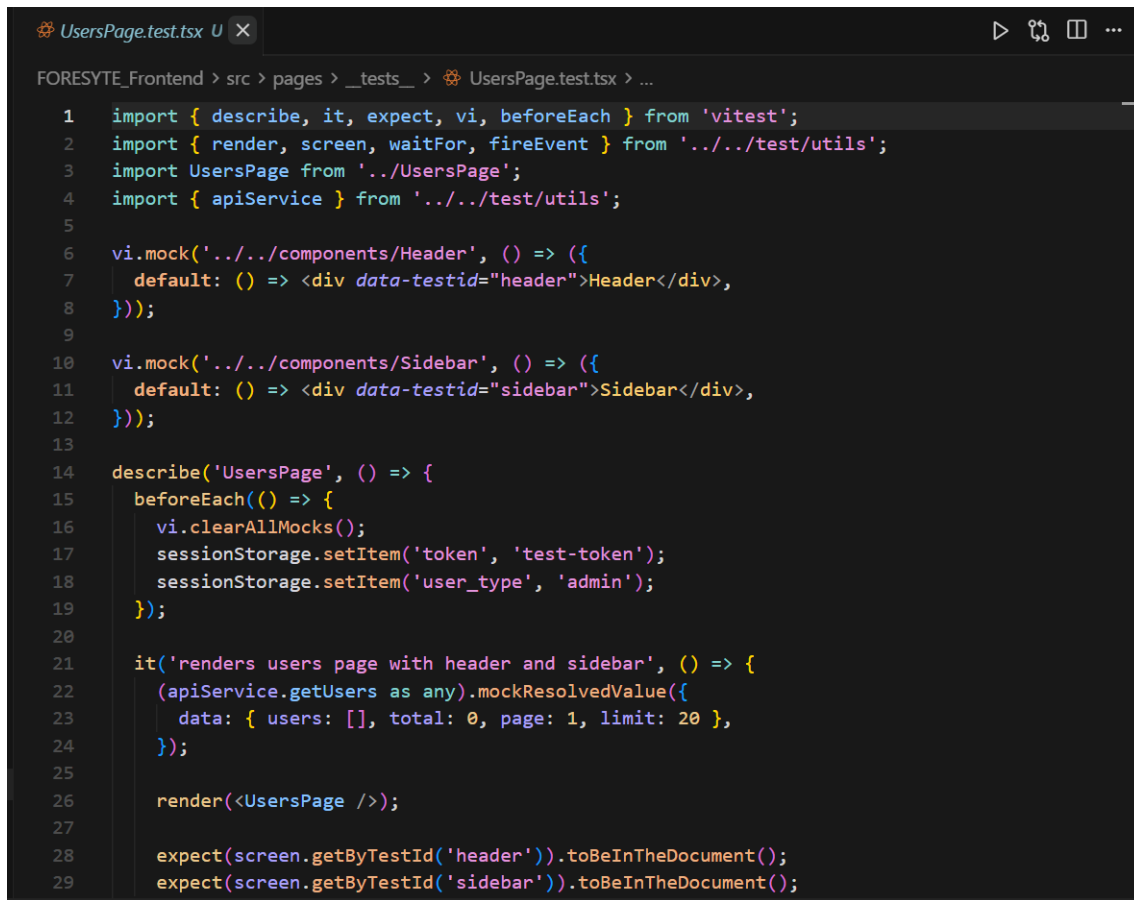


```
ExamsPage.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > ExamsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import ExamsPage from '../ExamsPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 describe('ExamsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders exams page with header and sidebar', () => {
22     (apiService.getExams as any).mockResolvedValue({
23       data: { exams: [], total: 0, page: 1, limit: 20 },
24     });
25     (apiService.getUsers as any).mockResolvedValue({ data: { users: [] } });
26
27     render(<ExamsPage />);
28
29     expect(screen.getByTestId('header')).toBeInTheDocument();
```

A.1.1.6 Users Page

Figure A.6: Admin Portal Users Management Page

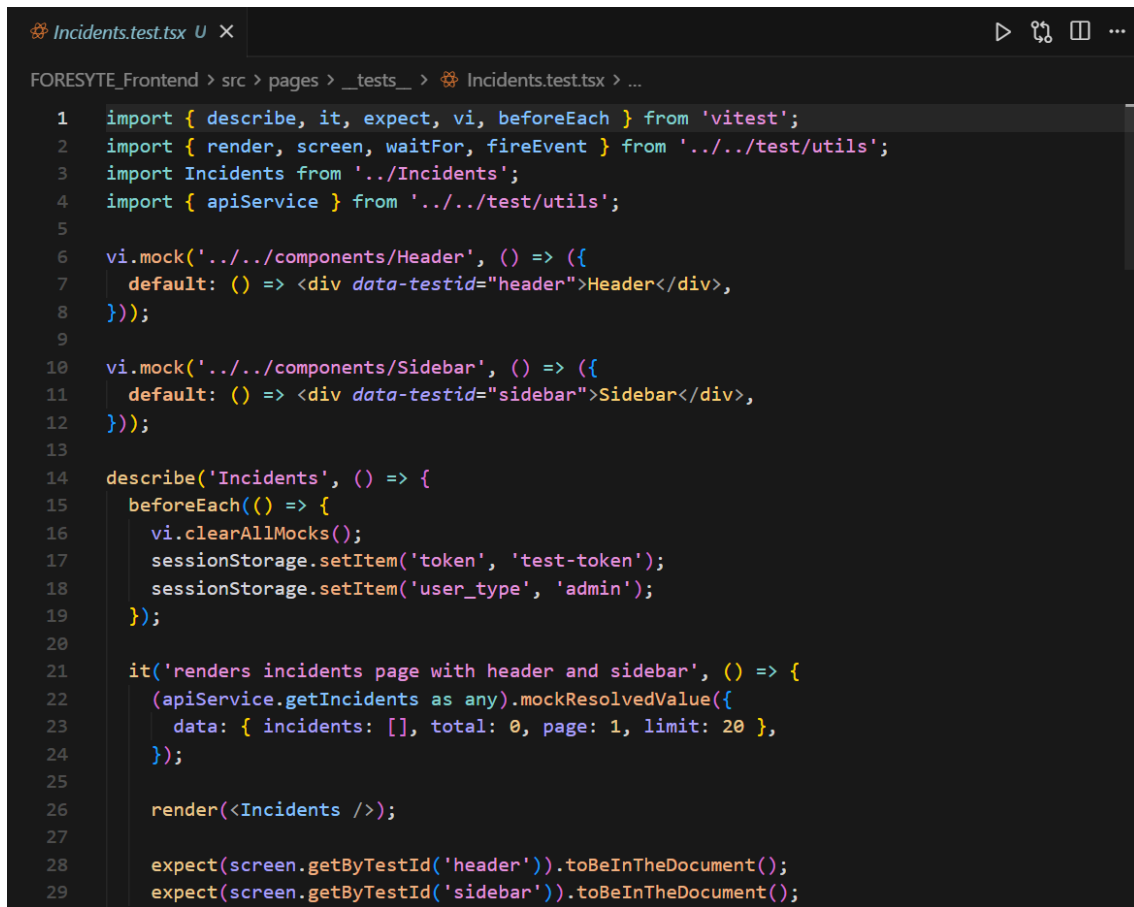


The image shows a code editor window with a file named `UsersPage.test.tsx`. The editor displays a series of test cases for the `UsersPage` component using `vitest` and `react-testing-library`. The tests include mocking the `Header` and `Sidebar` components, setting up session storage, and verifying that the page renders with the correct header and sidebar. The code is as follows:

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utls';
3 import UsersPage from '../UsersPage';
4 import { apiService } from '../test/utls';
5
6 vi.mock('../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 describe('UsersPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders users page with header and sidebar', () => {
22     (apiService.getUsers as any).mockResolvedValue({
23       data: { users: [], total: 0, page: 1, limit: 20 },
24     });
25
26     render(<UsersPage />);
27
28     expect(screen.getByTestId('header')).toBeInTheDocument();
29     expect(screen.getByTestId('sidebar')).toBeInTheDocument();
30   });
31 });
```

A.1.1.7 Incidents Page

Figure A.7: Admin Portal Incidents Page

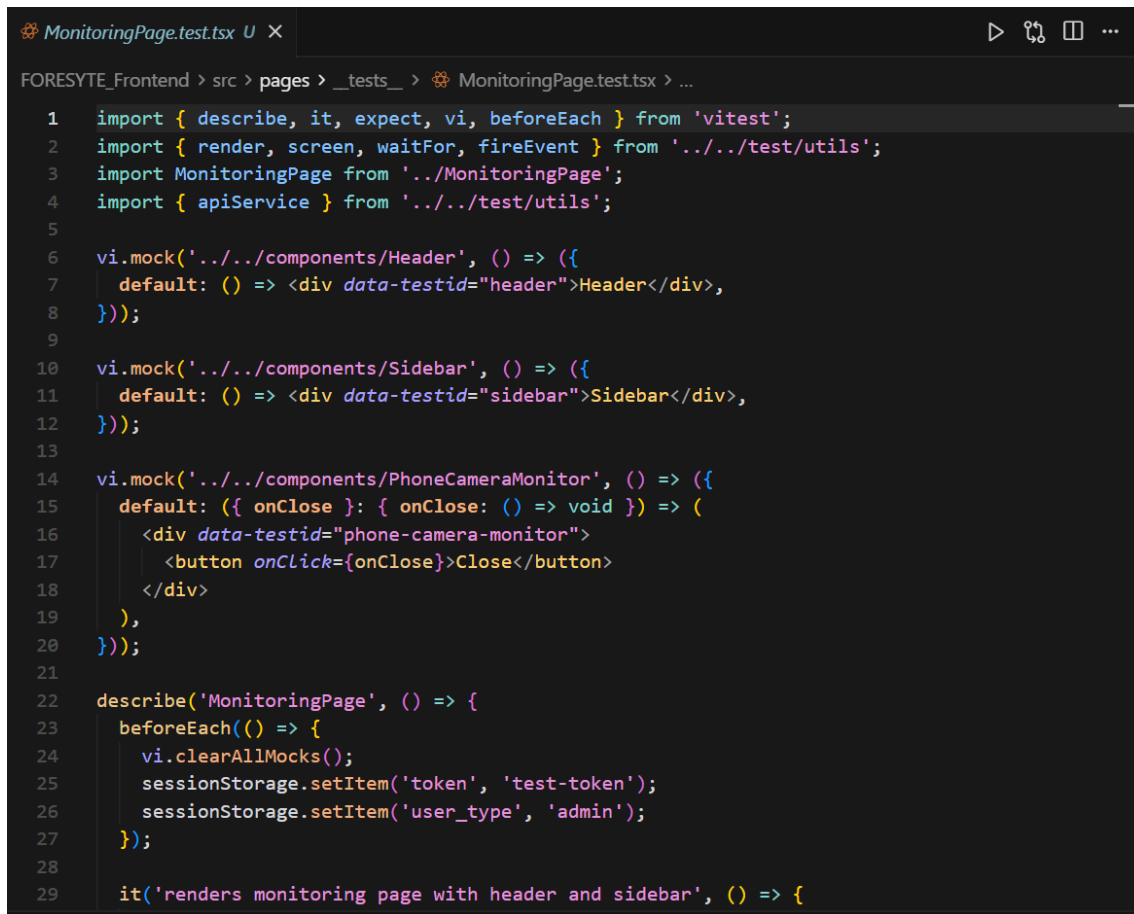


```
Incidents.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > Incidents.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import Incidents from '../Incidents';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 describe('Incidents', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders incidents page with header and sidebar', () => {
22     (apiService.getIncidents as any).mockResolvedValue({
23       data: { incidents: [], total: 0, page: 1, limit: 20 },
24     });
25
26     render(<Incidents />);
27
28     expect(screen.getByTestId('header')).toBeInTheDocument();
29     expect(screen.getByTestId('sidebar')).toBeInTheDocument();
30   });
31 });
```

A.1.1.8 Monitoring Page

Figure A.8: Admin Portal Monitoring Page

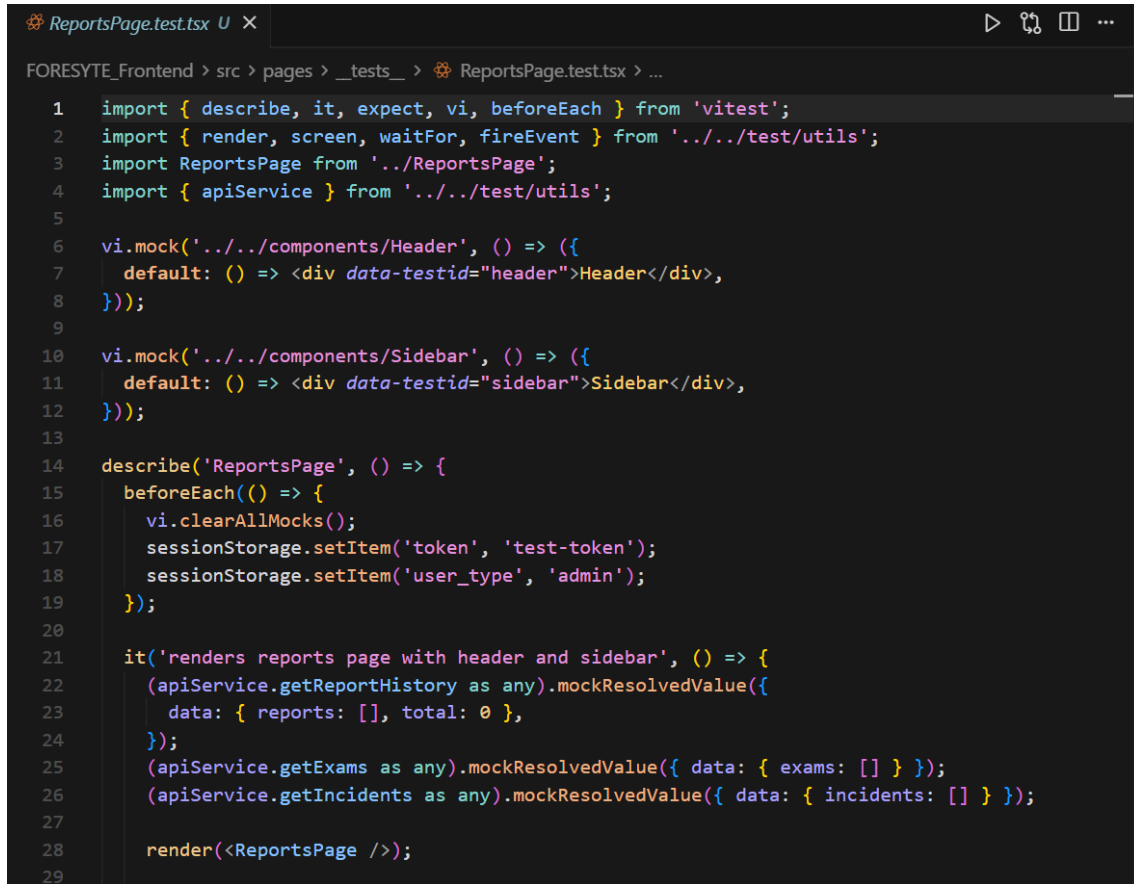


```
MonitoringPage.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > MonitoringPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import MonitoringPage from '../MonitoringPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 vi.mock('../components/PhoneCameraMonitor', () => ({
15   default: ({ onClose }: { onClose: () => void }) => (
16     <div data-testid="phone-camera-monitor">
17       <button onClick={onClose}>Close</button>
18     </div>
19   ),
20 });
21
22 describe('MonitoringPage', () => {
23   beforeEach(() => {
24     vi.clearAllMocks();
25     sessionStorage.setItem('token', 'test-token');
26     sessionStorage.setItem('user_type', 'admin');
27   });
28
29   it('renders monitoring page with header and sidebar', () => {
```

A.1.1.9 Reports Page

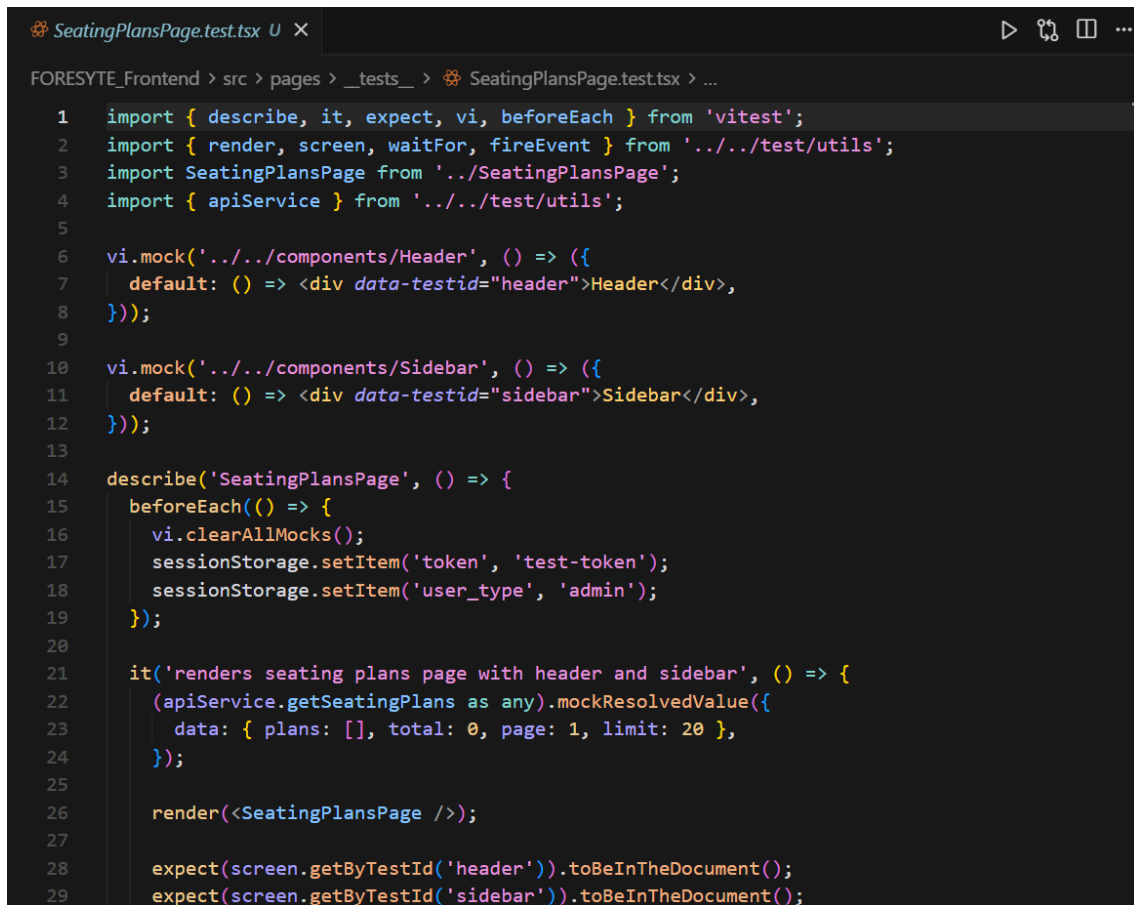
Figure A.9: Admin Portal Reports Page



```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utils';
3 import ReportsPage from '../ReportsPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 describe('ReportsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders reports page with header and sidebar', () => {
22     (apiService.getReportHistory as any).mockResolvedValue({
23       data: { reports: [], total: 0 },
24     });
25     (apiService.getExams as any).mockResolvedValue({ data: { exams: [] } });
26     (apiService.getIncidents as any).mockResolvedValue({ data: { incidents: [] } });
27
28     render(<ReportsPage />);
29   });
30 }
```

A.1.1.10 Seating Plan Page

Figure A.10: Admin Portal Seating Plan Management Page

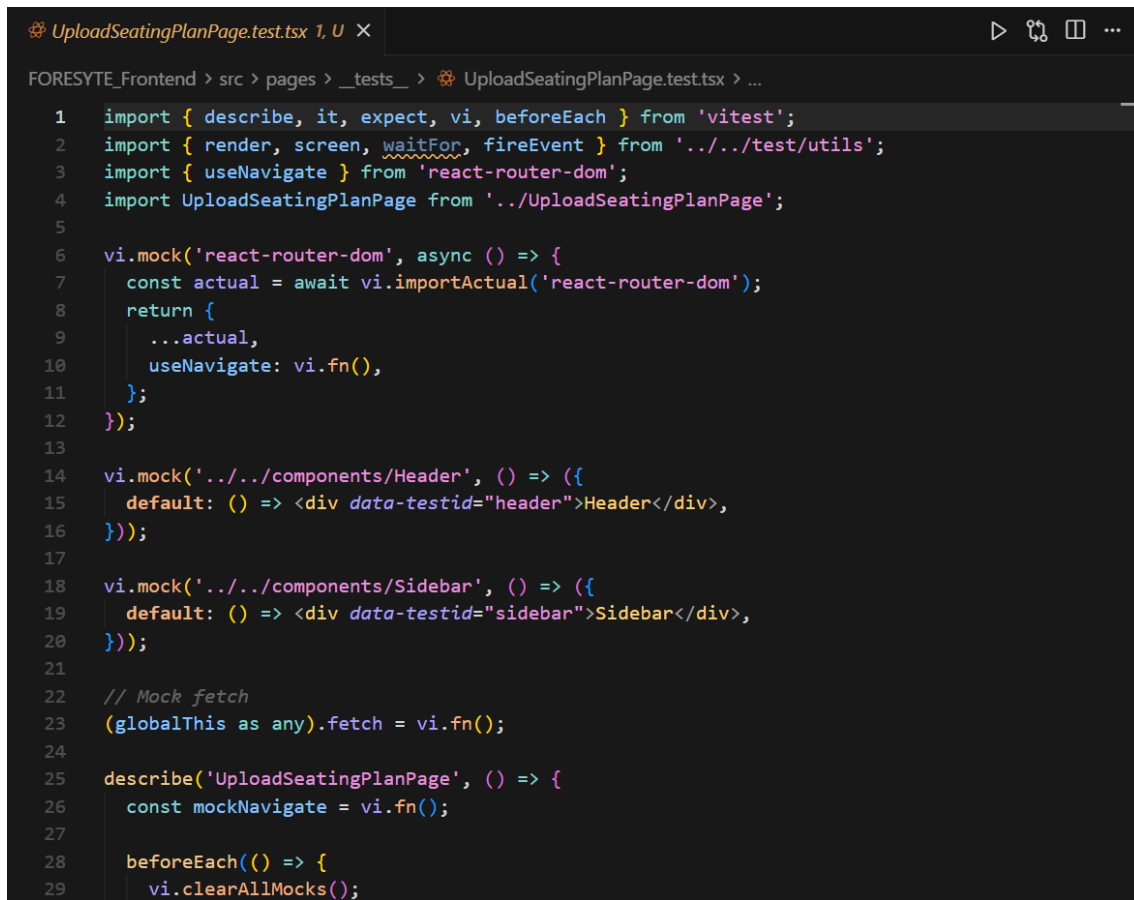


The screenshot shows a code editor with a file named `SeatingPlansPage.test.tsx` open. The editor's breadcrumb navigation shows the path: `FORESYTE_Frontend > src > pages > _tests_ > SeatingPlansPage.test.tsx > ...`. The code is written in TypeScript and uses Vitest for testing. It includes imports for `describe`, `it`, `expect`, `vi`, `beforeEach` from `'vitest'`, and `render`, `screen`, `waitFor`, `fireEvent` from `'../../test/utils'`. It also imports `SeatingPlansPage` from `'../SeatingPlansPage'` and `apiService` from `'../../test/utils'`. The code defines two mocks: one for `Header` and one for `Sidebar`, both returning a `div` with a specific `data-testid`. A `describe` block for `'SeatingPlansPage'` contains a `beforeEach` hook that clears mocks and sets session storage. The main test is an `it` block that renders the `SeatingPlansPage` and expects the `header` and `sidebar` to be in the document.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../../test/utils';
3 import SeatingPlansPage from '../SeatingPlansPage';
4 import { apiService } from '../../test/utils';
5
6 vi.mock('../../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 describe('SeatingPlansPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders seating plans page with header and sidebar', () => {
22     (apiService.getSeatingPlans as any).mockResolvedValue({
23       data: { plans: [], total: 0, page: 1, limit: 20 },
24     });
25
26     render(<SeatingPlansPage />);
27
28     expect(screen.getByTestId('header')).toBeInTheDocument();
29     expect(screen.getByTestId('sidebar')).toBeInTheDocument();
30   });
31 }
```

A.1.1.11 Upload Seating Plan Page

Figure A.11: Admin Portal Upload Seating Plan Page

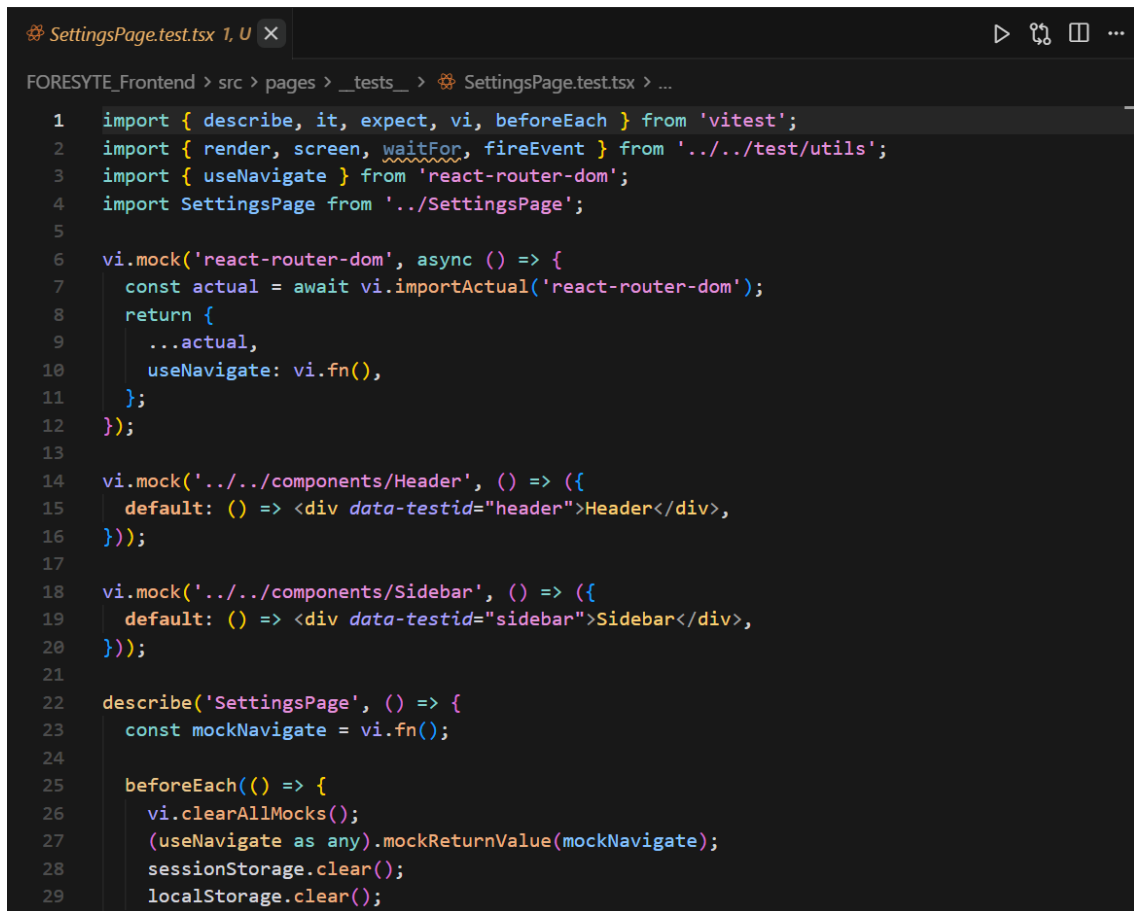


The screenshot shows a code editor with a file named `UploadSeatingPlanPage.test.tsx` open. The editor's breadcrumb navigation shows the path: `FORESYTE_Frontend > src > pages > _tests_ > UploadSeatingPlanPage.test.tsx > ...`. The code is written in TypeScript and uses Vitest for testing. It includes imports for `describe`, `it`, `expect`, `vi`, and `beforeEach` from `'vitest'`, and `render`, `screen`, `waitFor`, and `fireEvent` from `'../../test/utils'`. It also imports `useNavigate` from `'react-router-dom'` and the component `UploadSeatingPlanPage` from `'../UploadSeatingPlanPage'`. The code uses `vi.mock` to mock `react-router-dom`, the `Header` component, and the `Sidebar` component. It also mocks the `fetch` method on `globalThis`. The test suite is defined with `describe('UploadSeatingPlanPage', () => {` and includes a `beforeEach` hook that calls `vi.clearAllMocks()`.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../../test/utils';
3 import { useNavigate } from 'react-router-dom';
4 import UploadSeatingPlanPage from '../UploadSeatingPlanPage';
5
6 vi.mock('react-router-dom', async () => {
7   const actual = await vi.importActual('react-router-dom');
8   return {
9     ...actual,
10    useNavigate: vi.fn(),
11  };
12 });
13
14 vi.mock('../../components/Header', () => ({
15   default: () => <div data-testid="header">Header</div>,
16 }));
17
18 vi.mock('../../components/Sidebar', () => ({
19   default: () => <div data-testid="sidebar">Sidebar</div>,
20 }));
21
22 // Mock fetch
23 (globalThis as any).fetch = vi.fn();
24
25 describe('UploadSeatingPlanPage', () => {
26   const mockNavigate = vi.fn();
27
28   beforeEach(() => {
29     vi.clearAllMocks();
```

A.1.1.12 Settings Page

Figure A.12: Admin Portal Settings Page



```
SettingsPage.test.tsx 1, U X
FORESYTE_Frontend > src > pages > __tests__ > SettingsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utlis';
3  import { useNavigate } from 'react-router-dom';
4  import SettingsPage from '../SettingsPage';
5
6  vi.mock('react-router-dom', async () => {
7    const actual = await vi.importActual('react-router-dom');
8    return {
9      ...actual,
10     useNavigate: vi.fn(),
11   };
12 });
13
14 vi.mock('../components/Header', () => ({
15   default: () => <div data-testid="header">Header</div>,
16 }));
17
18 vi.mock('../components/Sidebar', () => ({
19   default: () => <div data-testid="sidebar">Sidebar</div>,
20 }));
21
22 describe('SettingsPage', () => {
23   const mockNavigate = vi.fn();
24
25   beforeEach(() => {
26     vi.clearAllMocks();
27     (useNavigate as any).mockReturnValue(mockNavigate);
28     sessionStorage.clear();
29     localStorage.clear();
```

A.1.1.13 Test Results

Figure A.13: Admin Portal Unit Test Results

```
This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

✓ src/pages/__tests__/AnalyticsPage.test.tsx (6) 486ms
✓ src/pages/__tests__/DashboardPage.test.tsx (6) 584ms
✓ src/pages/__tests__/ExamsPage.test.tsx (6) 597ms
✓ src/pages/__tests__/Incidents.test.tsx (5) 365ms
✓ src/pages/__tests__/LoginPage.test.tsx (7) 956ms
✓ src/pages/__tests__/MonitoringPage.test.tsx (5) 555ms
✓ src/pages/__tests__/ReportsPage.test.tsx (4) 611ms
✓ src/pages/__tests__/SeatingPlansPage.test.tsx (4) 571ms
✓ src/pages/__tests__/SettingsPage.test.tsx (7) 747ms
✓ src/pages/__tests__/SignUpPage.test.tsx (8) 1363ms
✓ src/pages/__tests__/UploadSeatingPlanPage.test.tsx (5) 1339ms
✓ src/pages/__tests__/UsersPage.test.tsx (8) 895ms

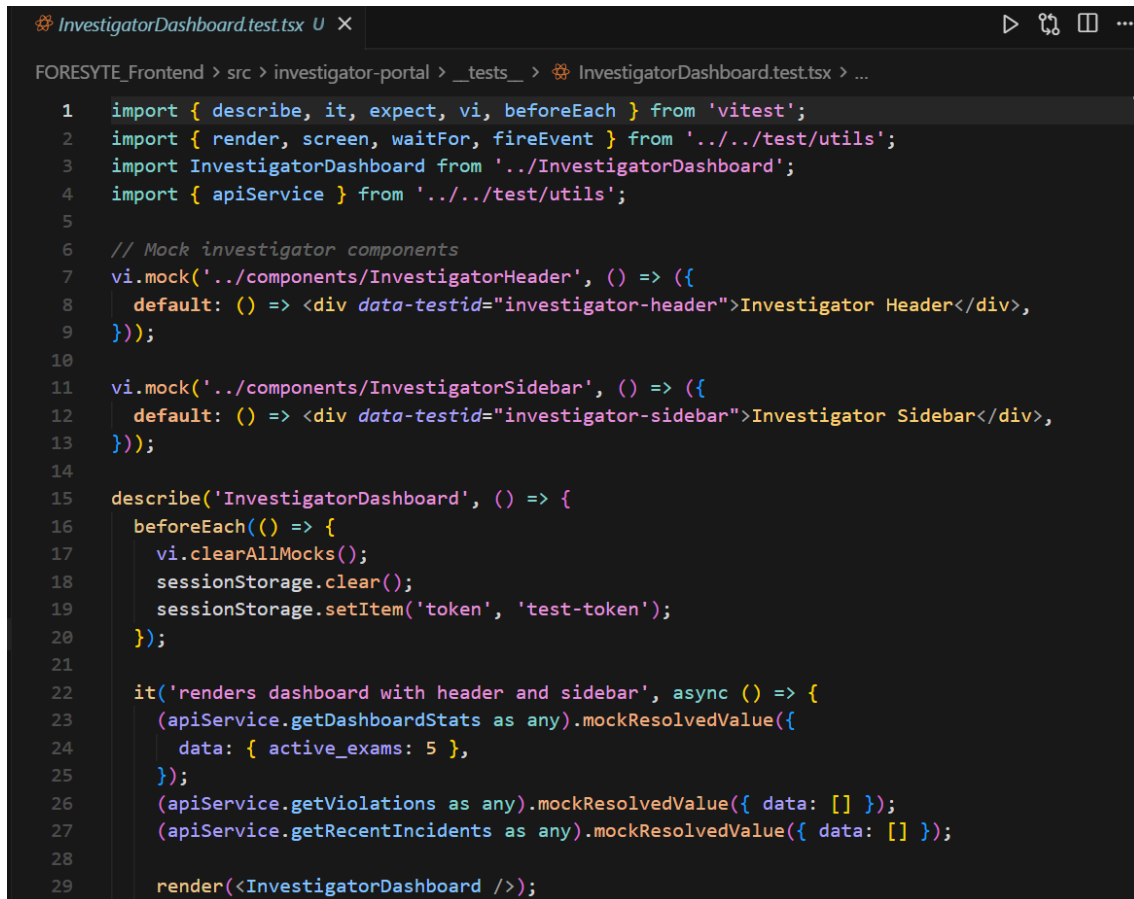
Test Files 12 passed (12)
Tests 71 passed (71)
Start at 04:33:35
Duration 9.82s (transform 1.17s, setup 4.18s, collect 4.44s, tests 9.07s, environment 11.99s, prepare 2.47s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

A.1.2 Investigator Portal Screenshots

A.1.2.1 Investigator Dashboard

Figure A.14: Investigator Portal Dashboard

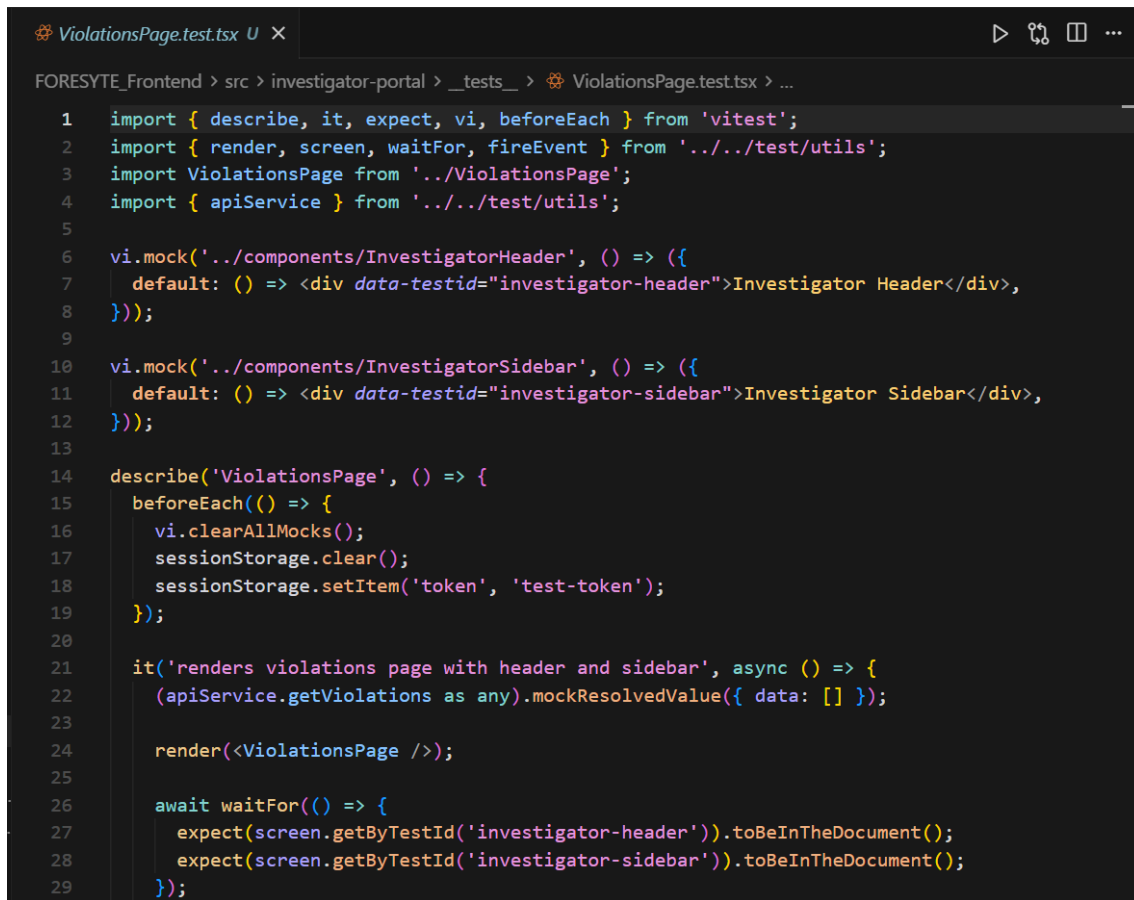


```
InvestigatorDashboard.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > InvestigatorDashboard.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import InvestigatorDashboard from '../InvestigatorDashboard';
4  import { apiService } from '../test/utils';
5
6  // Mock investigator components
7  vi.mock('../components/InvestigatorHeader', () => ({
8    default: () => <div data-testid="investigator-header">Investigator Header</div>,
9  }));
10
11 vi.mock('../components/InvestigatorSidebar', () => ({
12   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
13 });
14
15 describe('InvestigatorDashboard', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20   });
21
22   it('renders dashboard with header and sidebar', async () => {
23     (apiService.getDashboardStats as any).mockResolvedValue({
24       data: { active_exams: 5 },
25     });
26     (apiService.getViolations as any).mockResolvedValue({ data: [] });
27     (apiService.getRecentIncidents as any).mockResolvedValue({ data: [] });
28
29     render(<InvestigatorDashboard />);
```

A.1.2.2 Violations Page

Figure A.15: Investigator Portal Violations Management Page



The image shows a code editor window with a dark theme. The title bar at the top reads "ViolationsPage.test.tsx U X". Below the title bar, the breadcrumb path is "FORESYTE_Frontend > src > investigator-portal > __tests__ > ViolationsPage.test.tsx > ...". The code is written in TypeScript and uses Jest and React Testing Library for testing. It includes imports for testing utilities, mocks for the InvestigatorHeader and InvestigatorSidebar components, and a describe block for the ViolationsPage component. The test suite includes a beforeEach hook to clear mocks and session storage, and an it block to verify that the page renders with the correct header and sidebar.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utils';
3 import ViolationsPage from '../ViolationsPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/InvestigatorHeader', () => ({
7   default: () => <div data-testid="investigator-header">Investigator Header</div>,
8 }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 }));
13
14 describe('ViolationsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders violations page with header and sidebar', async () => {
22     (apiService.getViolations as any).mockResolvedValue({ data: [] });
23
24     render(<ViolationsPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.3 Invigilator Activities Page

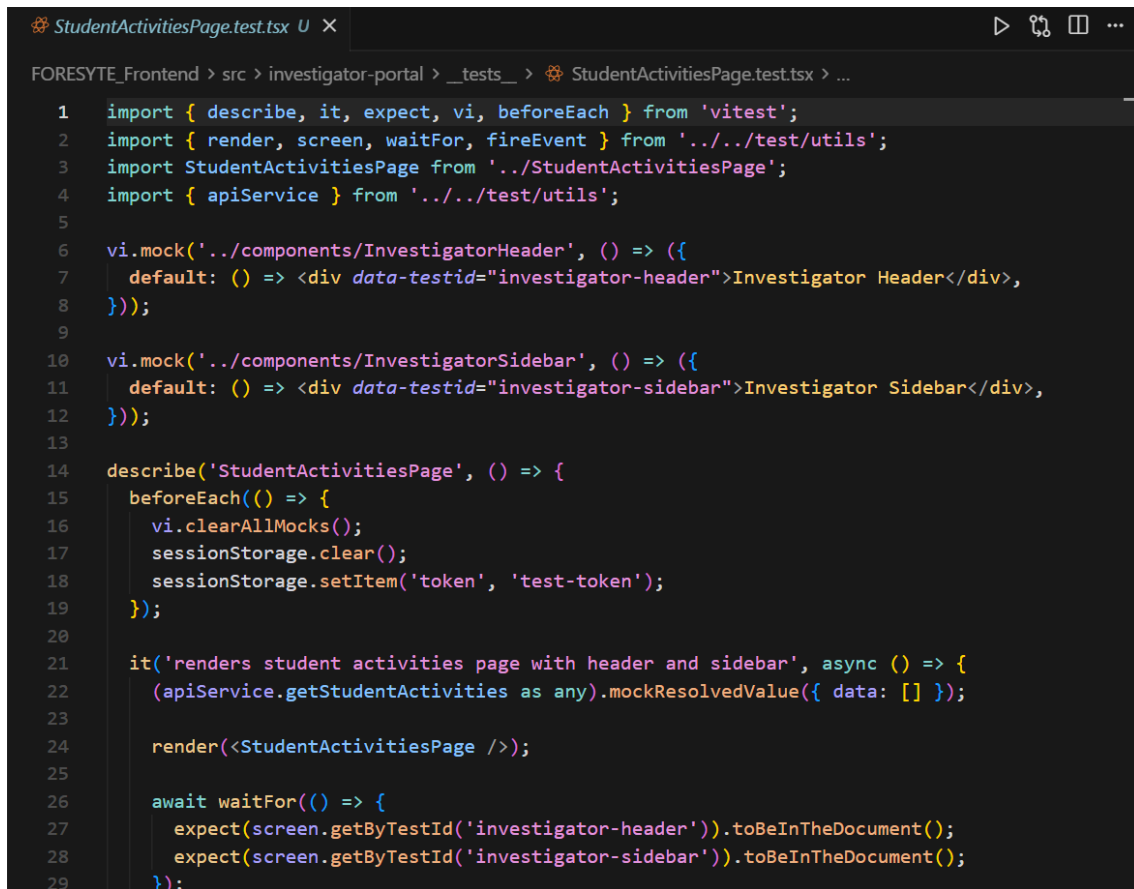
Figure A.16: Investigator Portal Invigilator Activities Page

```
InvigilatorActivitiesPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > InvigilatorActivitiesPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import InvigilatorActivitiesPage from '../InvigilatorActivitiesPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('InvigilatorActivitiesPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders invigilator activities page with header and sidebar', async () => {
22     (apiService.getInvigilatorActivities as any).mockResolvedValue({ data: [] });
23
24     render(<InvigilatorActivitiesPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.4 Student Activities Page

Figure A.17: Investigator Portal Student Activities Page



```
StudentActivitiesPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > StudentActivitiesPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentActivitiesPage from '../StudentActivitiesPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('StudentActivitiesPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders student activities page with header and sidebar', async () => {
22     (apiService.getStudentActivities as any).mockResolvedValue({ data: [] });
23
24     render(<StudentActivitiesPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.5 Reports Page

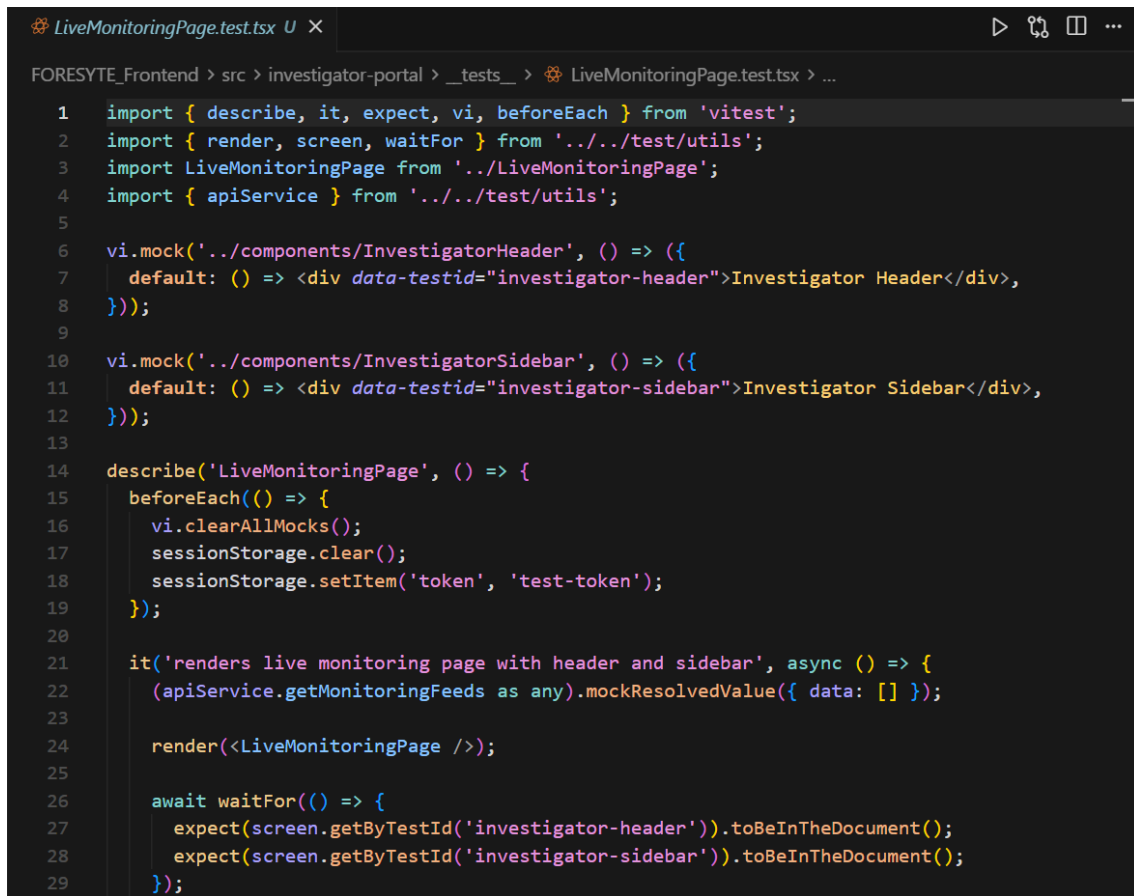
Figure A.18: Investigator Portal Reports Page

```
ReportsPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > ReportsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import ReportsPage from '../ReportsPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('ReportsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders reports page with header and sidebar', async () => {
22     (apiService.getReports as any).mockResolvedValue({ data: { reports: [] } });
23
24     render(<ReportsPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.6 Live Monitoring Page

Figure A.19: Investigator Portal Live Monitoring Page



```
LiveMonitoringPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > LiveMonitoringPage.test.tsx > ...

1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor } from '../test/utils';
3 import LiveMonitoringPage from '../LiveMonitoringPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/InvestigatorHeader', () => ({
7   default: () => <div data-testid="investigator-header">Investigator Header</div>,
8 }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 }));
13
14 describe('LiveMonitoringPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders live monitoring page with header and sidebar', async () => {
22     (apiService.getMonitoringFeeds as any).mockResolvedValue({ data: [] });
23
24     render(<LiveMonitoringPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.7 Video Processing Page

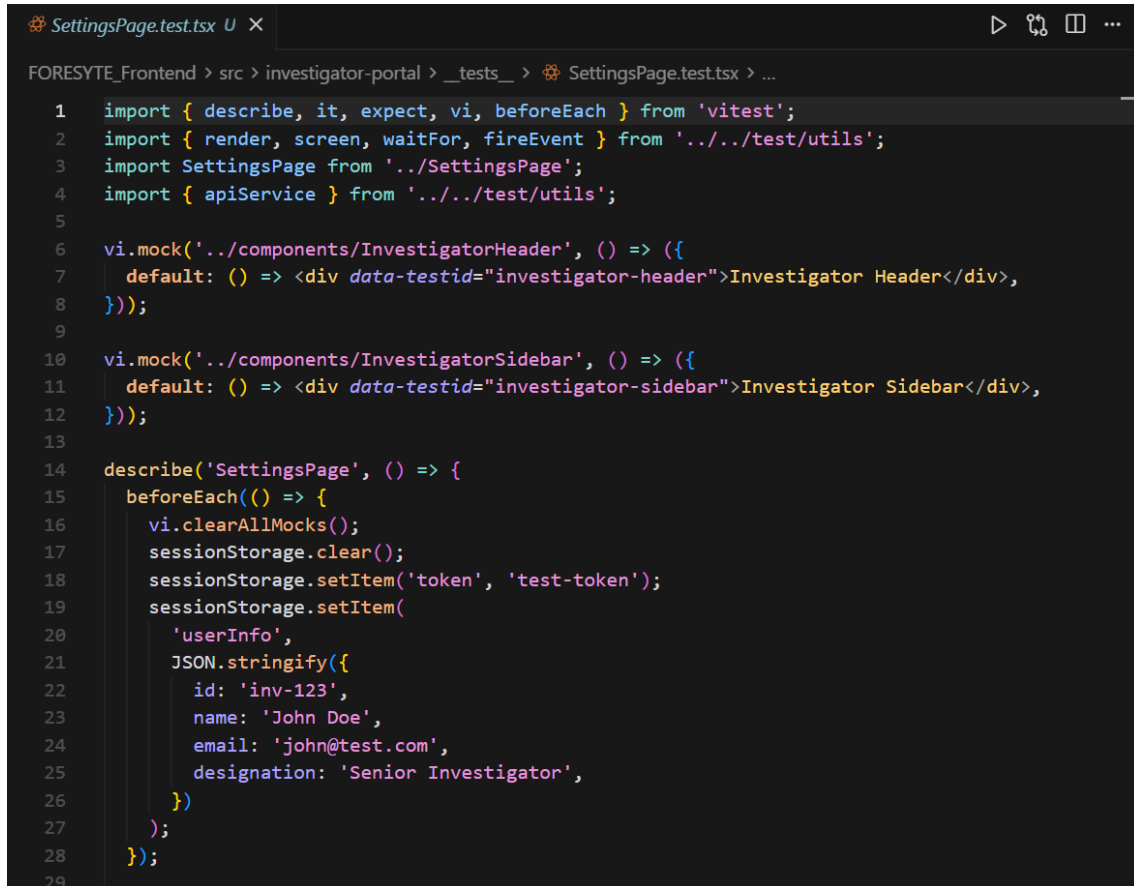
Figure A.20: Investigator Portal Video Processing Page

```
VideoProcessingPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > VideoProcessingPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor } from '../test/utils';
3  import VideoProcessingPage from '../VideoProcessingPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('VideoProcessingPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders video processing page with header and sidebar', async () => {
22     (apiService.getVideoStreams as any).mockResolvedValue({ data: [] });
23
24     render(<VideoProcessingPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.8 Settings Page

Figure A.21: Investigator Portal Settings Page

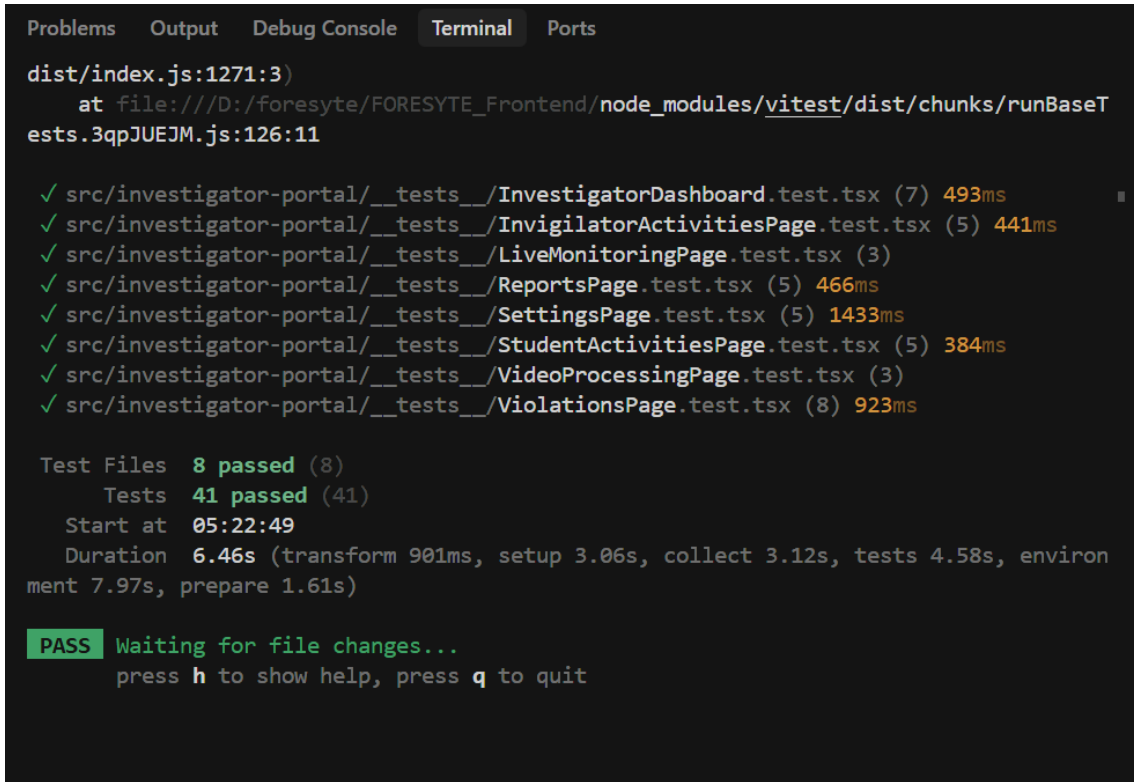


```
SettingsPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > SettingsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import SettingsPage from '../SettingsPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('SettingsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19     sessionStorage.setItem(
20       'userInfo',
21       JSON.stringify({
22         id: 'inv-123',
23         name: 'John Doe',
24         email: 'john@test.com',
25         designation: 'Senior Investigator',
26       })
27     );
28   });
29 }
```

A.1.2.9 Test Results

Figure A.22: Investigator Portal Unit Test Results



```
Problems  Output  Debug Console  Terminal  Ports

dist/index.js:1271:3)
  at file:///D:/foresyte/FORESYTE_Frontend/node_modules/vitest/dist/chunks/runBaseT
ests.3qpJUEJM.js:126:11

✓ src/investigator-portal/__tests__/InvestigatorDashboard.test.tsx (7) 493ms
✓ src/investigator-portal/__tests__/InvigilatorActivitiesPage.test.tsx (5) 441ms
✓ src/investigator-portal/__tests__/LiveMonitoringPage.test.tsx (3)
✓ src/investigator-portal/__tests__/ReportsPage.test.tsx (5) 466ms
✓ src/investigator-portal/__tests__/SettingsPage.test.tsx (5) 1433ms
✓ src/investigator-portal/__tests__/StudentActivitiesPage.test.tsx (5) 384ms
✓ src/investigator-portal/__tests__/VideoProcessingPage.test.tsx (3)
✓ src/investigator-portal/__tests__/ViolationsPage.test.tsx (8) 923ms

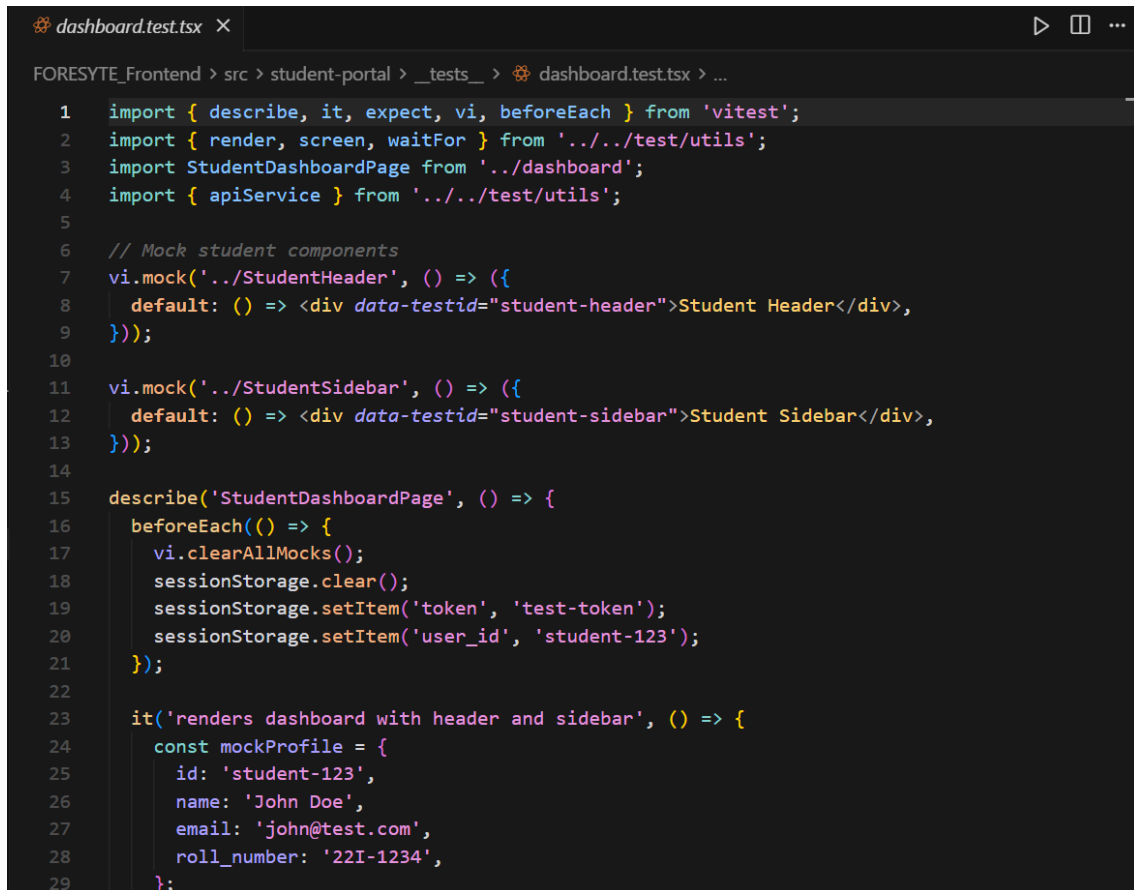
Test Files  8 passed (8)
Tests      41 passed (41)
Start at   05:22:49
Duration   6.46s (transform 901ms, setup 3.06s, collect 3.12s, tests 4.58s, environ
ment 7.97s, prepare 1.61s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

A.1.3 Student Portal Screenshots

A.1.3.1 Student Dashboard

Figure A.23: Student Portal Dashboard



```
dashboard.test.tsx X
FORESYTE_Frontend > src > student-portal > _tests_ > dashboard.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor } from '../test/utils';
3  import StudentDashboardPage from '../dashboard';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentDashboardPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     sessionStorage.setItem('user_id', 'student-123');
21   });
22
23   it('renders dashboard with header and sidebar', () => {
24     const mockProfile = {
25       id: 'student-123',
26       name: 'John Doe',
27       email: 'john@test.com',
28       roll_number: '22I-1234',
29     };
30   });
31 });
```

A.1.3.2 Disciplinary Records Page

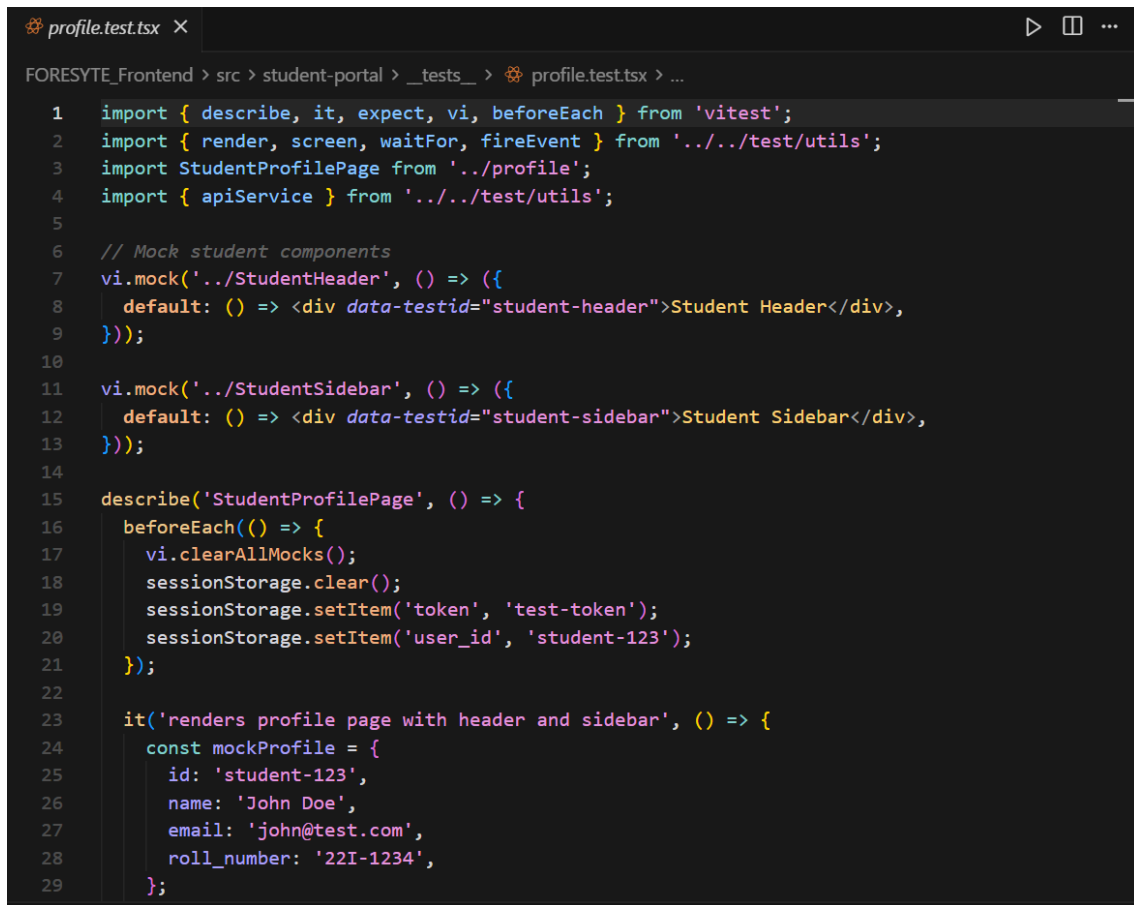
Figure A.24: Student Portal Disciplinary Records Page

```
dcRecords.test.tsx M X
FORESYTE_Frontend > src > student-portal > __tests__ > dcRecords.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentDisciplinaryRecordsPage from '../dcRecords';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentDisciplinaryRecordsPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     sessionStorage.setItem('user_id', 'student-123');
21   });
22
23   it('renders disciplinary records page with header and sidebar', () => {
24     const mockProfile = { id: 'student-123', name: 'John Doe', email: 'john@test.com' };
25     (apiService.getStudentProfile as any).mockResolvedValue({ data: mockProfile });
26     (apiService.getStudentViolations as any).mockResolvedValue({ data: [] });
27
28     render(<StudentDisciplinaryRecordsPage />);
29   });
30 });
```

A.1.3.3 Profile Page

Figure A.25: Student Portal Profile Page

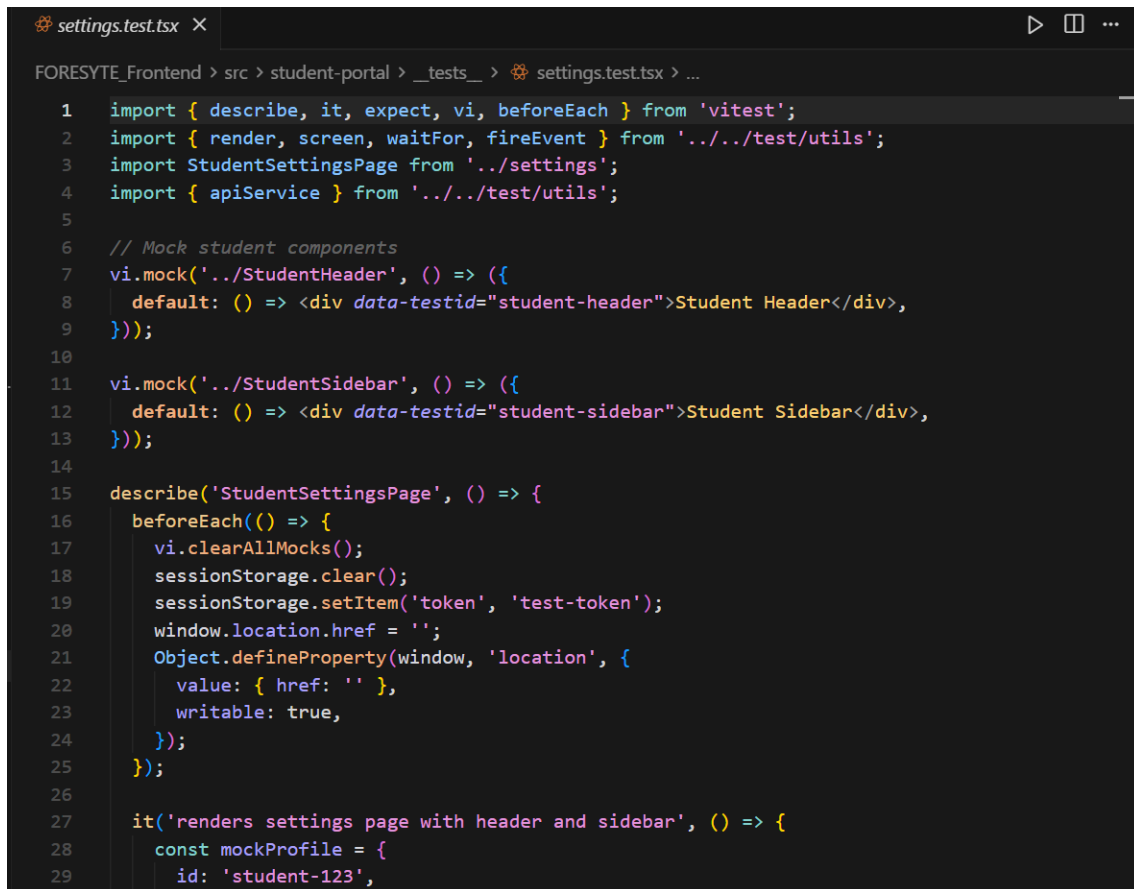


The image shows a code editor window with a file named `profile.test.tsx`. The breadcrumb navigation indicates the file is located at `FORESYTE_Frontend > src > student-portal > __tests__ > profile.test.tsx > ...`. The code is a Jest test for the `StudentProfilePage` component, using `vitest` and `vi` for mocking and assertions. It includes imports for testing utilities, mocks for `StudentHeader` and `StudentSidebar`, and a `describe` block for the `StudentProfilePage` component. Inside the `describe` block, there is a `beforeEach` hook that clears mocks and session storage, and an `it` block that renders the profile page with a specific mock profile.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../../test/utils';
3 import StudentProfilePage from '../profile';
4 import { apiService } from '../../test/utils';
5
6 // Mock student components
7 vi.mock('../StudentHeader', () => ({
8   default: () => <div data-testid="student-header">Student Header</div>,
9 }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 }));
14
15 describe('StudentProfilePage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     sessionStorage.setItem('user_id', 'student-123');
21   });
22
23   it('renders profile page with header and sidebar', () => {
24     const mockProfile = {
25       id: 'student-123',
26       name: 'John Doe',
27       email: 'john@test.com',
28       roll_number: '22I-1234',
29     };
30   });
31 });
```

A.1.3.4 Settings Page

Figure A.26: Student Portal Settings Page



```
settings.test.tsx X
FORESYTE_Frontend > src > student-portal > _tests_ > settings.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentSettingsPage from '../settings';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentSettingsPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     window.location.href = '';
21     Object.defineProperty(window, 'location', {
22       value: { href: '' },
23       writable: true,
24     });
25   });
26
27   it('renders settings page with header and sidebar', () => {
28     const mockProfile = {
29       id: 'student-123',
```

A.1.3.5 Test Results

Figure A.27: Student Portal Unit Test Results

```
✓ src/student-portal/__tests__/dashboard.test.tsx (9) 467ms
✓ src/student-portal/__tests__/dcRecords.test.tsx (9) 949ms
✓ src/student-portal/__tests__/profile.test.tsx (7) 1021ms
✓ src/student-portal/__tests__/settings.test.tsx (11) 1617ms
✓ src/student-portal/__tests__/uploadImage.test.tsx (9) 317ms

Test Files 5 passed (5)
  Tests 45 passed (45)
  Start at 05:01:47
  Duration 5.36s (transform 787ms, setup 1.66s, collect 2.11s, tests 4.37s, environ
ment 5.15s, prepare 1.00s)

PASS Waiting for file changes...
  press h to show help, press q to quit
```